

Payments

REST API

Visa Platform Connect



Cybersource Contact Information

For general information about our company, products, and services, go to <https://www.cybersource.com>.

For sales questions about any Cybersource service, email sales@cybersource.com or call 650-432-7350 or 888-330-2300 (toll free in the United States).

For support information about any Cybersource service, visit the Support Center: <https://www.cybersource.com/support>

Copyright

© 2020. Cybersource Corporation. All rights reserved. Cybersource Corporation ("Cybersource") furnishes this document and the software described in this document under the applicable agreement between the reader of this document ("You") and Cybersource ("Agreement"). You may use this document and/or software only in accordance with the terms of the Agreement. Except as expressly set forth in the Agreement, the information contained in this document is subject to change without notice and therefore should not be interpreted in any way as a guarantee or warranty by Cybersource. Cybersource assumes no responsibility or liability for any errors that may appear in this document. The copyrighted software that accompanies this document is licensed to You for use only in strict accordance with the Agreement. You should read the Agreement carefully before using the software. Except as permitted by the Agreement, You may not reproduce any part of this document, store this document in a retrieval system, or transmit this document, in any form or by any means, electronic, mechanical, recording, or otherwise, without the prior written consent of Cybersource.

Restricted Rights Legends

For Government or defense agencies: Use, duplication, or disclosure by the Government or defense agencies is subject to restrictions as set forth in the Rights in Technical Data and Computer Software clause at DFARS 252.227-7013 and in similar clauses in the FAR and NASA FAR Supplement.

For civilian agencies: Use, reproduction, or disclosure is subject to restrictions set forth in subparagraphs (a) through (d) of the Commercial Computer Software Restricted Rights clause at 52.227-19 and the limitations set forth in Cybersource Corporation's standard commercial agreement for this software. Unpublished rights reserved under the copyright laws of the United States.

Trademarks

Authorize.Net, eCheck.Net, and The Power of Payment are registered trademarks of Cybersource Corporation. Cybersource, Cybersource Payment Manager, Cybersource Risk Manager, Cybersource Decision Manager, and Cybersource Connect are trademarks and/or service marks of Cybersource Corporation. Visa, Visa International, Cybersource, the Visa logo, and the Cybersource logo are the registered trademarks of Visa International in the United States and other countries. All other trademarks, service marks, registered marks, or registered service marks are the property of their respective owners.

Confidentiality Notice

This document is furnished to you solely in your capacity as a client of Cybersource and as a participant in the Visa payments system.

By accepting this document, you acknowledge that the information contained herein (the "Information") is confidential and subject to the confidentiality restrictions contained in Visa's operating regulations and/or other confidentiality agreements, which limit our use of the Information. You agree to keep the Information confidential and not to use the Information for any purpose other than its intended purpose and in your capacity as a customer of Cybersource or as a participant in the Visa payments system. The Information may only be disseminated within your organization on a need-to-know basis to enable your participation in the Visa payments system. Please be advised that the Information may constitute material non-public information under U.S. federal securities laws and that purchasing or selling securities of Visa Inc. while being aware of material non-public information would constitute a violation of applicable U.S. federal securities laws.

Revision

Version: 23.01

Contents

Recent Revisions to This Document.....	9
About This Guide.....	10
Introduction to Payments.....	11
Payment Services.....	11
Financial Institutions and Payment Networks.....	11
Merchant Financial Institutions (Acquirers).....	11
Issuing (Customer) Financial Institutions.....	12
Payment Networks.....	12
Payment Processors.....	13
Payment Types.....	13
Co-Badged Cards.....	13
Co-Branded Cards.....	14
Credit Cards.....	14
Debit Cards.....	14
Prepaid Cards.....	14
Private Label Cards.....	14
Quasi-Cash.....	14
Transaction Types.....	14
Card-Not-Present Transactions.....	14
Card-Present Transactions.....	15
Authorizations with Card Verification Numbers.....	15
Token Management Service.....	16
Payment Processing.....	17
Authorizations.....	17
Sales.....	19
Authorization Reversals.....	19
Credits.....	20
Capture or Credit Voids.....	21
Capture.....	22
Basic Processing.....	23
Basic Authorizations.....	23
Required Fields for Processing Basic Authorizations Using the REST API.....	23

Declined Authorizations.....	24
REST Interactive Example: Processing an Authorization.....	25
REST Example: Processing a Basic Authorization.....	25
Authorizing Payments Using the Business Center.....	28
Authorization with Line Itemization.....	29
Required Fields.....	30
Country Specific Required Fields.....	31
REST Example: Processing a Basic Authorization with Line Items.....	31
Incremental Authorization.....	34
Required Fields for Processing Incremental Authorizations Using REST APIs.....	34
Optional Fields for Processing Incremental Authorizations Using REST APIs.....	35
Example: Processing an Incremental Authorization Using the REST API.....	35
Processing an Authorization with a Card Verification Number.....	37
Required Fields for Processing Basic Authorizations with a Card Verification Number Using the REST API.....	38
Optional Fields for an Authorization with a Card Verification Number.....	39
REST Example: Processing an Authorization with a CVN.....	39
Zero Amount Authorizations.....	41
Required Fields for Processing Zero Amount Authorizations Using the REST API.....	41
Country-Specific Required Fields for Processing Basic Zero Amount Authorizations Using the REST API.....	41
REST Example: Processing a Zero Amount Authorization.....	42
Authorization with an SCA Exemption.....	44
Required Fields for an Authorization with an SCA Exemption.....	46
REST Example: Authorization with an SCA Exemption for Low Value Transactions.....	46
Authorization Reversals.....	48
Required Fields for Processing Authorization Reversals Using REST API.....	49
Example: Processing an Authorization Reversal Using the REST API.....	49
Captures.....	50
Required Fields for Capturing a Payment Using the REST API.....	50
REST Interactive Example: Capturing a Payment.....	50
REST Example: Capturing an Authorization.....	50
Multiple Captures.....	51
Required Fields for Multiple Captures.....	51
REST Example: Processing Multiple Partial Captures.....	52
Credits.....	53
Required Fields for Processing a Credit Using the REST API.....	53
REST Interactive Example: Processing a Credit.....	54
REST Example: Processing a Credit.....	54
Refunds.....	55
Required Fields for Processing Refunds Using the REST API.....	55
REST Interactive Example: Processing a Refund.....	56
Example: Processing a Refund Using the REST API.....	56

Sales.....	57
Required Fields for Processing a Sale Using the REST API.....	58
REST Example: Requesting a Sale.....	58
Voids.....	60
Required Fields for Processing Voids Using REST APIs.....	60
REST Example: Processing a Void.....	60
Card-Present Authorizations.....	62
EMV Chip and Online PIN Authorization.....	62
Fallback.....	62
Required Fields for a Contact EMV Authorization with Online PIN Using the REST API.....	63
Country-Specific Required Fields for an EMV Credit Card Transaction Using the REST API.....	63
REST Example: EMV Chip and Online PIN Authorization.....	64
EMV Chip (Contact) and Offline PIN Authorization.....	65
Fallback.....	65
Required Fields for a Contact EMV Authorization with Offline PIN Using the REST API.....	66
Country-Specific Required Fields for an EMV Credit Card Transaction Using the REST API.....	66
REST Example: EMV Chip (Contact) with an Offline PIN.....	66
EMV Chip (Contact) and Signature.....	68
Fallback.....	68
Required Fields for a Contact EMV Authorization with a Signature Using the REST API.....	68
Country-Specific Required Fields for an EMV Credit Card Transaction Using the REST API.....	69
REST Example: EMV Chip (Contact) with a Signature.....	69
EMV Chip (Contactless) and Online PIN.....	71
Fallback.....	71
Required Fields for an EMV Contactless Authorization with an Online PIN Using the REST API.....	71
Country-Specific Required Fields for an EMV Credit Card Transaction Using the REST API.....	72
REST Example: EMV Chip (Contactless) with an Online PIN.....	72
EMV Chip (Contactless) and Offline PIN.....	73
Fallback.....	74
Required Fields for an EMV Contactless Authorization with an Offline PIN Using the REST API.....	74
Country-Specific Required Fields for an EMV Credit Card Transaction Using the REST API.....	75
REST Example: EMV Chip (Contactless) with an Offline PIN.....	75
EMV Chip (Contactless with a Signature.....	77
EMV Fallback.....	77
Required Fields for an EMV Contactless Authorization with a Signature Using the REST API.....	77

Country-Specific Required Fields for an EMV Credit Card Transaction Using the REST API.....	78
REST Example: EMV Chip (Contactless) with a Signature.....	78
Payer Authentication Processing.....	80
Visa Secure.....	80
Required Fields for an Authorization with Visa Secure.....	81
REST Example: Authorization with Visa Secure.....	82
American Express SafeKey.....	84
Required Fields for an Authorization with American Express SafeKey.....	85
Optional Field for an Authorization with American Express SafeKey.....	85
REST Example: Authorization with American Express SafeKey.....	86
JCB J/Secure.....	88
Required Fields for an Authorization using JCB J/Secure Authentication.....	88
REST Example: Authorization with JCB J/Secure.....	89
Mastercard Identity Check.....	91
Required Fields for an Authorization with Mastercard Identity Check.....	92
REST Example: Authorizing Mastercard Identity Check.....	93
Authorization with an SCA Exemption.....	94
Required Fields for an Authorization with an SCA Exemption.....	97
REST Example: Authorization with an SCA Exemption for Low Value Transactions.....	97
Stored Credential Processing.....	100
Customer-Initiated Transactions with Credentials on File.....	101
Storing Customer Credentials with a Customer-Initiated Transaction.....	102
Using Stored Customer Credentials During a Customer-Initiated Transaction.....	104
Merchant-Initiated Transactions.....	107
Industry Practice Transactions.....	107
Standing Instruction Transactions.....	122
Processing Token Management Service Payments.....	142
Authorize a Payment with a Customer Token.....	143
Required Fields for Authorizing a Payment with a Customer Token Using the REST API.....	143
REST Example: Authorizing a Payment with a Customer Token.....	144
REST Example: Authorizing a Payment Using a Customer Token Linked to a Network Token.....	146
Authorize a Payment with a Non-Default Shipping Address.....	148
Required Fields for Authorizing a Payment with a Non-Default Shipping Address Using the REST API.....	148
Example: Authorize a Payment with a Non-Default Shipping Address Using the REST API.....	149
Authorize a Payment with Non-Default Payment Instrument.....	151
Required Fields for Authorizing a Payment with Non-Default Payment Instrument Using the REST API.....	151
Optional Fields for Authorizing a Payment with Non-Default Payment Instrument Using the REST API.....	151

Example: Authorize a Payment with Non-Default Payment Instrument Using the REST API.....	152
Authorize a Payment with a Payment Instrument.....	153
Required Fields for Authorizing a Payment with a Payment Instrument Using the REST API.....	154
Optional Fields for Authorizing a Payment with a Payment Instrument Using the REST API.....	154
REST Example: Authorizing a Payment with a Payment Instrument.....	154
Authorize a Payment with an Instrument Identifier.....	156
Required Fields for Authorizing a Payment with an Instrument Identifier Using the REST API.....	156
Example: Authorize a Payment with an Instrument Identifier Using the REST API.....	157
Authorize a Payment with Legacy Token.....	159
Required Fields for Authorizing a Payment with Legacy Token Using the REST API.....	159
Example: Authorize a Payment with Legacy Token Using the REST API.....	159
Make a Credit with a Customer Token.....	161
Required Fields for Making a Credit with a Customer Token Using the REST API.....	161
Example: Make a Credit with a Customer Token Using the REST API.....	162
Make a Credit with Non-Default Payment Instrument.....	163
Required Fields for Making a Credit with Non-Default Payment Instrument Using the REST API.....	164
Optional Fields for Making a Credit with Non-Default Payment Instrument Using the REST API.....	164
Example: Make a Credit with Non-Default Payment Instrument Using the REST API.....	164
Make a Credit with a Payment Instrument.....	166
Required Fields for Making a Credit with a Payment Instrument Using the REST API.....	166
REST Example: Making a Credit with a Payment Instrument.....	167
Make a Credit with an Instrument Identifier.....	168
Required Fields for Making a Credit with an Instrument Identifier Using the REST API.....	169
Example: Make a Credit with an Instrument Identifier Using the REST API.....	169
Make a Credit with Legacy Token.....	171
Required Fields for Making a Credit with Legacy Token Using the REST API.....	171
Example: Make a Credit with Legacy Token Using the REST API.....	171
Debit and Prepaid Card Processing.....	174
Processing Debit and Prepaid Authorizations.....	174
Required Fields for Debit and Prepaid Authorizations Using REST APIs.....	174
Optional Field for Debit and Prepaid Authorizations Using REST APIs.....	176
REST Example: Debit and Prepaid Authorizations.....	176
Enabling Debit and Prepaid Partial Authorizations.....	178

Required Field for Enabling Debit and Prepaid Partial Authorizations Using REST APIs.....	178
Optional Field for Enabling Debit and Prepaid Partial Authorizations Using REST APIs.....	179
Enabling Partial Authorizations.....	179
Example: Enabling Debit and Prepaid Partial Authorizations Using the REST API.....	179
Disabling Debit and Prepaid Partial Authorizations.....	181
Required Field for Disabling Debit and Prepaid Partial Authorizations Using REST APIs.....	181
Optional Field for Disabling Debit and Prepaid Partial Authorizations Using REST APIs.....	181
Disabling Partial Authorizations.....	181
Example: Disabling Debit and Prepaid Partial Authorizations Using the REST API.....	181

Recent Revisions to This Document

23.01

Multiple Partial Captures

Added the section [Multiple Captures](#) on page 51.

Authorizations with a Card Verification Number

Added the section [Authorizations with Card Verification Numbers](#) on page 15.

22.02

Authorization with an SCA Exemption Use Case

Added an Authorization with an SCA exemption use case to the Basic Authorization section of the Payment Guide. See [Authorization with an SCA Exemption](#) on page 44.

Basic Authorization Reversals Use Case

Added a basic authorization reversal use case. See [Authorization Reversals](#) on page 48.

22.01

New Payment Guide

This new payment guide, replaces the now deprecated *Credit Card Guide* is re-written to better align with customer requirements.

About This Guide

This section describes the audience and purpose of this guide as well as conventions and related documentation. See below for information about how to use this guide and where to find further information.

Audience and Purpose

This guide is written for application developers who want to use the REST API to integrate payment card processing into an order management system.

Implementing the credit card services requires software development skills. You must write code that uses the API request and response fields to integrate the credit card services into your existing order management system.

Conventions

These special statements are used in this document:



Important

An Important statement contains information essential to successfully completing a task or learning a concept.



Warning

A Warning contains information or instructions, which, if not heeded, can result in a security risk, irreversible loss of data, or significant cost in time or revenue or both.

Related Documentation

Refer to the Technical Documentation Portal for complete technical documentation:

<https://docs.cybersource.com/en/index.html>

Customer Support

For support information about any service, visit the Support Center:

<http://www.cybersource.com/support>

Introduction to Payments

This introduction provides the basic information that you will need to successfully process payment transactions. It also provides an overview of the payments industry and provides workflows for each process.

Payment Services

With Cybersource payment services, you can process payment cards (tokenized or non-tokenized), digital payments, such as Apple Pay and Google Pay, and customer ID transactions. You can process payments across the globe and across multiple channels with scalability and security. Cybersource supports a large number of payment cards and offers a wide choice of gateways and financial institutions, all through one connection.

Financial Institutions and Payment Networks

Financial institutions and payment networks enables payment services. These entities work together and complete the full payment cycle.

Merchant Financial Institutions (Acquirers)

A merchant financial institution, also known as an *acquirer*, offers accounts to businesses that accept payment cards. Before you can accept payments, you must have a merchant account from an acquirer. Your merchant account must be configured to process card-not-present, card-present, or mail-order/telephone-order (MOTO) transactions.

Each acquirer has connections to a limited number of payment processors. You must choose a payment processor that your acquirer supports.

You can expect your acquirer to charge these fees:

- Discount rates—your acquirer charges a fee and collects a percentage of every transaction. The combination of the fee and the percentage is called the discount rate.

These charges can be *bundled* (combined into a single charge) or *unbundled* (charged separately).

- Interchange fees—payment networks, such as Visa or Mastercard, each have a base fee, called the interchange fee, for each type of transaction. Your acquirer and processor can show you ways to reduce this fee.
- Chargebacks—when cardholders dispute charges, you can incur chargebacks. A chargeback occurs when a charge on a customer's account is reversed. Your acquirer removes the money from your account and could charge you a fee for processing the chargeback.

Take these precautions to prevent chargebacks:

- Use accurate merchant descriptors, so that customers can recognize the transactions on their statements.
- Provide good customer support.
- Ensure rapid problem resolution.
- Maintain a high level of customer satisfaction.
- Minimize fraudulent transactions.

If excessive chargebacks or fraudulent changes occur, these actions may be taken:

- You may be required to change your business processes to reduce the number chargebacks and/or fraud.
- Your acquiring institution might increase your discount rate.
- Your acquiring institution might revoke your merchant account.

Contact your sales representative for information about products that can help prevent fraud.

Issuing (Customer) Financial Institutions

An issuing (customer) financial institution, also known as an issuer, provides payment cards to and underwrites lines of credit for their customers. The issuer provides monthly statements and collects payments. The issuer must follow the rules of the payment card companies to which they belong.

Payment Networks

Payment networks manage communications between acquiring financial institutions and issuing financial institutions. They also develop industry standards, support their brands, and establish fees for acquiring institutions.

Some payment networks, such as Visa and Mastercard, are trade associations that do not issue cards. Issuers are members of these associations, and they issue cards under license from the association.

Other networks, such as Discover and American Express, issue their own cards. Before you process cards from these companies, you must sign agreements with them.

Payment Processors

Payment processors connect with acquirers. Before you can accept payments, you must register with a payment processor. An acquirer may require you to use a payment processor with an existing relationship with the acquirer.

Your payment processor assigns one or more merchant IDs (MIDs) to your business. These unique codes identify your business during payment transactions.

Payment Types

You can make payments with these payment types:

- Co-badged cards
- Co-branded cards
- Credit cards
- Debit cards
- Prepaid cards
- Private label cards
- Quasi-cash

Co-Badged Cards

Co-badged cards are credit cards that integrate two or more payment networks.

mada Co-Badged Cards

mada is Saudi Arabia's domestic payment network.

These mada co-badged debit cards are supported:

- Visa and mada
- Mastercard and mada

mada co-badged debit cards are processed as follows:

- Only domestic Saudi Arabian processing is supported.
- Transactions are sent directly to the Saudi Arabia Monetary Authority (SAMA) for processing.
- Payer authentication is supported. Visa Secure is supported for co-badged Visa and mada cards. Mastercard Identity Check is supported for co-badged Mastercard and mada cards.
- For acquirers, the card type is identified as MD.
- In reports, the card type is identified as either Visa or Mastercard.
- Dual-message processing is not supported. Only single-message processing is supported.

Co-Branded Cards

Co-branded cards are credit cards that are branded with a merchant's logo, brand, or other identifier as well as the payment network logo. These cards are not limited for use at the branded merchant and can be used at any merchant that accepts credit cards.

Credit Cards

Cardholders use credit cards to borrow money from issuing banks to pay for goods and services offered by merchants that accept credit cards.

Debit Cards

A Debit card is linked to a cardholder's checking account. A merchant who accepts the debit card can deduct funds directly from the linked cardholder's account.

Prepaid Cards

Prepaid cards allow cardholders to pay for goods and services using money stored directly on the card.

Private Label Cards

Private label cards are issued by private companies. They enable cardholders to borrow money to pay for goods exclusively at the issuing company's stores.

Quasi-Cash

Quasi-cash transactions involve instruments that are directly convertible to cash such as web wallets, travelers checks, cryptocurrency, and lottery tickets.

Transaction Types

This topic provides information about card-present, card-not-present, and international transactions.

Card-Not-Present Transactions

When a customer provides a card number, but the card and the customer is not physically present at the merchant's location, the purchase is known as a *card-not-present* transaction. Typical card-not-present transactions are internet and phone transactions. Card-not-present transactions pose an additional level of risk to your business because the customer's identification cannot be verified. You can reduce that risk by using features such as, the Address Verification System (AVS) and Card Verification Numbers (CVNs). The AVS and CVNs provide additional protection from fraud. AVS validates the cardholder's entered address in relation to the known customer's address. CVN requires the cardholder to include the card's CVN number during a transaction.

Card-Present Transactions

When a customer uses a card that is physically present in a retail environment, the purchase is known as a *card-present transaction*.

Authorizations with Card Verification Numbers

Card verification numbers (CVNs) are a required feature for the authorization service. The CVN is printed on a payment card, and only the cardholder can access it. The CVN is used in card-not-present transactions as a verification feature. Using the CVN helps reduce the risk of fraud.

CVNs are not included in payment card track data and cannot be obtained from a card swipe, tap, or dip.

CVNs must not be stored after authorization.

Important

In Europe, Visa mandates that you not include a CVN for mail-order transactions and not record a CVN on any physical format such as a mail-order form.

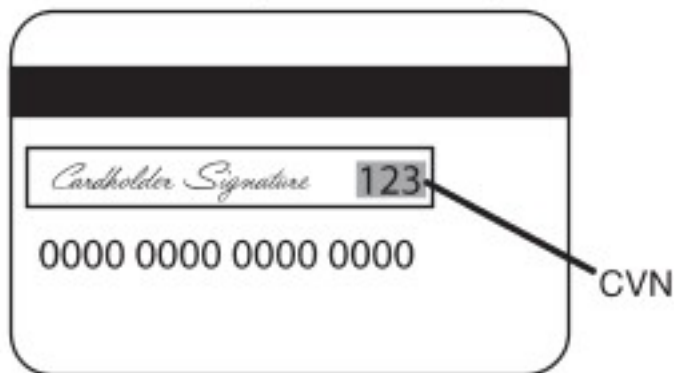
Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

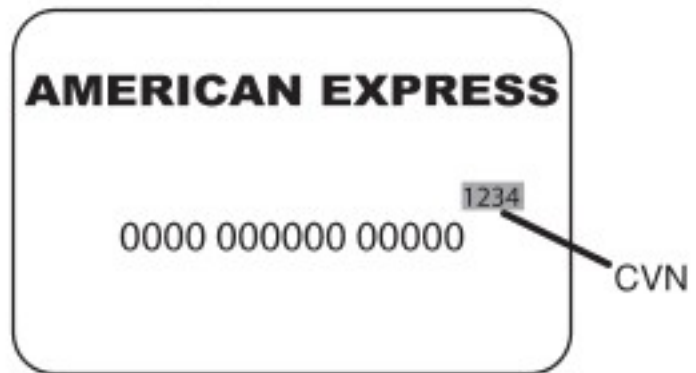
CVN Locations and Terminology

For most cards, the CVN is a three-digit number printed on the back of the card, to the right of the signature field. For American Express, the CVN is a four-digit number printed on the front of the card above the card number.

All Cards Except American Express



American Express Cards



CVN Locations

Each payment card company has its own name for the CVN value:

- American Express and Discover call it the Card Identification Number (CID).
- JCB calls it the Card Authentication Value (CAV2).
- Mastercard calls it the Card Validation Code (CVC2).
- Visa calls it the Card Verification Value (CVV2).

International Transactions

Consider these issues when processing international transactions.

Dynamic Currency Conversion

Dynamic Currency Conversion (DCC) is a regulated service from Mastercard and Visa. It converts the price of a transaction from a merchant's local currency to the customer's billing currency in real time.

Note that when you use DCC with an order, you cannot split the order into multiple shipments.

Merchant Remittance Funding

You can request that the transaction proceeds be converted to another currency.

Currency conversion uses a foreign exchange rate to calculate the conversion to the requested currency. The foreign exchange rate might be explicitly stated as a rate or implicitly stated as a transaction amount. The funded amount and can vary from day to day. The foreign exchange rate might also include a mark-up for the foreign exchange risk, sales commissions, and handling costs.

Token Management Service

The Token Management Service (TMS) tokenizes, securely stores, and manages customer and payment data. TMS enables you to:

- Securely store a customer's payment details and their billing and shipping addresses.
- Create a network token of a customer's payment card.

TMS simplifies your PCI DSS compliance. TMS passes back to you tokens that represent this data. You then store these tokens in your environment and databases instead of customer payment details.

TMS Token Types

- Customer — Stores the buyer's email address and the merchant's account ID for that buyer plus any other custom fields.
- Shipping Address — Stores a shipping address for a specific customer.
- Instrument Identifier — An Instrument Identifier object stores either:
 - Payment card number
 - [or] ACH bank account and routing number

This resource creates either:

- An Instrument Identifier token using details of a payment card or an ACH bank account.
- A payment network token using the details of a payment card; also uses the card expiration date and billing address, which are pass-through only fields.
- Payment Instrument — Stores a Payment Instrument using an Instrument Identifier token. It does not store the card number and cannot exist without an associated Instrument Identifier. It stores:
 - Card expiration date
 - Billing address

You can also choose to store this information yourself instead and store only the card number or bank account and routing number in an Instrument Identifier object.

- Customer Payment Instrument — Creates and stores a payment instrument for a specific customer ID and an Instrument Identifier token.

TMS Features

- Create, retrieve, update, and delete tokens.
- Set a default payment instrument and shipping address for a customer.
- Process follow-on payment transactions with token IDs.
- Create and update tokens through bundled payment transactions.

Payment Processing

The various services used to process payments are at the heart of payment processing. These services enable customers to purchase goods and services, merchants receive payments from the customer's accounts, merchants to provide refunds, and merchants to void transactions.

Authorizations

An authorization confirms that a payment card account holds enough funds to pay for a purchase. Authorizations fall into two categories: online and offline.

Online Authorizations

Online authorizations provide immediate funds availability confirmations. The customer's financial institution also reduces the amount of credit available in the customer's account, setting aside the authorized funds for the merchant to capture a later time. Authorizations for most payment cards are processed online. Typically, it is safe to start fulfilling the order once you receive an authorization confirmation.

An online authorization confirmation and the subsequent hold on funds expires after a specific length of time. Thus it is important to capture funds in a timely manner. The issuing bank sets the expiration time interval but most authorizations expire within five to seven days.

The issuing bank does not inform Cybersource when an authorization confirmation expires. By default, the authorization information for each transaction remains in the Cybersource database for 60 days after the authorization date. To capture an authorization that expired with the issuing bank, you can resubmit the authorization request.

Offline Authorizations

Online transactions require an internet connection. In situations where the internet is not available, for example, due to an outage, merchants can continue to take credit card payments using offline transactions. An offline authorization is an authorization request for which you do not receive an immediate confirmation about the availability of funds.

Offline authorizations have a higher level of risk than online transactions because they do not provide fund availability confirmations or set aside the funds for later capture. Further, it can take up to five days to receive payment confirmations for offline transactions. To mitigate this risk, merchants may choose to fulfill orders only after receiving payment confirmations.

Incremental Authorizations

Incremental authorizations are useful when a customer adds products and services to a purchase. After a successful initial authorization, you can request subsequent authorizations and request one capture for the initial authorization and the incremental authorizations.

The incremental authorization service is not the same as the incremental authorization scenario for a merchant-initiated transaction.

Example

1. The customer reserves a hotel room for two nights at a cost of 200.00 per night. You request an authorization for 400.00. The authorization request is approved.
2. The customer orders dinner through room service the first night. You request an incremental authorization of 50.00 for the dinner.
3. The customer decides to stay an extra night. You request an incremental authorization of 200.00 for the additional night.
4. The customer uses items from the mini-bar. The cost of the mini-bar items is 50.00. You request an incremental authorization of 50.00.
5. When the customer checks out, they sign a receipt for 700.00, which is the total of all costs incurred.
6. You request a capture for 700.00.

Payment Network Tokens Authorizations

You can integrate authorizations with payment network tokens into your existing order management system. For an incremental authorization, you do not need to include any payment network tokenization fields in the authorization request because Cybersource obtains the payment network tokenization information from the original authorization request.

Authorization Workflow



1. The customer purchases goods or service from the merchant using a payment card.
2. The merchant sends an authorization request to the acquiring (merchant) bank.
3. The acquiring (merchant) bank forwards the authorization request to the payment network.

4. The payment network forwards the authorization request to the issuer (customer) bank.
5. If funds are available, the issuer (customer) bank reserves the amount of the authorization request and returns an authorization approval to the payment network. If the issuer (customer) bank denies the request, it returns an authorization denial.
6. The payment network forwards the message to the acquiring (merchant) bank.
7. The acquiring (merchant) bank forwards the message to the merchant.

Sales

A sale is a bundled authorization and capture. Some processors and acquirers require a sale transaction instead of using separate authorization and capture requests. For other processors and acquirers, you can request a sale instead of a separate authorization and capture when you provide the goods or services immediately after taking an order. There are two types of sale processing: Dual-Message Processing and Single-Message Processing.

Dual Message Processing

Dual message processing, as the name indicates, is a two-step process. The authorization is processed first. If the authorization is successful, the capture is processed immediately afterward. The response includes the authorization and the capture information. If the authorization is declined, the capture is not processed, and the response message only includes the authorization information.

Partial Authorizations

All debit and prepaid card processors as well as a limited number of credit card processors support partial authorizations when using dual message processing.

When partial authorization is enabled, the issuing financial institution can approve a partial amount when the balance on the card is less than the requested amount. When a partial amount is authorized, the capture is not processed. This enables the merchant to use a second card to cover the balance, adjust the total cost, or void the transaction.

Single Message Processing

Single message processing, as the name implies, treats the authorization and capture as a single transaction. There are important differences for single message processing:

- Single message processing treats the request as a full-financial transaction, and with a successful transaction, funds are immediately transferred from the customer account to the merchant account.
- Authorization and capture amounts must be the same.
- Some features cannot be used with single message processing.

Authorization Reversals

An authorization reversal releases the hold that an authorization placed on the customer's payment card funds.

Each card-issuing financial institution has its own rules for deciding whether an authorization reversal succeeds or fails. When a reversal fails, contact the card-issuing financial institution to learn whether there is a different way to reverse the authorization. If your processor supports authorization reversal after void (ARAV), you can reverse an authorization after you void the associated capture. If your processor does not support ARAV, you can use the authorization reversal service only for an authorization that has not been captured and settled.

An authorization reversal is a follow-on transaction that uses the request ID returned from an authorization. The main purpose of a follow-on transaction is to link two transactions. The request ID links the follow-on transaction to the original transaction. The authorization request ID is used to look up the customer's billing and account information in the Cybersource database. You are not required to include those fields in the full authorization reversal request. The original transaction and follow-on transaction are linked in the database and in the Business Center.

For processors that support debit cards and prepaid cards, authorization reversal works for debit cards and prepaid cards in addition to credit cards.



Important

You cannot perform an authorization reversal if a transaction is in a review state, which can occur if you use a fraud management service. You must reject the transaction prior to authorization reversal. For more information, see the fraud management documentation in the Business Center.

Credits

Credits are cash refunds from a merchant to the cardholder after a cardholder pays for a product or service and that payment is captured by the merchant. When a credit request is successful, the issuing bank transfers funds from the merchant bank (acquirers) account to the customer's account. It typically takes two to four days for your acquiring bank to transfer funds from your merchant's account.



Warning

You should carefully control access to the credit service. Do not request this service directly from your customer interface. Instead, incorporate this service as part of your customer service process. This process reduces the potential for fraudulent transactions.

There are two basic types of credits:

- Refunds
- Stand-Alone Credits

Refunds

Refunds, also known as follow-on credits, use the capture request ID to link the refund to a specific transaction. This request ID is returned during the capture request (also known as a settlement) and is used in all subsequent refunds associated with the original

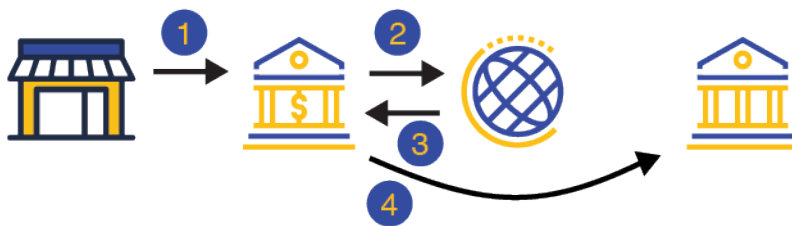
capture. The request ID links the transaction to the customer's billing and account information, so you are not required to include those fields in the credit request. However, when you combine a request for a refund with a request for another service, such as the tax calculation service, you must provide the customer's billing and account information. Unless otherwise specified, refunds must be requested within 60 days of a settlement. You can request multiple refunds against a single capture. To perform multiple refunds, use the same request ID in each request.

Stand-Alone Credits

Stand-alone credits are not tied to an original transaction. Stand-alone credits do not have a time restriction, and they can be used to issue refunds more than 60 days after a transaction settlement.

Credit Workflow

A credit does not happen in real time. All of the credit requests for a day are typically placed in a batch file and sent to the processor as a single batch transaction. In most cases, the batch transaction is settled overnight.



1. The merchant requests a credit.
2. The order information is validated.
3. The credit request is sent to the acquirer.
4. The acquirer transfers the requested funds to the issuer.

Capture or Credit Voids

A void cancels a capture or credit request that was submitted, but not yet processed by the processor.

Capture and credit requests are usually submitted once a day, so the window for voiding a capture or credit request is relatively short. A void request is declined when the capture or credit request has already been sent to the processor.

After a void is processed, you cannot credit or capture the funds. You must perform a new transaction to capture or credit the funds. Further, when you void a capture, a hold remains on the authorized funds. If you are not going to re-capture the authorization and if your processor supports authorization reversal after void, you should request an authorization reversal to release the hold on the unused credit card funds.

A void uses the capture or credit request ID to link the transactions. The authorization request ID is used to look up the customer's billing and account information, thus there is no need to include those fields in the void request. You cannot perform a follow-on credit against a capture that has been voided.

Capture

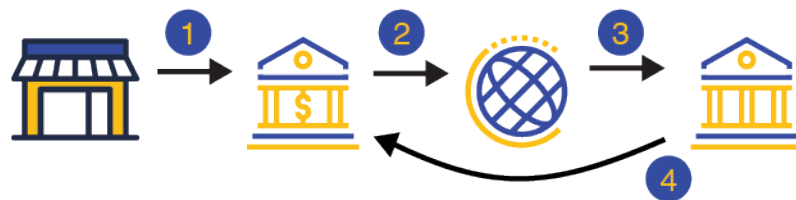
A capture is a follow-on transaction to an authorization. It is used to transfer the authorized funds from the customer's account to the merchant account. To link the authentication transaction to the capture transaction, you include a request ID in your capture request. This request ID is returned to you during the authentication response. Capture are typically not performed in real time. They are placed in a batch file and sent to the processor and the processor settles all of the captures at one time. In most cases, these batch files are sent and processed outside of the merchant's business hours. It usually takes two to four days for the acquiring financial institution to deposit the funds into the merchant account.

When fulfilling only part of a customer's order, do not capture the full amount of the authorization. Capture only the cost of the delivered items. When you deliver the remaining items, request a new authorization, and then capture the new authorization.

Important

It is not possible to perform a capture if a transaction is in a review state, which can occur if you use a fraud management service. You must accept the transaction prior to capture. For more information, see the fraud management documentation in the Business Center.

Capture Workflow



1. The merchant sends one or more transaction capture requests to the merchant bank (acquirer).
2. The merchant bank sends the capture package to the payment network.
3. The payment network forwards the capture package to correct the customer bank (issuer).
4. The customer bank settles the transactions and transfers the money to the merchant bank (acquirer).

Important

The payment processor does not notify Cybersource that the money has been transferred. To ensure that all captures are processed correctly, you should reconcile your capture requests with the capture reports from your processor.

Basic Processing

This section provides the minimum requirements for successfully processing the standard payment services.

Basic Authorizations

This section provides the minimal set of information required to perform a successful authorization.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Required Fields for Processing Basic Authorizations Using the REST API

orderInformation.amountDetails.currency
orderInformation.amountDetails.totalAmount
orderInformation.billTo.address1
orderInformation.billTo.administrativeArea
orderInformation.billTo.country
orderInformation.billTo.email
orderInformation.billTo.firstName
orderInformation.billTo.lastName
orderInformation.billTo.locality
orderInformation.billTo.postalCode
paymentInformation.card.expirationMonth

paymentInformation.card.expirationYear
paymentInformation.card.number
paymentInformation.card.type

Country-Specific Required Fields for Processing Basic Authorizations Using the REST API

Argentina

merchantInformation.taxId Required for Mastercard transactions.
merchantInformation.transactionLocalDateTime Required in Argentina when the time zone is not included in your account. Otherwise, this field is optional.

Brazil

paymentInformation.card.sourceAccountType Required for combo card transactions.
paymentInformation.card.sourceAccountType Required for combo card line-of-credit and prepaid-card transactions.

Chile

merchantInformation.taxId Required for Mastercard transactions.

Paraguay

merchantInformation.taxId Required for Mastercard transactions.

Saudi Arabia

processingInformation.authorizationOptions.requireOnlyFor Required only for merchants in Saudi Arabia.

Taiwan

paymentInformation.card.hashNumber Required only for merchants in Taiwan.

Declined Authorizations

If an authorization is declined, you can use response categories to help decide if you should retry or block a declined transaction. These response fields provide additional information:

- **paymentInsightsInformation.responseInsights.category**
- **paymentInsightsInformation.responseInsights.categoryCode**

Category codes have possible values (such as 01) that correspond to a category, which contains a description. These are the possible values:

- You cannot retry the following category code and category:
 - 01 ISSUER_WILL_NEVER_APPROVE
- For the following values, you can retry the transaction a maximum of 15 times over a period of 30 days:
 - 02 ISSUER_CANNOT_APPROVE_AT_THIS_TIME
 - 03 ISSUER_CANNOT_APPROVE_WITH_THESE_DETAILS: Data quality issue. Revalidate data prior to retrying the transaction.
 - 04 GENERIC_ERROR
 - 97 PAYMENT_INSIGHTS_INTERNAL_ERROR
 - 98 OTHERS
 - 99 PAYMENT_INSIGHTS_RESPONSE_CATEGORY_MATCH_NOT_FOUND

REST Interactive Example: Processing an Authorization

Simple Authorization(Internet)

Live Console URL: https://developer.cybersource.com/api-reference-assets/index.html#payments_payments_process-a-payment

REST Example: Processing a Basic Authorization

Endpoint: POST <https://api.cybersource.com/pts/v2/payments>

Request with Minimum Required Fields

```
{
  "orderInformation": {
    "billTo": {
      "country": "US",
      "lastName": "Kim",
      "address1": "201 S. Division St.",
      "postalCode": "48104-2201",
      "locality": "Ann Arbor",
      "administrativeArea": "MI",
      "firstName": "Kyong-Jin",
      "email": "test@cybs.com"
    },
    "amountDetails": {
      "totalAmount": "100.00",
      "currency": "usd"
    }
  },
  "paymentInformation": {
    "card": {
      "expirationYear": "2031",
      "number": "4111111111111111",
      "expirationMonth": "12",
      "type": "001"
    }
  }
}
```

```

    }
  }
}

```

Response: Successful Authorization

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6461731521426399003473/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6461731521426399003473"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6461731521426399003473/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "1646173152047"
  },
  "id": "6461731521426399003473",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "100.00",
      "currency": "usd"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "paymentInsightsInformation": {
    "responseInsights": {
      "categoryCode": "01"
    }
  },
  "processorInformation": {
    "systemTraceAuditNumber": "862481",
    "approvalCode": "831000",
    "merchantAdvice": {
      "code": "01",
      "codeRaw": "M001"
    }
  },

```

```

"responseDetails": "ABC",
"networkTransactionId": "016153570198200",
"consumerAuthenticationResponse": {
  "code": "2",
  "codeRaw": "2"
},
"transactionId": "016153570198200",
"responseCode": "00",
"avs": {
  "code": "Y",
  "codeRaw": "Y"
}
},
"reconciliationId": "6461731521426399003473",
"status": "AUTHORIZED",
"submitTimeUtc": "2022-03-01T22:19:12Z"
}

```

Response: Declined Authorization

```

{
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "errorInformation": {
    "reason": "PROCESSOR_ERROR",
    "message": "Invalid account"
  },
  "id": "6583553837826789303954",
  "paymentInsightsInformation": {
    "responseInsights": {
      "categoryCode": "01",
      "category": "ISSUER_WILL_NEVER_APPROVE"
    }
  },
  "pointOfSaleInformation": {
    "amexCapnData": "1009S0600100"
  },
  "processorInformation": {
    "systemTraceAuditNumber": "004544",
    "merchantNumber": "1231231222",
    "networkTransactionId": "431736869536459",
    "transactionId": "431736869536459",
    "responseCode": "111",
    "avs": {
      "code": "Y",
      "codeRaw": "Y"
    }
  },
  "status": "DECLINED"
}

```

Authorizing Payments Using the Business Center

The Virtual Terminal in the Business Center enables you to process one-time card-present and card-not-present transactions on the One-Time Payment page just as you would using a register in a retail environment.

Step 1: Enter Order Details

Order Details

Transaction Type *
Authorization

Currency
United States Dollar

Amount *
100.00

Merchant Contact Number

Merchant Descriptor

Order or Merchant Reference Number

Transaction Reference Number

Comments

Next → Cancel

1. Click **One-Time Payment**. The One-Time Payment page appears.
2. Under Customer Presence, click **Present** if the card is present or **Not Present** if the card is not present. The One-Time Payment page updates to reflect your choice.
3. Select **Authorization** in the Transaction Type field.
4. Enter the transaction amount in the Amount field.
5. Click Next.

Step 2: Enter Billing Information

Billing Information

Customer ID

First Name *
Last Name *

Company

Street Address 1 *
Street Address 2

City *
Sacramento

State *
CA

Zip/Postal Code *
95875

Country
United States

Phone Number

Email Address

☐ Shipping information same as billing

Shipping Information

← Previous Next → Cancel

1. Enter the cardholder's first name in the First Name field.
2. Enter the cardholder's last name in the Last Name field.
3. Enter the cardholder's billing address in the Street Address 1 field.
If a second line is required for the billing address, enter the second line in the Street Address 2 field.
4. Enter the billing city in the City field.
5. Enter the billing state in the State field.
6. Enter the billing postal code in the Zip/Postal Code field.
7. Click Next.

Step 3: Enter Payment Details

The screenshot shows a 'Payment Information' form with the following fields and values:

- Payment Type:** Credit Card
- Transaction Source *:** MOTO - Mail/Phone Order
- Credit Card Number *:** (Empty text field)
- Expiration Date *:** March / 2022
- Credit Card Type *:** Select card type

At the bottom of the form are three buttons: 'Previous' (with a left arrow), 'Submit' (in blue), and 'Cancel'.

1. Select the transaction type in the Transaction Source drop-down field.
2. Enter the card number in the Credit Card Number field.
3. Select card's expiration month and year in the Expiration date drop-down fields.
4. Select the card type in the Card Type drop-down field.
5. Click Submit.

Authorization with Line Itemization

This section shows you how to process an authorization request with line items.

Fields Specific to this Use Case

Remove the **orderInformation.amountDetails.totalAmount** field used in a basic authorization and include one or more line items in the **lineItem[]** array.

Using Line Items

These fields are required for each line item that you use:

- **orderInformation.lineItems[].unitPrice**
- **orderInformation.lineItems[].quantity**
- **orderInformation.lineItems[].productCode**

- **orderInformation.lineItems[].productSku** (optional when **item_#_productCode** is set to `default`, `shipping_only`, `handling_only`, or `shipping_and_handling`)
- **orderInformation.lineItems[].productName** (optional when **item_#_productCode** is set to `default`, `shipping_only`, `handling_only`, or `shipping_and_handling`)

At a minimum, you must include the **item_#_unitPrice** field in order to include a line item in an authorization. When this field is the only field included in the authorization, the system sets:

- **orderInformation.lineItems[].productCode**: `default`
- **orderInformation.lineItems[].quantity**: `1`

For example, these three line items are valid.

```
"orderInformation": {
  "lineItems": [
    {
      "unitPrice": "10.00"
    },
    {
      "unitPrice": "5.99",
      "quantity": "3",
      "productCode": "shipping_only"
    },
    {
      "unitPrice": "29.99",
      "quantity": "3",
      "productCode": "electronic_good",
      "productSku": "12384569",
      "productName": "receiver"
    }
  ]
}
```

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Required Fields

orderInformation.amountDetails.currency
orderInformation.amountDetails.totalAmount
orderInformation.billTo.address1
orderInformation.billTo.administrativeArea
orderInformation.billTo.country
orderInformation.billTo.email
orderInformation.billTo.firstName
orderInformation.billTo.lastName

`orderInformation.billTo.locality`
`orderInformation.billTo.postalCode`
`paymentInformation.card.expirationMonth`
`paymentInformation.card.expirationYear`
`paymentInformation.card.number`

Country Specific Required Fields

Argentina

`merchantInformation.taxId` Required for Mastercard transactions.
`merchantInformation.transactionLocalDateTime` Required in Argentina when the time zone is not included in your account. Otherwise, this field is optional.

Brazil

`paymentInformation.card.sourceAccountType` Required for combo card transactions.
`paymentInformation.card.sourceAccountType` Required for combo card line-of-credit and prepaid-card transactions.

Chile

`merchantInformation.taxId` Required for Mastercard transactions.

Paraguay

`merchantInformation.taxId` Required for Mastercard transactions.

Saudi Arabia

`processingInformation.authorizationOptions` Required only for merchants in Saudi Arabia.

REST Example: Processing a Basic Authorization with Line Items

Endpoint: `POST https://api.cybersource.com/pts/v2/payments`

Request

```
{
  "currencyConversion": {
    "indicator": "Y"
  },
  "paymentInformation": {
    "card": {
```



```

    "number": "4111111111111111",
    "expirationMonth": "12",
    "expirationYear": "2031"
  },
  "orderInformation": {
    "amountDetails": {
      "currency": "USD",
      "exchangeRate": ".91",
      "originalAmount": "107.33",
      "originalCurrency": "eur"
    },
    "billTo": {
      "firstName": "John",
      "lastName": "Doe",
      "address1": "1 Market St",
      "locality": "san francisco",
      "administrativeArea": "CA",
      "postalCode": "94105",
      "country": "US",
      "email": "test@cybs.com"
    },
    "lineItems": [
      {
        "unitPrice": "10.00"
      },
      {
        "unitPrice": "5.99",
        "quantity": "3",
        "productCode": "shipping_only"
      },
      {
        "unitPrice": "29.99",
        "quantity": "3",
        "productCode": "electronic_good",
        "productSku": "12384569",
        "productName": "receiver"
      }
    ]
  }
}

```

Response

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6482385519226028804003/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6482385519226028804003"
    },
    "capture": {

```

```

    "method": "POST",
    "href": "/pts/v2/payments/6482385519226028804003/captures"
  },
  "clientReferenceInformation": {
    "code": "1648238551902"
  },
  "id": "6482385519226028804003",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "117.94",
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "processorInformation": {
    "systemTraceAuditNumber": "191521",
    "approvalCode": "831000",
    "merchantAdvice": {
      "code": "01",
      "codeRaw": "M001"
    }
  },
  "responseDetails": "ABC",
  "networkTransactionId": "016153570198200",
  "consumerAuthenticationResponse": {
    "code": "2",
    "codeRaw": "2"
  },
  "transactionId": "016153570198200",
  "responseCode": "00",
  "avs": {
    "code": "Y",
    "codeRaw": "Y"
  }
},
"reconciliationId": "6482385519226028804003",
"status": "AUTHORIZED",
"submitTimeUtc": "2022-03-25T20:02:32Z"
}

```

Incremental Authorization

Incremental authorizations allow merchants to add additional products and services to an existing authorization. This section will show you how to append an original authorization to include additional transactions.

The incremental authorization service has these limitations:

- Maximum of 100 incremental authorizations per transaction, in addition to the initial authorization.
- Interchange optimization is not supported.
- Split shipments are not supported.

Supported Card Types

The supported card types for incremental authorizations are:

- Mastercard
- Visa

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Required Fields for Processing Incremental Authorizations Using REST APIs

clientReferenceInformation.code

id

Set the id URL parameter to the request ID that was included in the original authorization response message.

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

processingInformation.authorizationOptions.settlementMethod **InitiatedTransaction.reason**

Country Specific Required Fields for Processing Basic Authorizations Using REST APIs

Argentina

merchantInformation.transactionLocalDateTime Required in Argentina when the time zone is not included in your account. Otherwise, this field is optional.

Optional Fields for Processing Incremental Authorizations Using REST APIs

travellInformation.duration

You can include the following optional field in a request for an incremental authorization:

Example: Processing an Incremental Authorization Using the REST API

Request

Endpoint:

```
{
  "processingInformation": {
    "authorizationOptions": {
      "initiator": {
        "storedCredentialUsed": "true"
      },
      "merchantInitiatedTransaction": {
        "reason": "5"
      }
    }
  },
  "orderInformation": {
    "billTo": {
      "country": "US",
      "lastName": "Singh",
      "address1": "201 S. Division St.",
      "postalCode": "48104-2201",
      "locality": "Ann Arbor",
      "administrativeArea": "MI",
      "firstName": "Ranjeeth",
      "district": "MI",
      "email": "test@cybs.com"
    },
    "amountDetails": {
      "additionalAmount": "101.00",
      "totalAmount": "100.00",
      "currency": "usd"
    }
  },
  "paymentInformation": {
    "card": {
      "expirationYear": "2031",
      "number": "4111111111111111",
      "securityCode": "123",
      "expirationMonth": "12",
      "type": "001"
    }
  },
  "merchantInformation": {
    "transactionLocalDateTime": "20191002080000"
  }
}
```

```

},
"id": "3434254100000181552556"
}

```

Response

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6479624584536070903093/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6479624584536070903093"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6479624584536070903093/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "33557799"
  },
  "id": "6479624584536070903093",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "100.00",
      "currency": "usd"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "processorInformation": {
    "systemTraceAuditNumber": "819203",
    "approvalCode": "831000",
    "cardVerification": {
      "resultCodeRaw": "M",
      "resultCode": "M"
    },
    "merchantAdvice": {
      "code": "01",
      "codeRaw": "M001"
    }
  },
  "responseDetails": "ABC",

```

```

"networkTransactionId" : "016153570198200",
"retrievalReferenceNumber" : "208115819203",
"consumerAuthenticationResponse" : {
  "code" : "2",
  "codeRaw" : "2"
},
"transactionId" : "016153570198200",
"responseCode" : "00",
"avs" : {
  "code" : "Y",
  "codeRaw" : "Y"
}
},
"reconciliationId" : "6479624584536070903093",
"status" : "AUTHORIZED",
"submitTimeUtc" : "2022-03-22T15:20:58Z"
}

```

Processing an Authorization with a Card Verification Number

This section provides the minimum information required to perform a successful authorization with a card verification number (CVN).

Fields Specific to This Use Case

Include this field in a standard authorization request with a CVN:

- **paymentInformation.card.securityCode**

CVN Results

The response includes a raw response code and a mapped response code:

- The raw response code is the value returned by the processor. This value is returned in the **processorInformation.cardVerification.resultCodeRaw** field. Use this value only for debugging purposes; do not use it to determine the card verification response.
- The mapped response code is the pre-defined value that corresponds to the raw response code. This value is returned in the **processorInformation.cardVerification.resultCode** field.

Even when the CVN does not match the expected value, the issuing bank might still authorize the transaction. You will receive a CVN decline, but you can still capture the transaction because it has been authorized by the bank. However, you must review the order to ensure that it is legitimate.

Settling authorizations that fail the CVN check might have an impact on the fees charged by your bank. Contact your bank for details about how card verification management might affect your discount rate.

When a CVN decline is received for the authorization in a sale request, the capture request is not processed unless you set the **processorInformation.authorizationOptions.ignoreCvResult** field to **true**.

CVN Results for American Express

A value of **1** in the **processorInformation.cardVerification.resultCode** field indicates that your account is not configured to use card verification. Contact customer support to have your account enabled for this feature.

CVN Results for Discover

When the CVN does not match, Discover refuses the card and the request is declined. The reply message does not include the **processorInformation.cardVerification.resultCode** field, which indicates that the CVN failed.

CVN Results for Visa and Mastercard

A CVN code of **D** or **N** causes the request to be declined with a reason code value of **230**. You can still capture the transaction, but you must review the order to ensure that it is legitimate.

Cybersource, not the issuer, assigns the CVN decline to the authorization. You can capture any authorization that has a valid authorization code from the issuer, even when the request receives a CVN decline.

When the issuer does not authorize the transaction and the CVN does not match, the request is declined because the card is refused. You cannot capture the transaction.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Required Fields for Processing Basic Authorizations with a Card Verification Number Using the REST API

These fields are required in an authorization request with a CVN:

orderInformation.amountDetails.currency
orderInformation.amountDetails.totalAmount
orderInformation.billTo.address1
orderInformation.billTo.administrativeArea
orderInformation.billTo.country
orderInformation.billTo.email
orderInformation.billTo.firstName
orderInformation.billTo.lastName
orderInformation.billTo.locality
orderInformation.billTo.postalCode
paymentInformation.card.expirationMonth
paymentInformation.card.expirationYear

paymentInformation.card.number
paymentInformation.card.securityCode
paymentInformation.card.type
paymentInformation.card.securityCode

Optional Fields for an Authorization with a Card Verification Number

You can include these optional fields in an authorization request with a card verification number:

paymentInformation.card.securityCodeIndicator
processingInformation.authorizationOptions.ignoreCvResult

REST Example: Processing an Authorization with a CVN

Request

```
{
  "paymentInformation": {
    "card": {
      "number": "4111111111111111",
      "expirationMonth": "12",
      "expirationYear": "2031",
      "type": "001",
      "securityCode": "999"
    }
  },
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "49.95",
      "currency": "USD"
    }
  },
  "billTo": {
    "firstName": "John",
    "lastName": "Doe",
    "address1": "1295 Charleston Rd.",
    "locality": "Mountain View",
    "administrativeArea": "CA",
    "postalCode": "94043",
    "country": "US",
    "email": "jdoe@example.com",
    "phoneNumber": "650-965-6000"
  }
}
```

Response

```
{
  "_links": {
```



```

    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6554147587216874903954/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6554147587216874903954"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6554147587216874903954/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "1655414758839"
  },
  "id": "6554147587216874903954",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "49.95",
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "pointOfSaleInformation": {
    "terminalId": "111111"
  },
  "processorInformation": {
    "approvalCode": "888888",
    "networkTransactionId": "123456789619999",
    "transactionId": "123456789619999",
    "responseCode": "100",
    "avs": {
      "code": "X",
      "codeRaw": "I1"
    }
  },
  "reconciliationId": "67546603C43Z6JWN",
  "status": "AUTHORIZED",
  "submitTimeUtc": "2022-06-16T21:25:58Z"
}

```

Zero Amount Authorizations

This section shows you how to process zero amount authorizations.

Zero amount authorizations enable you to validate a card without charging the account.

Processor Specific Information

Visa Platform Connect

Do not set the commerce indicator to recurring.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Required Fields for Processing Zero Amount Authorizations Using the REST API

orderInformation.amountDetails.currency
orderInformation.amountDetails.totalAmount
orderInformation.billTo.address1
orderInformation.billTo.administrativeArea
orderInformation.billTo.country
orderInformation.billTo.email
orderInformation.billTo.firstName
orderInformation.billTo.lastName
orderInformation.billTo.locality
orderInformation.billTo.postalCode
paymentInformation.card.expirationMonth
paymentInformation.card.expirationYear
paymentInformation.card.number

Country-Specific Required Fields for Processing Basic Zero Amount Authorizations Using the REST API

Argentina

merchantInformation.taxId Required for Mastercard transactions.

merchantInformation.transactionLocalDate Time Required in Argentina when the time zone is not included in your account. Otherwise, this field is optional.

Brazil

paymentInformation.card.sourceAccountType Required for combo card transactions.

paymentInformation.card.sourceAccountType Required for combo card line-of-credit and prepaid-card transactions.

Saudi Arabia

processingInformation.authorizationOptions.transactionMode

Taiwan

paymentInformation.card.hashedException

REST Example: Processing a Zero Amount Authorization

Endpoint: POST <https://api.cybersource.com/pts/v2/payments>

Request

```
{
  "orderInformation": {
    "billTo": {
      "country": "US",
      "lastName": "Kim",
      "address1": "201 S. Division St.",
      "postalCode": "48104-2201",
      "locality": "Ann Arbor",
      "administrativeArea": "MI",
      "firstName": "Kyong-Jin",
      "email": "test@cybs.com"
    },
    "amountDetails": {
      "totalAmount": "0.00",
      "currency": "usd"
    }
  },
  "paymentInformation": {
    "card": {
      "expirationYear": "2031",
      "number": "4111111111111111",
      "expirationMonth": "12"
    }
  }
}
```

Response

```
{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6461731521426399003473/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6461731521426399003473"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6461731521426399003473/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "1646173152047"
  },
  "id": "6461731521426399003473",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "0",
      "currency": "usd"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "processorInformation": {
    "systemTraceAuditNumber": "862481",
    "approvalCode": "831000",
    "merchantAdvice": {
      "code": "01",
      "codeRaw": "M001"
    }
  },
  "responseDetails": "ABC",
  "networkTransactionId": "016153570198200",
  "consumerAuthenticationResponse": {
    "code": "2",
    "codeRaw": "2"
  },
  "transactionId": "016153570198200",
  "responseCode": "00",
  "avs": {
```

```

    "code" : "Y",
    "codeRaw" : "Y"
  }
},
"reconciliationId" : "6461731521426399003473",
"status" : "AUTHORIZED",
"submitTimeUtc" : "2022-03-01T22:19:12Z"
}

```

Authorization with an SCA Exemption

This section demonstrates how to process an authorization with a strong customer authentication (SCA) exemption. Merchants cannot decide whether an exemption applies to a transaction. Only the card issuing bank can grant a SCA exemption during card authentication.

A SCA exemption enables you to remain in compliance with the European Union's second Payment Services Directive. Depending on your processor, use one of the following exemption fields:



Important

If you send more than one SCA exemption field with a single authentication, the transaction is denied.

- Authentication Outage: Payer authentication is not available for this transaction due to a system outage.
- B2B Corporate Card: Payment cards specifically for business-to-business transactions are exempt.
- Delegated Authentication: Payer authentication was performed outside of the authorization workflow.
- Follow-On Installment Payment: Installment payments of a fixed amount are exempt after the first transaction.
- Follow-On Recurring Payment: Recurring payments of a fixed amount are exempt after the first transaction.
- Low Risk: The average fraud levels associated with this transaction are considered low.
- Low Value: The transaction value does not warrant SCA.
- Merchant Initiated Transactions: As follow-on transactions, merchant-initiated transactions are exempt.
- Stored Credential Transaction: Credentials are authenticated before storing, so stored credential transactions are exempt.
- Trusted Merchant: Merchants registered as trusted beneficiaries.

Exemption Fields Specific to This Use Case

Use one of these fields to request an SCA exemption:

Exemption Type	Field	Value
Authentication Outage	consumerAuthenticationInformation. strongAuthentication. authenticationOutageExemptionIndicator	1
B2B Corporate Card Transaction	consumerAuthenticationInformation. strongAuthentication. secureCorporatePaymentIndicator	1
Delegated Authentication	consumerAuthenticationInformation. strongAuthentication. delegatedAuthenticationExemptionIndicator	1
Low-Risk Transaction	consumerAuthenticationInformation. strongAuthentication. riskAnalysisExemptionIndicator	1
Low-Value Transaction	consumerAuthenticationInformation. strongAuthentication. lowValueExemptionIndicator	1
Stored Credential Transaction	processingInformation. authorizationOptions.initiator. storedCredentialUsed	1
Trusted Merchant Transaction	consumerAuthenticationInformation. strongAuthentication. trustedMerchantExemptionIndicator	1

Country-Specific Requirements

These fields are specific to certain countries and regions.

Argentina

merchantInformation.taxId Required for Mastercard transactions.

merchantInformation.transactionLocalDateTime Required when the time zone is not included in your account. Otherwise, this field is optional.

Brazil

paymentInformation.card.sourceAccountType Required for combo card transactions.

paymentInformation.card.sourceAccountType Required for combo card line-of-credit and prepaid-card transactions.

Chile

merchantInformation.taxId Required for Mastercard transactions.

Paraguay

`merchantInformation.taxId`

Required for Mastercard transactions.

Saudi Arabia

`processingInformation.authorizationOptions.transactionMode`

Taiwan

`paymentInformation.card.hashedException`

Endpoint

POST `https://api.cybersource.com/pts/v2/payments`

Required Fields for an Authorization with an SCA Exemption

You must also include one SCA exemption field from the [SCA Exemption Fields](#) list.`orderInformation.amountDetails.currency``orderInformation.amountDetails.totalAmount``orderInformation.billTo.address1``orderInformation.billTo.administrativeArea``orderInformation.billTo.country``orderInformation.billTo.email``orderInformation.billTo.firstName``orderInformation.billTo.lastName``orderInformation.billTo.locality``orderInformation.billTo.postalCode``paymentInformation.card.expirationMonth``paymentInformation.card.expirationYear``paymentInformation.card.number``paymentInformation.card.type`

REST Example: Authorization with an SCA Exemption for Low Value Transactions

Request

Endpoint: POST `https://api.cybersource.com/pts/v2/payments`

```
{
  "consumerAuthenticationInformation": {
    "strongAuthentication": {
      "lowValueExemptionIndicator": "1"
    }
  }
},
```

```

"orderInformation": {
  "billTo": {
    "country": "US",
    "lastName": "Kim",
    "address1": "201 S. Division St.",
    "postalCode": "48104-2201",
    "locality": "Ann Arbor",
    "administrativeArea": "MI",
    "firstName": "Kyong-Jin",
    "email": "test@cybs.com"
  },
  "amountDetails": {
    "totalAmount": "100.00",
    "currency": "eur"
  }
},
"paymentInformation": {
  "card": {
    "expirationYear": "2031",
    "number": "4111111111111111",
    "expirationMonth": "12"
  }
}
}

```

Response

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6709780221406171803955/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6709780221406171803955"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6709780221406171803955/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "1670978022258"
  },
  "id": "6709780221406171803955",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "100.00",
      "currency": "eur"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  }
}

```



```

},
"paymentInformation": {
  "tokenizedCard": {
    "type": "001"
  },
  "card": {
    "type": "001"
  }
},
"pointOfSaleInformation": {
  "terminalId": "123456"
},
"processorInformation": {
  "approvalCode": "888888",
  "networkTransactionId": "123456789619999",
  "transactionId": "123456789619999",
  "responseCode": "100",
  "avs": {
    "code": "X",
    "codeRaw": "I1"
  }
},
"reconciliationId": "62859554PBDEMI43",
"status": "AUTHORIZED",
"submitTimeUtc": "2022-12-14T00:33:42Z"
}

```

Authorization Reversals

Reversing an authorization releases the hold on the customer's payment card funds that the issuing bank placed when processing the authorization.

Each card-issuing financial institution has its own rules for deciding whether an authorization reversal succeeds or fails. When a reversal fails, contact the card-issuing financial institution to learn whether there is a different way to reverse the authorization. If your processor supports authorization reversal after void (ARAV), you can reverse an authorization after you void the associated capture. If your processor does not support ARAV, you can use the full authorization reversal service only for an authorization that has not been captured and settled.

An authorization reversal is a follow-on transaction that uses the request ID returned from an authorization transaction to link the transactions together. The authorization request ID is used to look up the customer's billing and account information in the Cybersource database. You are not required to include those fields in the full authorization reversal request. The original transaction and follow-on transaction are linked in the database and in the Business Center.

For processors that support debit cards and prepaid cards, the full authorization reversal service works for debit cards and prepaid cards in addition to credit cards.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments/{id}/reversals>

Required Fields for Processing Authorization Reversals Using REST API

These fields are required in a request for an authorization reversal:

id	Set the id URL parameter to the request ID that was included in the authorization response message.
reversalInformation.amountDetails.totalAmount	The amount of the reversal must be the same as the authorization amount that was included in the authorization response message. Do not use the amount that was requested in the authorization request message.

Related information

- [API Field Reference for the REST API](#)

Example: Processing an Authorization Reversal Using the REST API

Request

Endpoint: **POST** <https://api.cybersource.com/pts/v2/payments/{id}/reversals>

```
{
  "reversalInformation": {
    "amountDetails": {
      "totalAmount": "100.00"
    }
  }
}
```

Response

```
{
  "_links": {
    "self": {
      "method": "GET",
      "href": "/pts/v2/reversals/6522010713906068903093"
    }
  },
  "authorizationInformation": {
    "approvalCode": "100"
  },
  "clientReferenceInformation": {
    "code": "1652201071257"
  },
  "id": "6522010713906068903093",
  "orderInformation": {
    "amountDetails": {
      "currency": "USD"
    }
  }
}
```

```

    },
    "processorInformation": {
      "responseCode": "0"
    },
    "reversalAmountDetails": {
      "reversedAmount": "100.00",
      "currency": "USD"
    },
    "status": "REVERSED",
    "submitTimeUtc": "2022-05-10T16:44:31Z"
  }
}

```

Captures

This section shows you how to capture an authorized transaction.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments/{id}/captures>

`id` is the transaction ID returned in the authorization response.

Required Fields for Capturing a Payment Using the REST API

`orderInformation.amountDetails.currency`

`orderInformation.amountDetails.totalAmount`

REST Interactive Example: Capturing a Payment

Capture a Payment

Live Console URL: https://developer.cybersource.com/api-reference-assets/index.html#payments_capture_capture-a-payment

REST Example: Capturing an Authorization

Request

Endpoint: POST <https://api.cybersource.com/pts/v2/payments/{id}/captures>

```

{
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "100.00"
    }
  }
}

```

Response

```

{

```

```

    "_links": {
      "void": {
        "method": "POST",
        "href": "/pts/v2/captures/6662994431376681303954/voids"
      },
      "self": {
        "method": "GET",
        "href": "/pts/v2/captures/6662994431376681303954"
      }
    },
    "clientReferenceInformation": {
      "code": "1666299443215"
    },
    "id": "6662994431376681303954",
    "orderInformation": {
      "amountDetails": {
        "totalAmount": "100.00",
        "currency": "EUR"
      }
    },
    "reconciliationId": "66535942B9CGT52U",
    "status": "PENDING",
    "submitTimeUtc": "2022-10-20T20:57:23Z"
  }
}

```

Multiple Captures

This feature enables you to request multiple partial captures for one authorization. A multiple partial capture allows you to incrementally settle authorizations over time. Ensure that the total amount of all the captures does not exceed the authorized amount.

Prerequisite

Contact customer support to have your account enabled for this feature.

Limitations

Your account can be enabled for multiple partial captures or split shipments; it cannot be enabled for both features.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments/{id}/captures>

`id` is the transaction ID returned in the authorization response.

Required Fields for Multiple Captures

These fields are required in a request for a capture when you are requesting multiple partial captures:

processingInformation.captureOptions.captureRequiredInUs Required if you are in the U.S.

`processingInformation.captureOptions.totalCaptureCount` When you do not know the total number of captures that you are going to request, set the capture total count to an estimated value or 99 for all capture requests except the final one. For the final capture request, set the capture total count and the capture sequence to the same value.

Required if you are in the U.S.

Related information

- API Field Reference for the REST API

REST Example: Processing Multiple Partial Captures

Request

Endpoint: `POST https://api.cybersource.com/pts/v2/payments/{id}/captures`
`id` is the transaction ID returned in the authorization response.

```
{
  {
    "clientReferenceInformation": {
      "code": "TC50171_3"
    },
    "processingInformation": {
      "captureOptions": {
        "captureSequenceNumber": null,
        "totalCaptureCount": null
      }
    },
    "orderInformation": {
      "amountDetails": {
        "totalAmount": "102.21",
        "currency": "USD"
      }
    }
  }
}
```

Response

```
{
  "_links": {
    "void": {
      "method": "POST",
      "href": "/pts/v2/captures/6742496815656503003954/voids"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/captures/6742496815656503003954"
    }
  },
  "clientReferenceInformation": {
```

```

    "code": "TC50171_3"
  },
  "id": "6742496815656503003954",
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "102.21",
      "currency": "USD"
    }
  },
  "reconciliationId": "67332020GD2G1001",
  "status": "PENDING",
  "submitTimeUtc": "2023-01-20T21:21:21Z"
}

```

Credits

This topic shows you how to process a credit.

Endpoint

POST <https://api.cybersource.com/pts/v2/credits/>

Required Fields for Processing a Credit Using the REST API

These fields are required in a credit request:

orderInformation.amountDetails.currency
orderInformation.amountDetails.totalAmount
orderInformation.billTo.address1
orderInformation.billTo.administrationArea
orderInformation.billTo.country
orderInformation.billTo.email
orderInformation.billTo.firstName
orderInformation.billTo.lastName
orderInformation.billTo.locality
orderInformation.billTo.postalCode
paymentInformation.card.expirationMonth
paymentInformation.card.expirationYear
paymentInformation.card.number

REST Interactive Example: Processing a Credit

Credit

Live Console URL: https://developer.cybersource.com/api-reference-assets/index.html#payments_credit_process-a-credit

REST Example: Processing a Credit

Request

Endpoint: POST <https://api.cybersource.com/pts/v2/credits/>

```
{
  "orderInformation": {
    "billTo": {
      "country": "US",
      "lastName": "Kim",
      "address1": "201 S. Division St.",
      "postalCode": "48104-2201",
      "locality": "Ann Arbor",
      "administrativeArea": "MI",
      "firstName": "Kyong-Jin",
      "email": "test@cybs.com"
    },
    "amountDetails": {
      "totalAmount": "100.00",
      "currency": "eur"
    }
  },
  "paymentInformation": {
    "card": {
      "expirationYear": "2031",
      "number": "4111111111111111",
      "expirationMonth": "12"
    }
  }
}
```

Response

```
{
  "_links": {
    "void": {
      "method": "POST",
      "href": "/pts/v2/credits/6663069906146706403954/voids"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/credits/6663069906146706403954"
    }
  },
  "clientReferenceInformation": {
    "code": "1666306990717"
  },
}
```

```

"creditAmountDetails": {
  "currency": "eur",
  "creditAmount": "100.00"
},
"id": "6663069906146706403954",
"orderInformation": {
  "amountDetails": {
    "currency": "eur"
  }
},
"paymentAccountInformation": {
  "card": {
    "type": "001"
  }
},
"paymentInformation": {
  "tokenizedCard": {
    "type": "001"
  },
  "card": {
    "type": "001"
  }
},
"processorInformation": {
  "approvalCode": "888888",
  "responseCode": "100"
},
"reconciliationId": "66490108K9CLFJPN",
"status": "PENDING",
"submitTimeUtc": "2022-10-20T23:03:10Z"
}

```

Refunds

This section shows you how to process a refund. You must request a refund within 180 days of the authorization.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments/{id}/refunds>

id is the transaction ID returned during the authorization request.

Required Fields for Processing Refunds Using the REST API

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

orderInformation.billTo.address1

orderInformation.billTo.administrationArea

orderInformation.billTo.country

orderInformation.billTo.email
orderInformation.billTo.firstName
orderInformation.billTo.lastName
orderInformation.billTo.locality
orderInformation.billTo.postalCode
paymentInformation.card.expirationMonth
paymentInformation.card.expirationYear
paymentInformation.card.number

REST Interactive Example: Processing a Refund

Refund a Payment

Live Console URL: https://developer.cybersource.com/api-reference-assets/index.html#payments_refund_refund-a-payment

Example: Processing a Refund Using the REST API

Request

Endpoint: POST <https://api.cybersource.com/pts/v2/payments/{id}/refunds>

```
{
  "orderInformation": {
    "billTo": {
      "country": "US",
      "lastName": "Kim",
      "address1": "201 S. Division St.",
      "postalCode": "48104-2201",
      "locality": "Ann Arbor",
      "administrativeArea": "MI",
      "firstName": "Kyong-Jin",
      "email": "test@cybs.com"
    },
    "amountDetails": {
      "totalAmount": "100.00",
      "currency": "eur"
    }
  },
  "paymentInformation": {
    "card": {
      "expirationYear": "2031",
      "number": "4111111111111111",
      "expirationMonth": "12"
    }
  }
}
```

Response

```
{
  "_links": {
    "void": {
      "method": "POST",
      "href": "/pts/v2/credits/6699964581696622603955/voids"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/credits/6699964581696622603955"
    }
  },
  "clientReferenceInformation": {
    "code": "1669996458298"
  },
  "creditAmountDetails": {
    "currency": "eur",
    "creditAmount": "100.00"
  },
  "id": "6699964581696622603955",
  "orderInformation": {
    "amountDetails": {
      "currency": "eur"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "processorInformation": {
    "approvalCode": "888888",
    "responseCode": "100"
  },
  "reconciliationId": "618733290AILG3Q6",
  "status": "PENDING",
  "submitTimeUtc": "2022-12-02T15:54:18Z"
}
```

Sales

This section shows you how to process a sale transaction.

A sale transaction combines and authorization and a capture into a single transaction.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Required Fields for Processing a Sale Using the REST API

The following fields are required in a request for a sale:

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

orderInformation.billTo.address1

orderInformation.billTo.administrativeArea

orderInformation.billTo.country

orderInformation.billTo.email

orderInformation.billTo.firstName

orderInformation.billTo.LastName

orderInformation.billTo.locality

orderInformation.billTo.postalCode

paymentInformation.card.expirationMonth

paymentInformation.card.expirationYear

paymentInformation.card.number

paymentInformation.card.securityCode Required only for Visa Platform Connect.

paymentInformation.card.type

REST Example: Requesting a Sale

Request

Endpoint: POST <https://api.cybersource.com/pts/v2/payments>

```
{
  "processingInformation": {
    "capture": true
  },
  "orderInformation": {
    "billTo": {
      "country": "US",
      "lastName": "VDP",
      "address1": "201 S. Division St.",
      "postalCode": "48104-2201",
      "locality": "Ann Arbor",
      "administrativeArea": "MI",
      "firstName": "RTS",
      "email": "test@cybs.com"
    },
    "amountDetails": {
```

```

    "totalAmount": "100.00",
    "currency": "usd"
  },
  "paymentInformation": {
    "card": {
      "expirationYear": "2031",
      "number": "4111111111111111",
      "expirationMonth": "12",
      "type": "001",
      "securityCode": "999" <!-- Required only for Visa Platform Connect. -->
    }
  }
}

```

Response

Most processors do not return all of the fields shown in this example.

```

{
  "_links": {
    "void": {
      "method": "POST",
      "href": "/pts/v2/payments/6485004068966546103093/voids"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6485004068966546103093"
    }
  },
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "id": "6485004068966546103093",
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "100.00",
      "authorizedAmount": "100.00",
      "currency": "usd"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
}

```

```

"processorInformation": {
  "systemTraceAuditNumber": "841109",
  "approvalCode": "831000",
  "merchantAdvice": {
    "code": "01",
    "codeRaw": "M001"
  },
  "responseDetails": "ABC",
  "networkTransactionId": "016153570198200",
  "retrievalReferenceNumber": "208720841109",
  "consumerAuthenticationResponse": {
    "code": "2",
    "codeRaw": "2"
  },
  "transactionId": "016153570198200",
  "responseCode": "00",
  "avs": {
    "code": "Y",
    "codeRaw": "Y"
  }
},
"reconciliationId": "6485004068966546103093",
"status": "AUTHORIZED",
"submitTimeUtc": "2022-03-28T20:46:47Z"
}

```

Voids

This section shows you how to successfully process a void.

Endpoint

POST <https://api.cybersource.com/pts/v2/captures/{id}/voids>

POST <https://api.cybersource.com/pts/v2/credits/{id}/voids>

id is the transaction ID returned during the request to void.

Required Fields for Processing Voids Using REST APIs

The following fields are required in a request for a void:

id

Set the **id URL parameter to the request ID that was included in the authorization response message.**

REST Example: Processing a Void

Request

Endpoint: POST <https://api.cybersource.com/pts/v2/captures/{id}/voids>

**Important**

POST requests for voids require a body. If you have no fields to pass, use empty braces as shown below.

```
{  
}
```

Response

```
{  
  "_links": {  
    "self": {  
      "method": "GET",  
      "href": "/pts/v2/voids/6541933390746728203005"  
    }  
  },  
  "clientReferenceInformation": {  
    "code": "1654193339056"  
  },  
  "id": "6541933390746728203005",  
  "orderInformation": {  
    "amountDetails": {  
      "currency": "USD"  
    }  
  },  
  "status": "VOIDED",  
  "submitTimeUtc": "2022-06-02T18:08:59Z",  
  "voidAmountDetails": {  
    "currency": "usd",  
    "voidAmount": "100.00"  
  }  
}
```

Card-Present Authorizations

This section describes the types of card-present transactions.

The presence of a card is determined during the card authorization. Thus, the main difference between a card present transaction and a card not-present transaction is in the authorization service. There are three basic types of card-present authorizations:

- EMV Authorizations: Authorizations using the EMV chip embedded in the user's card
- Magnetic Stripe Authorization: Authorizations based on the magnetic stripe on the user's card
- Hand-Keyed Authorizations: Authorizations based on the merchant manually entering the card information into the terminal

This section will guide you through the various types of card present transactions available to you.

EMV Chip and Online PIN Authorization

For an EMV chip-based authorization, the customer inserts the card into a POS terminal, which causes the chip to be in direct physical contact with the terminal. For an online PIN transaction, the customer uses a PIN to verify their identity and the issuer verifies the PIN.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Fallback

EMV fallback is an option that allows you to use an alternative authorization method if the EMV authorization experiences a technical failure. To enable EMV fallback:

- Specify that EMV Fallback is allowed.
pointOfSaleInformation.emv.fallback=true

- Specify the condition that triggers EMV Fallback for American Express or Mastercard transactions.

pointOfSaleInformation.emv.fallbackCondition=<condition>

- Provide the service code from the track data for American Express or Mastercard transactions.

pointOfSaleInformation.serviceCode=<service_code>

Required Fields for a Contact EMV Authorization with Online PIN Using the REST API

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

pointOfSaleInformation.cardPresent Set this field to **true**.

pointOfSaleInformation.encryptedPin

pointOfSaleInformation.entryMode Set this field to **contact** for an EMV payment.

pointOfSaleInformation.tags

pointOfSaleInformation.trackData

processingInformation.commerceIndicator Set this field to **retail**.

Country-Specific Required Fields for an EMV Credit Card Transaction Using the REST API

Argentina

merchantInformation.transactionLocalDateTime Required when the time zone is not set in your account.

processorInformation.salesSlipNumber

India

pointOfSaleInformation.terminalCompliance

Japan

processorInformation.salesSlipNumber

REST Example: EMV Chip and Online PIN Authorization

Request

Endpoint: POST <https://api.cybersource.com/pts/v2/payments>

[illegible]

Response

```
{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6559185445566394203955/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6559185445566394203955"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6559185445566394203955/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "1655918544653"
  },
  "id": "6559185445566394203955",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "99.98",
      "currency": "usd"
    }
  },
  "paymentAccountInformation": {
```

```

    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "pointOfSaleInformation": {
    "terminalId": "111111"
  },
  "processorInformation": {
    "approvalCode": "888888",
    "networkTransactionId": "123456789619999",
    "transactionId": "123456789619999",
    "responseCode": "100",
    "avs": {
      "code": "1"
    }
  },
  "reconciliationId": "68164611U43Z1LRC",
  "status": "AUTHORIZED",
  "submitTimeUtc": "2022-06-22T17:22:24Z"
}

```

EMV Chip (Contact) and Offline PIN Authorization

During an EMV chip authorization, the customer inserts the card into the terminal, which causes the chip to be in contact with the terminal. When processing an offline PIN transaction, chip verifies the customer PIN.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Fallback

EMV fallback is an option that allows you to use an alternative authorization method if the EMV authorization experiences a technical failure. To enable EMV fallback:

- Specify that EMV Fallback is allowed.
pointOfSaleInformation.emv.fallback=true
- Specify the condition that triggers EMV Fallback for American Express or Mastercard transactions.
pointOfSaleInformation.emv.fallbackCondition=<condition>

- Provide the service code from the track data for American Express or Mastercard transactions.

`pointOfSaleInformation.serviceCode=<service_code>`

Required Fields for a Contact EMV Authorization with Offline PIN Using the REST API

`orderInformation.amountDetails.currency`

`orderInformation.amountDetails.totalAmount`

`pointOfSaleInformation.cardPresent` Set this field to `true`.

`pointOfSaleInformation.entryMode` Set this field to `contact`.

`pointOfSaleInformation.terminalCapability` Set this field to `4`. Set this field to `0` if the terminal does not support PINs.

`pointOfSaleInformation.tags`

`pointOfSaleInformation.trackData`

`processingInformation.authorizationOptions.settlementType` Set this field to `verbal`.

`processingInformation.commerceIndicator` Set this field to `retail`.

Country-Specific Required Fields for an EMV Credit Card Transaction Using the REST API

Argentina

`merchantInformation.transactionLocalDateTime` Required when the time zone is not set in your account.

`processorInformation.salesSlipNumber`

India

`pointOfSaleInformation.terminalCompliance`

Japan

`processorInformation.salesSlipNumber`

REST Example: EMV Chip (Contact) with an Offline PIN

Request

Endpoint: `POST https://api.cybersource.com/pts/v2/payments`

```
{
  "processingInformation": {
```

```

    "commerceIndicator": "retail",
    "authorizationOptions": {
      "verbalAuthType": "verbal"
    }
  },
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "99.98",
      "currency": "usd"
    }
  },
  "pointOfSaleInformation": {
    "cardPresent": "Y",
    "entryMode": "contact",
    "terminalCapability": "4",
    "tags":
    "9F33032040009505000000000009F3704518823719F100706011103A000009F26081E1756ED0E2134E29F3602001582020000
    "trackData": "%B4111111111111111^LEE/
TANYA^25121200123456789025**XX*****?;4111111111111111=25121200XXXX000000000?"
  }
}

```

Response

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6473256016126696903006/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6473256016126696903006"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6473256016126696903006/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "OrderABC12"
  },
  "id": "6473256016126696903006",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "99.98",
      "currency": "usd"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {

```

```

    "type": "001"
  },
  "card": {
    "type": "001"
  }
},
"pointOfSaleInformation": {
  "processorInformation": {
    "approvalCode": "888888",
    "networkTransactionId": "123456789619999",
    "transactionId": "123456789619999",
    "systemTraceAuditNumber": "246802",
    "responseCode": "100"
  },
  "reconciliationId": "6580310700JCYR8J",
  "status": "AUTHORIZED",
  "submitTimeUtc": "2022-03-15T06:26:41Z"
}

```

EMV Chip (Contact) and Signature

For an EMV contact payment, the customer inserts the card into the terminal, which causes the chip to be in direct physical contact with the terminal. For a signature transaction, the customer uses a signature to verify their identity.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Fallback

EMV fallback is an option that allows you to use an alternative authorization method if the EMV authorization experiences a technical failure. To enable EMV fallback:

- Specify that EMV Fallback is allowed.
pointOfSaleInformation.emv.fallback=true
- Specify the condition that triggers EMV Fallback for American Express or Mastercard transactions.
pointOfSaleInformation.emv.fallbackCondition=<condition>
- Provide the service code from the track data for American Express or Mastercard transactions.
pointOfSaleInformation.serviceCode=<service_code>

Required Fields for a Contact EMV Authorization with a Signature Using the REST API

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

pointOfSaleInformation.cardPresent

Set this field to `true`.

pointOfSaleInformation.entryMode

Set this field to `contact` for an EMV payment.

pointOfSaleInformation.tags

pointOfSaleInformation.trackData

processingInformation.commerceIndicator Set this field to retail.

Country-Specific Required Fields for an EMV Credit Card Transaction Using the REST API

Argentina

merchantInformation.transactionLocalDateTime required when the time zone is not set in your account.

processorInformation.salesSlipNumber

India

pointOfSaleInformation.terminalCompliance

Japan

processorInformation.salesSlipNumber

REST Example: EMV Chip (Contact) with a Signature

Request

Endpoint: POST <https://api.cybersource.com/pts/v2/payments>

```
{
  "processingInformation": {
    "commerceIndicator": "retail"
  },
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "99.98",
      "currency": "usd"
    }
  },
  "pointOfSaleInformation": {
    "cardPresent": "Y",
    "entryMode": "contact",
    "tags":
"9F33032040009505000000000009F3704518823719F100706011103A0000009F26081E1756ED0E2134E29F3602001582020000",
    "trackData": "%B41111111111111^LEE/
TANYA^25121200123456789025**XXX*****?·41111111111111=25121200XXXX00000000?*"
  }
}
```

```
}
}
```

Response

```
{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6473256016126696903006/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6473256016126696903006"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6473256016126696903006/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "OrderABC12"
  },
  "id": "6473256016126696903006",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "99.98",
      "currency": "usd"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "pointOfSaleInformation": {
    "processorInformation": {
      "approvalCode": "888888",
      "networkTransactionId": "123456789619999",
      "transactionId": "123456789619999",
      "systemTraceAuditNumber": "246802",
      "responseCode": "100"
    },
    "reconciliationId": "6580310700JCVR8J",
    "status": "AUTHORIZED",
    "submitTimeUtc": "2022-03-15T06:26:41Z"
  }
}
```

EMV Chip (Contactless) and Online PIN

For an EMV contactless payment, the customer taps the card on the terminal. The terminal and chip use NFC (near-field communication) to communicate with each other. For an online PIN transaction, the customer uses a PIN to verify their identity and the issuer verifies the PIN.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Fallback

EMV fallback is an option that allows you to use an alternative authorization method if the EMV authorization experiences a technical failure. To enable EMV fallback:

- Specify that EMV Fallback is allowed.
pointOfSaleInformation.emv.fallback=true
- Specify the condition that triggers EMV Fallback for American Express or Mastercard transactions.
pointOfSaleInformation.emv.fallbackCondition=<condition>
- Provide the service code from the track data for American Express or Mastercard transactions.
pointOfSaleInformation.serviceCode=<service_code>

Required Fields for an EMV Contactless Authorization with an Online PIN Using the REST API

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

pointOfSaleInformation.cardPresent

Set this field to **true**.

pointOfSaleInformation.encryptedPin

Required for online PIN transactions. Not used for offline PIN or signature transactions.

pointOfSaleInformation.entryMode

Set this field to **contactless** for an EMV payment. For an EMV fallback payment, set this field to **swiped** or **keyed**.

pointOfSaleInformation.tags

pointOfSaleInformation.trackData

processingInformation.commerceIndicator

Set this field to **retail**.

Country-Specific Required Fields for an EMV Credit Card Transaction Using the REST API

Argentina

merchantInformation.transactionLocalDateTime Required when the time zone is not set in your account.

processorInformation.salesSlipNumber

India

pointOfSaleInformation.terminalCompliance

Japan

processorInformation.salesSlipNumber

REST Example: EMV Chip (Contactless) with an Online PIN

Request

Endpoint POST <https://api.cybersource.com/pts/v2/payments>

```
{  
    "processingInformation": {  
        "commerceIndicator": "retail"  
    },  
    "orderInformation": {  
        "amountDetails": {  
            "totalAmount": "99.98",  
            "currency": "usd"  
        }  
    },  
    "pointOfSaleInformation": {  
        "cardPresent": "true",  
        "entryMode": "contactless",  
        "encryptedPin": "1234567812345678",  
        "tags":  
"9F3303204000095050000000000009F3704518823719F100706011103A0000009F26081E1756ED0E2134E29F3602001582020000"  
        "trackData": "%B4111111111111^LEE/  
TANYA^25121200123456789025**XXX*****?;41111111111111=25121200XXXX00000000?"  
    }  
}
```

Response

```
{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6473256016126696903006/reversals"
```

```

    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6473256016126696903006"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6473256016126696903006/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "OrderABC12"
  },
  "id": "6473256016126696903006",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "99.98",
      "currency": "usd"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "pointOfSaleInformation": {
    "processorInformation": {
      "approvalCode": "888888",
      "networkTransactionId": "123456789619999",
      "transactionId": "123456789619999",
      "systemTraceAuditNumber": "246802",
      "responseCode": "100"
    },
    "reconciliationId": "6580310700JCYR8J",
    "status": "AUTHORIZED",
    "submitTimeUtc": "2022-03-15T06:26:41Z"
  }
}

```

EMV Chip (Contactless) and Offline PIN

For an EMV contactless payment, the customer taps the card on the terminal. The terminal and chip use NFC (near-field communication) to communicate with each other. For an offline PIN transaction, the customer uses a PIN to verify their identity and the chip verifies the PIN.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Fallback

EMV fallback is an option that allows you to use an alternative authorization method if the EMV authorization experiences a technical failure. To enable EMV fallback:

- Specify that EMV Fallback is allowed.
pointOfSaleInformation.emv.fallback=true
- Specify the condition that triggers EMV Fallback for American Express or Mastercard transactions.
pointOfSaleInformation.emv.fallbackCondition=<condition>
- Provide the service code from the track data for American Express or Mastercard transactions.
pointOfSaleInformation.serviceCode=<service_code>

Required Fields for an EMV Contactless Authorization with an Offline PIN Using the REST API

clientReferenceInformation.code

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

pointOfSaleInformation.cardPresent Set this field to **Y**.

pointOfSaleInformation.emv.fallback Required only for a fallback EMV payment. Set this field to **Y**.

pointOfSaleInformation.emv.fallbackCondition Required only for a fallback EMV payment with American Express or Mastercard.

pointOfSaleInformation.entryMode Set this field to **contactless** for an EMV payment. For an EMV fallback payment, set this field to **swiped** or **keyed**.

pointOfSaleInformation.serviceCode Required only for a fallback EMV payment with American Express or Mastercard.

pointOfSaleInformation.tags

pointOfSaleInformation.terminalPinCapability Set this field to **0** for offline PIN transactions when the terminal does not support PINs.

pointOfSaleInformation.trackData

processingInformation.authorizationOptions.authType Set this field to **verbal**.

processingInformation.commerceIndicator Set this field to **retail**.

Country-Specific Required Fields for an EMV Credit Card Transaction Using the REST API

Argentina

merchantInformation.transactionLocalDateTime Required when the time zone is not set in your account.

processorInformation.salesSlipNumber

India

pointOfSaleInformation.terminalCompliance

Japan

processorInformation.salesSlipNumber

REST Example: EMV Chip (Contactless) with an Offline PIN

Request

Endpoint: POST <https://api.cybersource.com/pts/v2/payments>

```
{  
    "processingInformation": {  
        "commerceIndicator": "retail",  
        "authorizationOptions" {  
            "verbalAuthType": "verbal"  
        }  
    },  
    "orderInformation": {  
        "amountDetails": {  
            "totalAmount": "99.98",  
            "currency": "usd"  
        }  
    },  
    "pointOfSaleInformation": {  
        "cardPresent": "Y",  
        "entryMode": "contactless",  
        "tags":  
"9F33032040009505000000000009F3704518823719F100706011103A000009F26081E1756ED0E2134E29F3602001582020000"  
        "terminalCapability": "5"  
        "trackData": "%B4111111111111^LEE/  
TANYA^25121200123456789025**XXX*****?;41111111111111=25121200XXXXX00000000?"  
    }  
}
```

Response

```
{
  "_links": {
```

```

    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6473256016126696903006/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6473256016126696903006"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6473256016126696903006/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "OrderABC12"
  },
  "id": "6473256016126696903006",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "99.98",
      "currency": "usd"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "pointOfSaleInformation": {
    "processorInformation": {
      "approvalCode": "888888",
      "networkTransactionId": "123456789619999",
      "transactionId": "123456789619999",
      "systemTraceAuditNumber": "246802",
      "responseCode": "100"
    }
  },
  "reconciliationId": "6580310700JCYR8J",
  "status": "AUTHORIZED",
  "submitTimeUtc": "2022-03-15T06:26:41Z"
}

```

EMV Chip (Contactless with a Signature)

For an EMV contactless payment, the customer taps the card on the terminal. The terminal and chip use NFC (near-field communication) to communicate with each other. For a signature transaction, the customer uses a signature to verify their identity.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

EMV Fallback

EMV Fallback is an option that allows you to "fallback" to an alternative authorization method if EMV authorization fails due to a technical failure. To enable EMV fallback:

- Specify that EMV Fallback is allowed.
pointOfSaleInformation.emv.fallback=true
- Specify the condition that triggers EMV Fallback.
pointOfSaleInformation.emv.fallbackCondition=<condition>

Required Fields for an EMV Contactless Authorization with a Signature Using the REST API

clientReferenceInformation.code

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

pointOfSaleInformation.cardPresent Set this field to **Y**.

pointOfSaleInformation.emv.fallback Required only for a fallback EMV payment. Set this field to **Y**.

pointOfSaleInformation.emv.fallbackCondition Required only for a fallback EMV payment with American Express or Mastercard.

pointOfSaleInformation.entryMode Set this field to **contactless** for an EMV payment. For an EMV fallback payment, set this field to **swiped** or **keyed**.

pointOfSaleInformation.serviceCode Required only for a fallback EMV payment with American Express or Mastercard.

pointOfSaleInformation.tags

pointOfSaleInformation.trackData

processingInformation.commerceIndicator Set this field to **retail**.

Country-Specific Required Fields for an EMV Credit Card Transaction Using the REST API

Argentina

merchantInformation.transactionLocalDateTime Required when the time zone is not set in your account.

processorInformation.salesSlipNumber

India

pointOfSaleInformation.terminalCompliance

Japan

processorInformation.salesSlipNumber

REST Example: EMV Chip (Contactless) with a Signature

Request

Endpoint: POST <https://api.cybersource.com/pts/v2/payments>

[illegible]

Response

```
{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6473256016126696903006/reversals"
    }
  }
}
```

```

    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6473256016126696903006"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6473256016126696903006/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "OrderABC12"
  },
  "id": "6473256016126696903006",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "99.98",
      "currency": "usd"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "pointOfSaleInformation": {
    "processorInformation": {
      "approvalCode": "888888",
      "networkTransactionId": "123456789619999",
      "transactionId": "123456789619999",
      "systemTraceAuditNumber": "246802",
      "responseCode": "100"
    }
  },
  "reconciliationId": "6580310700JCVR8J",
  "status": "AUTHORIZED",
  "submitTimeUtc": "2022-03-15T06:26:41Z"
}

```


Payer Authentication Processing

Payer Authorization uses a two-step verification process to add an extra layer of fraud protection during the payment process. During transactions, information about the transaction device, location, past purchasing habits, and other factors are analyzed in real-time for indications of fraudulent activity. This process collects customer data during the transaction from two of these three categories:

- Something you have: A payment card or a payment card number
- Something you know: A password or pin
- Something you are: Facial recognition or fingerprint

Each payment card company has its own payer authentication product.

- American Express: SafeKey
- Discover: ProtectBuy
- JCB: J/Secure
- Mastercard: Identity Check
- Visa: 3D Secure

Payer Authentication can be used to satisfy the Strong Customer Authentication (SCA) requirement of the Payment Services Directive (PSD2). SCA applies to the European Economic Area (EEA) and the United Kingdom. SCA requires banks to perform additional checks when consumers make payments to confirm their identity.

For more information on Payer Authentication, see: [Payer Authentication Feature Guide](#).

Visa Secure

Visa Secure is the authentication service in the Visa card network that uses the 3-D Secure protocol to validate customers at checkout.

Visa Secure enables the exchange of data between the merchant, card issuer and, when necessary, the consumer, to validate that the owner of the account initiated the transaction. This authentication is a two-step process. First, the cardholder is

authenticated by 3-D Secure. Then the transaction is authorized based on the 3-D Secure evaluation. In this section, you will learn how to authorize a card payment based on the 3-D Secure evaluation.

Before implementing Visa Secure, contact customer support to have your account configured for this feature.

Fields Specific to this Use Case

These API fields are required specifically for this use case.

processingInformation.commerceIndicator	Set this field to vbv for a successful authentication or vbv_attempted if authentication was attempted but did not succeed.
consumerAuthenticationInformation.cavv	Required when payer authentication is successful.

Processor-Specific Requirements

These API fields are required specifically for certain processors.

orderInformation.amountDetails.currency

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Required Fields for an Authorization with Visa Secure

The following fields are required in a request for an authorization with Visa Secure:

clientReferenceInformation.code	
consumerAuthenticationInformation.cavv	Required when payer authentication is successful. Otherwise, this field is optional.
consumerAuthenticationInformation.xid	
orderInformation.amountDetails.currency	
orderInformation.amountDetails.totalAmount	
orderInformation.billTo.address1	
orderInformation.billTo.administrativeArea	
orderInformation.billTo.country	
orderInformation.billTo.email	
orderInformation.billTo.firstName	
orderInformation.billTo.lastName	
orderInformation.billTo.locality	

orderInformation.billTo.postalCode**paymentInformation.card.expirationMonth****paymentInformation.card.expirationYear****paymentInformation.card.number****paymentInformation.card.type**

processingInformation.commerceIndicator Set this field to **vbv** for a successful authentication or **vbv_attempted** if authentication was attempted but did not succeed.

Related information

- [API Field Reference for the REST API](#)

REST Example: Authorization with Visa Secure

Request

Endpoint:

```
{
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "processingInformation": {
    "commerceIndicator": "vbv"
  },
  "paymentInformation": {
    "card": {
      "number": "4111111111111111",
      "expirationMonth": "01",
      "expirationYear": "2023"
    }
  },
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "100",
      "currency": "USD"
    },
    "billTo": {
      "firstName": "John",
      "lastName": "Smith",
      "address1": "201 S. Division St._1",
      "locality": "Foster City",
      "administrativeArea": "CA",
      "postalCode": "94404",
      "country": "US",
      "email": "accept@cybs.com",
      "phoneNumber": "6504327113"
    }
  }
}
```

```

"consumerAuthenticationInformation": {
  "cavv": "AceY+igABPs3jdwNaDg3MAACAAA=",
  "xid": "1234567890987654321ABCDEFabcdefABCDEF123"
}
}

```

Response

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6758954108726900304951/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6758954108726900304951"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6758954108726900304951/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "id": "6758954108726900304951",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "100.00",
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "pointOfSaleInformation": {
    "terminalId": "111111"
  },
  "processorInformation": {
    "approvalCode": "888888",
    "networkTransactionId": "123456789619999",
    "transactionId": "123456789619999",
    "responseCode": "100",
    "avs": {
      "code": "X",

```

```

    "codeRaw": "I1"
  }
},
"reconciliationId": "711764833DU1FCQD",
"status": "AUTHORIZED",
"submitTimeUtc": "2023-02-08T22:30:11Z"
}

```

American Express SafeKey

American Express SafeKey (AESK) is the authentication service in the American Express card network that uses the 3-D Secure protocol to validate customers at checkout. When you request an authorization using a supported card type and a supported processor, you can include payer authentication data in the request. The payer authentication services enable you to add payer authentication support to your website without running additional software on your server.

Before implementing payer authentication for AESK, contact customer support to have your account configured for this feature.

Fields Specific to this Use Case

These API fields are required specifically for this use case.

processingInformation.commerceIndicator	Set this field to aesk for a successful authentication or aesk_attempted if authentication was attempted but did not succeed.
consumerAuthenticationInformation.cavv	Required when payer authentication is successful.

Processor-Specific Requirements

These API fields are required specifically for certain processors.

orderInformation.amountDetails.currency

processingInformation.authorizationOptions.requirement Required only for merchants in Saudi Arabia.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Related information

- Payer Authentication Using the REST API

Required Fields for an Authorization with American Express SafeKey

These fields must be included in a request for an authorization with AESK. The values for these fields are in the response from the payer authentication validate service. When you request the payer authentication validate and authorization services together, the data is automatically passed from one service to the other.

clientReferenceInformation.code

consumerAuthenticationInformation.cavv

consumerAuthenticationInformation.eciRaw Required when the payer authentication validation service returns a raw ECI value.

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

orderInformation.billTo.address1

orderInformation.billTo.administrativeArea

orderInformation.billTo.country

orderInformation.billTo.email

orderInformation.billTo.firstName

orderInformation.billTo.lastName

orderInformation.billTo.locality

orderInformation.billTo.postalCode

paymentInformation.card.expirationMonth

paymentInformation.card.expirationYear

paymentInformation.card.number

paymentInformation.card.type

processingInformation.commerceIndicator Set this field to `aesk` for a successful authentication or `aesk_attempted` if authentication was attempted but did not succeed.

Related information

- [API Field Reference for the REST API](#)

Optional Field for an Authorization with American Express SafeKey

This field is optional in a request for an authorization with AESK. The value for this field is in the response from the payer authentication validate service. When you request the payer authentication validate and authorization services together, the data is automatically passed from one service to the other.

consumerAuthenticationInformation.xid

Related information

- API Field Reference for the REST API

REST Example: Authorization with American Express SafeKey

Request

```
{
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "processingInformation": {
    "commerceIndicator": "aesk"
  },
  "paymentInformation": {
    "card": {
      "number": "372425119311008",
      "expirationMonth": "01",
      "expirationYear": "2026",
      "type": "003"
    }
  },
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "102.21",
      "currency": "USD"
    },
    "billTo": {
      "firstName": "John",
      "lastName": "Smith",
      "address1": "201 S. Division St._1",
      "locality": "Foster City",
      "administrativeArea": "CA",
      "postalCode": "94404",
      "country": "US",
      "email": "accept@cybs.com",
      "phoneNumber": "6504327113"
    }
  },
  "consumerAuthenticationInformation": {
    "cavv": "AceY+igABPs3jdwNaDg3MAACAAA="
  }
}
```

Response

```
{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6758955906616096904953/reversals"
    }
  },
}
```

```

"self": {
  "method": "GET",
  "href": "/pts/v2/payments/6758955906616096904953"
},
"capture": {
  "method": "POST",
  "href": "/pts/v2/payments/6758955906616096904953/captures"
}
},
"clientReferenceInformation": {
  "code": "TC50171_3"
},
"id": "6758955906616096904953",
"orderInformation": {
  "amountDetails": {
    "authorizedAmount": "102.21",
    "currency": "USD"
  }
},
"paymentAccountInformation": {
  "card": {
    "type": "003"
  }
},
"paymentInformation": {
  "accountFeatures": {
    "currency": "usd",
    "balanceAmount": "70.00"
  },
  "tokenizedCard": {
    "type": "003"
  },
  "card": {
    "type": "003"
  }
},
"pointOfSaleInformation": {
  "terminalId": "144111"
},
"processorInformation": {
  "approvalCode": "888888",
  "networkTransactionId": "123456789619999",
  "transactionId": "123456789619999",
  "responseCode": "100",
  "avs": {
    "code": "X",
    "codeRaw": "I1"
  }
},
"reconciliationId": "711770243DU1FCXF",
"status": "AUTHORIZED",
"submitTimeUtc": "2023-02-08T22:33:10Z"
}

```


JCB J/Secure

JCB J/Secure is the authentication service in the JCB card network that uses the 3-D Secure protocol to validate customers at checkout. When you request an authorization using a supported card type and a supported processor, you can include payer authentication data in the request. The payer authentication services enable you to add payer authentication support to your website without running additional software on your server. JCB J/Secure reduces the risk of unauthorized use of a payment card account by adding a password identification step to the shopping process. Before you implement payer authentication for JCB J/Secure, you must contact customer support to have your account configured for this feature.

Fields Specific to this Use Case

These API fields are required specifically for this use case.

processingInformation.commerceIndicator	Set this field to <code>js</code> for a successful authentication or <code>js_attempted</code> if authentication was attempted but did not succeed.
consumerAuthenticationInformation.cavv	Required when payer authentication is successful.
xid	Required when payer authentication is successful.
eci_raw	Required when the payer authentication validation service returns a raw ECI value.

Processor-Specific Requirements

These API fields are required specifically for certain processors.

orderInformation.amountDetails.currency

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Related information

- Payer Authentication Using the REST API

Required Fields for an Authorization using JCB J/Secure Authentication

The following fields are required in a request for an authorization with JCB J/Secure:

clientReferenceInformation.code
consumerAuthenticationInformation.cavv
consumerAuthenticationInformation.xid
orderInformation.amountDetails.currency
orderInformation.amountDetails.totalAmount
orderInformation.billTo.address1
orderInformation.billTo.administrativeArea
orderInformation.billTo.country
orderInformation.billTo.email
orderInformation.billTo.firstName
orderInformation.billTo.lastName
orderInformation.billTo.locality
orderInformation.billTo.postalCode
paymentInformation.card.expirationMonth
paymentInformation.card.expirationYear
paymentInformation.card.number
paymentInformation.card.type
processingInformation.commerceIndicator Set this field to `js` for a successful authentication or `js_attempted` if authentication was attempted but did not succeed.

REST Example: Authorization with JCB J/Secure

Request

```

{
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "processingInformation": {
    "commerceIndicator": "internet"
  },
  "paymentInformation": {
    "card": {
      "number": "3566111111111113",
      "expirationMonth": "01",
      "expirationYear": "2026"
    }
  },
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "100",

```

```

    "currency": "USD"
  },
  "billTo": {
    "firstName": "John",
    "lastName": "Smith",
    "address1": "201 S. Division St._1",
    "address2": "",
    "locality": "Foster City",
    "administrativeArea": "CA",
    "postalCode": "94404",
    "country": "US",
    "email": "accept@cybs.com",
    "phoneNumber": "6504327113"
  },
  "consumerAuthenticationInformation": {
    "cavv": "AceY+igABPs3jdwNaDg3MAACAAA=",
    "xid": "1234567890987654321ABCDEFabcdefABCDEF123"
  }
}

```

Response

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6758966324476176704951/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6758966324476176704951"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6758966324476176704951/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "id": "6758966324476176704951",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "100.00",
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "007"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "007"
    }
  }
}

```

```

},
"card": {
  "type": "007"
},
},
"pointOfSaleInformation": {
  "terminalId": "111111"
},
},
"processorInformation": {
  "approvalCode": "888888",
  "responseCode": "100",
  "avs": {
    "code": "X",
    "codeRaw": "I1"
  }
},
},
"reconciliationId": "71177738FDU0YQC9",
"status": "AUTHORIZED",
"submitTimeUtc": "2023-02-08T22:50:32Z"
}

```

Mastercard Identity Check

Mastercard Identity Check is the authentication service in the Mastercard card network that uses the 3-D Secure protocol to validate customers at checkout.

Mastercard Identity Check adds security to online transactions by authenticating Identity Check account holders for specific transactions. Identity Check generates a unique, 32-character transaction token, called the account authentication value (AAV), each time an Identity Check-enabled account holder makes an online purchase. The AAV binds the account holder to a specific transaction. Identity Check transactions use the universal cardholder authentication field (UCAF) as a standard to collect and pass AAV data. For details about signing up for and using Identity Check or UCAF, contact your acquiring bank or go to the Mastercard website.

Before implementing payer authentication for Mastercard Identity Check, contact customer support to have your account configured for this feature.

Fields Specific to this Use Case

These API fields are required specifically for this use case.

`consumerAuthenticationInformation.authenticationTransactionIdSet` this field to the transaction ID returned by Identity Check during the authentication process.

`consumerAuthenticationInformation.paSpecificationVersionSet` this field to the Identity Check version returned by Identity Check during the authentication process.

`consumerAuthenticationInformation.ucafCollectionIndicatorSet` this value to the Collection Indicator returned by Identity Check during the authentication process.

`processingInformation.commerceIndicatorSet` this value to `spa` for an authentication that was successful or was attempted but did not process.

Processor-Specific Requirements

These API fields are required specifically for certain processors.

orderInformation.amountDetails.currency

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Required Fields for an Authorization with Mastercard Identity Check

The following fields are required in a request for an authorization with Mastercard Identity Check:

consumerAuthenticationInformation.authenticationTransactionId

consumerAuthenticationInformation.paSpecificationVersion

consumerAuthenticationInformation.ucafCollectionIndicator

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

orderInformation.billTo.address1

orderInformation.billTo.administrativeArea

orderInformation.billTo.country

orderInformation.billTo.email

orderInformation.billTo.firstName

orderInformation.billTo.lastName

orderInformation.billTo.locality

orderInformation.billTo.postalCode

paymentInformation.card.expirationMonth

paymentInformation.card.expirationYear

paymentInformation.card.number

processingInformation.commerceIndicator Set this field to **spa** for an authentication that was successful or was attempted but did not succeed.

Related information

- [API Field Reference for the REST API](#)

REST Example: Authorizing Mastercard Identity Check

Request

```
{
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "processingInformation": {
    "commerceIndicator": "spa"
  },
  "paymentInformation": {
    "card": {
      "number": "5555555555554444",
      "expirationMonth": "01",
      "expirationYear": "2026"
    }
  },
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "100",
      "currency": "USD"
    },
    "billTo": {
      "firstName": "John",
      "lastName": "Smith",
      "address1": "201 S. Division St._1",
      "locality": "Foster City",
      "administrativeArea": "CA",
      "postalCode": "94404",
      "country": "US",
      "email": "accept@cybs.com",
      "phoneNumber": "6504327113"
    }
  },
  "consumerAuthenticationInformation": {
    "ucafCollectionIndicator": "1",
    "paSpecificationVersion": "1",
    "authenticationTransactionId": "OiCtXA1j1AxtSNDh5lt1"
  }
}
```

Response

```
{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6758990751436655004951/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6758990751436655004951"
    },
    "capture": {
      "method": "POST",

```

```

    "href": "/pts/v2/payments/6758990751436655004951/captures"
  }
},
"clientReferenceInformation": {
  "code": "TC50171_3"
},
"id": "6758990751436655004951",
"orderInformation": {
  "amountDetails": {
    "authorizedAmount": "100.00",
    "currency": "USD"
  }
},
"paymentAccountInformation": {
  "card": {
    "type": "002"
  }
},
"paymentInformation": {
  "tokenizedCard": {
    "type": "002"
  },
  "card": {
    "type": "002"
  }
},
"pointOfSaleInformation": {
  "terminalId": "111111"
},
"processorInformation": {
  "approvalCode": "888888",
  "authIndicator": "1",
  "networkTransactionId": "123456789619999",
  "transactionId": "123456789619999",
  "responseCode": "100",
  "avs": {
    "code": "X",
    "codeRaw": "I1"
  }
},
"reconciliationId": "71183995FDU0YRTK",
"status": "AUTHORIZED",
"submitTimeUtc": "2023-02-08T23:31:15Z"
}

```

Authorization with an SCA Exemption

This section demonstrates how to process an authorization with a strong customer authentication (SCA) exemption. Merchants cannot decide whether an exemption applies to a transaction. Only the card issuing bank can grant a SCA exemption during card authentication.

A SCA exemption enables you to remain in compliance with the European Union's second Payment Services Directive. Depending on your processor, use one of the following exemption fields:



Important

If you send more than one SCA exemption field with a single authentication, the transaction is denied.

- **Authentication Outage:** Payer authentication is not available for this transaction due to a system outage.
- **B2B Corporate Card:** Payment cards specifically for business-to-business transactions are exempt.
- **Delegated Authentication:** Payer authentication was performed outside of the authorization workflow.
- **Follow-On Installment Payment:** Installment payments of a fixed amount are exempt after the first transaction.
- **Follow-On Recurring Payment:** Recurring payments of a fixed amount are exempt after the first transaction.
- **Low Risk:** The average fraud levels associated with this transaction are considered low.
- **Low Value:** The transaction value does not warrant SCA.
- **Merchant Initiated Transactions:** As follow-on transactions, merchant-initiated transactions are exempt.
- **Stored Credential Transaction:** Credentials are authenticated before storing, so stored credential transactions are exempt.
- **Trusted Merchant:** Merchants registered as trusted beneficiaries.

Exemption Fields Specific to This Use Case

Use one of these fields to request an SCA exemption:

Exemption Type	Field	Value
Authentication Outage	consumerAuthenticationInformation. strongAuthentication. authenticationOutageExemptionIndicator	1
B2B Corporate Card Transaction	consumerAuthenticationInformation. strongAuthentication. secureCorporatePaymentIndicator	1
Delegated Authentication	consumerAuthenticationInformation. strongAuthentication. delegatedAuthenticationExemptionIndicator	1
Low-Risk Transaction	consumerAuthenticationInformation. strongAuthentication. riskAnalysisExemptionIndicator	1

Exemption Type	Field	Value
Low-Value Transaction	consumerAuthenticationInformation.	1
	strongAuthentication.	
	lowValueExemptionIndicator	
Stored Credential Transaction	processingInformation.	1
	authorizationOptions.initiator.	
	storedCredentialUsed	
Trusted Merchant Transaction	consumerAuthenticationInformation.	1
	strongAuthentication.	
	trustedMerchantExemptionIndicator	

Country-Specific Requirements

These fields are specific to certain countries and regions.

Argentina

merchantInformation.taxId Required for Mastercard transactions.

merchantInformation.transactionLocalDateTime Required when the time zone is not included in your account. Otherwise, this field is optional.

Brazil

paymentInformation.card.sourceAccountType Required for combo card transactions.

paymentInformation.card.sourceAccountType Required for combo card line-of-credit and prepaid-card transactions.

Chile

merchantInformation.taxId Required for Mastercard transactions.

Paraguay

merchantInformation.taxId Required for Mastercard transactions.

Saudi Arabia

processingInformation.authorizationOptions.transactionMode

Taiwan

paymentInformation.card.hashNumber

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Required Fields for an Authorization with an SCA Exemption

You must also include one SCA exemption field from the [SCA Exemption Fields](#) list.

orderInformation.amountDetails.currency
orderInformation.amountDetails.totalAmount
orderInformation.billTo.address1
orderInformation.billTo.administrativeArea
orderInformation.billTo.country
orderInformation.billTo.email
orderInformation.billTo.firstName
orderInformation.billTo.lastName
orderInformation.billTo.locality
orderInformation.billTo.postalCode
paymentInformation.card.expirationMonth
paymentInformation.card.expirationYear
paymentInformation.card.number
paymentInformation.card.type

REST Example: Authorization with an SCA Exemption for Low Value Transactions

Request

Endpoint: POST <https://api.cybersource.com/pts/v2/payments>

```
{
  "consumerAuthenticationInformation": {
    "strongAuthentication": {
      "lowValueExemptionIndicator": "1"
    }
  },
  "orderInformation": {
    "billTo": {
      "country": "US",
      "lastName": "Kim",
      "address1": "201 S. Division St.",
      "postalCode": "48104-2201",
      "locality": "Ann Arbor",
      "administrativeArea": "MI",
      "firstName": "Kyong-Jin",
      "email": "test@cybs.com"
    },
    "amountDetails": {
      "totalAmount": "100.00",
      "currency": "eur"
    }
  }
}
```

```

    }
  },
  "paymentInformation": {
    "card": {
      "expirationYear": "2031",
      "number": "4111111111111111",
      "expirationMonth": "12"
    }
  }
}

```

Response

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6709780221406171803955/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6709780221406171803955"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6709780221406171803955/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "1670978022258"
  },
  "id": "6709780221406171803955",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "100.00",
      "currency": "eur"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "pointOfSaleInformation": {
    "terminalId": "123456"
  },
  "processorInformation": {
    "approvalCode": "888888",

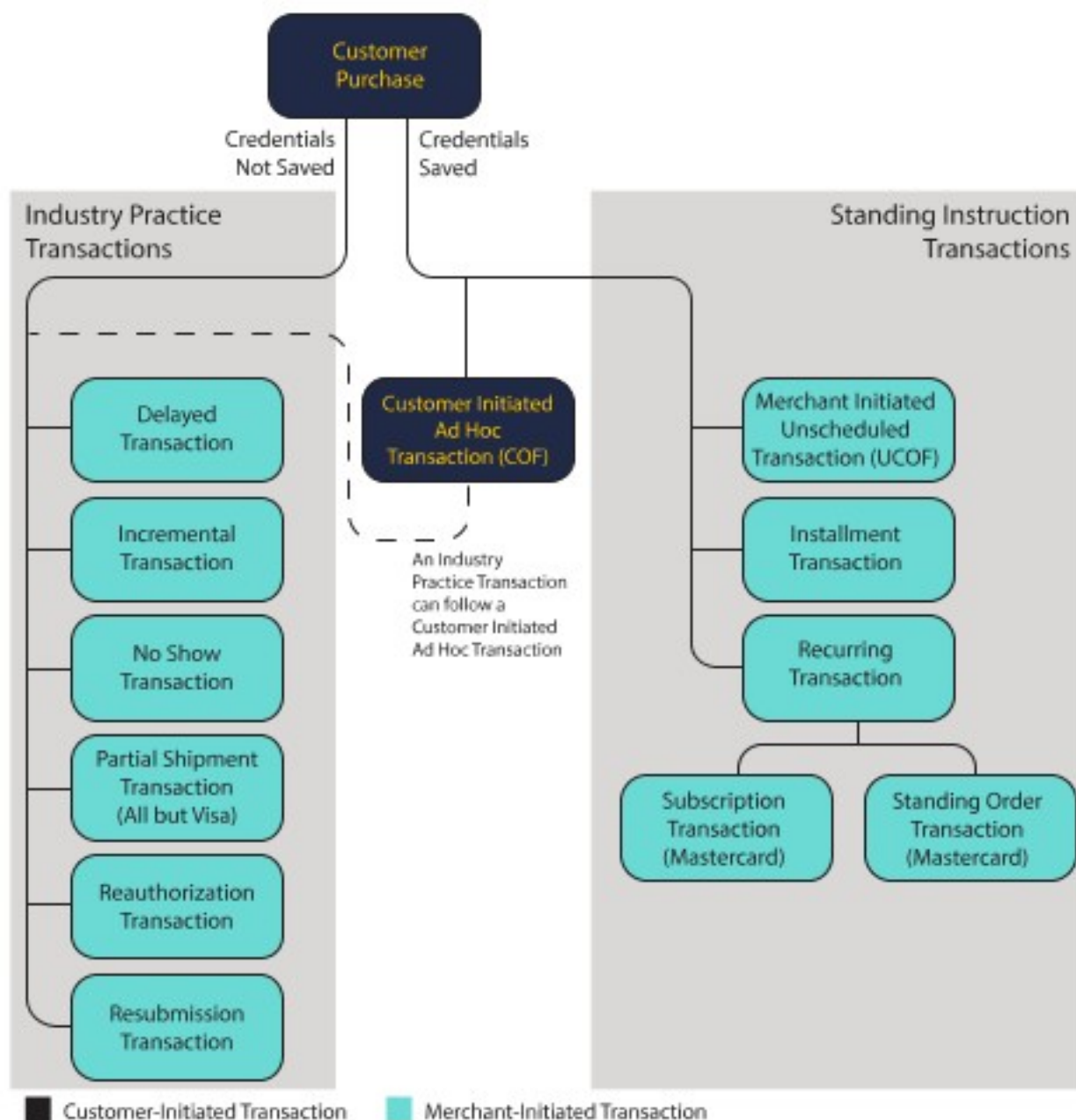
```

```
"networkTransactionId": "123456789619999",  
"transactionId": "123456789619999",  
"responseCode": "100",  
"avs": {  
  "code": "X",  
  "codeRaw": "I1"  
}  
},  
"reconciliationId": "62859554PBDEMI43",  
"status": "AUTHORIZED",  
"submitTimeUtc": "2022-12-14T00:33:42Z"  
}
```

Stored Credential Processing

There are several types of credentialed transactions:

- Customer-Initiated Transactions: Ad hoc customer-initiated transactions that use stored credentials.
- Merchant-Initiated Transactions:
 - Industry Practice Transactions: Merchant-initiated transactions that do not require stored credentials.
 - Standing Instruction Transactions: Merchant-initiated transactions that require stored credentials and customer consent to execute.



Customer-Initiated Transactions with Credentials on File

A customer-initiated transaction (CIT) is a transaction initiated by the customer. There are two types of CIT transactions:

- Customer transactions where the credentials are stored for future merchant initiated transactions
- Customer transactions where the credentials are stored for future customer initiated transaction

Customers can initiate a CIT at a merchant payment terminal, through an online purchase transaction, or by making a purchase using a previously stored credential.

Storing Customer Credentials with a Customer-Initiated Transaction

Before you can perform a merchant-initiated transaction (MIT) or a customer-initiated transaction (CIT) with credentials-on-file (COF), you must store the customer's credentials for later use. Further, before you can store the user's credentials, you must get the customer's consent to store their private information. This is also known as establishing a relationship with the customer.

Fields Specific to this Use Case

Include this field with a standard authorization request when storing customer credentials:

- **processingInformation.authorizationOptions.initiator.credentialStoredOnFile**—Set this field to `true` to indicate that the customer credentials will be stored for future use.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Required Fields

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

orderInformation.billTo.address1

orderInformation.billTo.administrativeArea

orderInformation.billTo.country

orderInformation.billTo.email

orderInformation.billTo.firstName

orderInformation.billTo.lastName

orderInformation.billTo.locality

orderInformation.billTo.phoneNumber

orderInformation.billTo.postalCode

paymentInformation.card.expirationMonth

paymentInformation.card.expirationYear

paymentInformation.card.number

processingInformation.authorizationOptions.initiator.credentialStoredOnFile—Set to `true`

REST Example: Storing Customer Credentials During a Customer-Initiated Transaction

Request

Endpoint: POST <https://api.cybersource.com/pts/v2/payments>

```
{
  "processingInformation": {
    "authorizationOptions": {
      "initiator": {
        "credentialStoredOnFile": "true"
      }
    }
  },
  "orderInformation": {
    "billTo": {
      "firstName": "John",
      "lastName": "Deo",
      "address1": "201 S. Division St.",
      "postalCode": "48104-2201",
      "locality": "Ann Arbor",
      "administrativeArea": "MI",
      "country": "US",
      "email": "test@cybs.com",
      "phoneNumber": "6504327113"
    },
    "amountDetails": {
      "totalAmount": "100.00",
      "currency": "USD"
    }
  },
  "paymentInformation": {
    "card": {
      "expirationYear": "2031",
      "number": "4111xxxxxxxxxx",
      "expirationMonth": "12"
    }
  }
}
```

Response

```
{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6528187198946076303004/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6528187198946076303004"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6528187198946076303004/captures"
    }
  }
}
```



```

    }
  },
  "clientReferenceInformation": {
    "code": "1652818719876"
  },
  "id": "6528187198946076303004",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "100.00",
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "pointOfSaleInformation": {
    "terminalId": "111111"
  },
  "processorInformation": {
    "approvalCode": "888888",
    "networkTransactionId": "123456789619999",
    "transactionId": "123456789619999",
    "responseCode": "100",
    "avs": {
      "code": "X",
      "codeRaw": "I1"
    }
  },
  "reconciliationId": "63165088Z3AHV91G",
  "status": "AUTHORIZED",
  "submitTimeUtc": "2022-05-17T20:18:40Z"
}

```

Using Stored Customer Credentials During a Customer-Initiated Transaction

After customers store their credentials on file, they can recall these credentials to use with subsequent transactions.

Fields Specific to this Use Case

When a customer-initiated transaction (CIT) uses stored credentials, the transaction must include the following information in the authorization request:

- **processingInformation.authorizationOptions.initiator.storedCredentialUsed**—Set the field to `true` to indicate that the previously stored customer credentials were used.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Required Fields

orderInformation.amountDetails.currency
orderInformation.amountDetails.totalAmount
orderInformation.billTo.address1
orderInformation.billTo.administrativeArea
orderInformation.billTo.country
orderInformation.billTo.email
orderInformation.billTo.lastName
orderInformation.billTo.locality
orderInformation.billTo.phoneNumber
orderInformation.billTo.postalCode
paymentInformation.card.expirationMonth
paymentInformation.card.expirationYear
paymentInformation.card.number
processingInformation.authorizationOptions.initiator.storedCredentialUsed—Set field to `true`

Card-Specific Required Fields

Some card companies require additional information when making authorizations with stored credentials.

Discover

Discover requires the authorization amount from the original transaction when sending a request:

processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.originalAuthorizedAmount

REST Example: Retrieving Customer Credentials During a Customer-Initiated Transaction

Request

Endpoint: POST <https://api.cybersource.com/pts/v2/payments>

```
{
```

```

"processingInformation": {
  "authorizationOptions": {
    "initiator": {
      "storedCredentialUsed": "true"
    }
  }
},
"orderInformation": {
  "billTo": {
    "firstName": "John",
    "lastName": "Deo",
    "address1": "201 S. Division St.",
    "postalCode": "48104-2201",
    "locality": "Ann Arbor",
    "administrativeArea": "MI",
    "country": "US",
    "email": "test@cybs.com",
    "phoneNumber": "6504327113"
  },
  "amountDetails": {
    "totalAmount": "100.00",
    "currency": "USD",
    "originalAmount": "100"
    // Discover card Only
  }
},
"paymentInformation": {
  "card": {
    "expirationYear": "2031",
    "number": "4111xxxxxxxxxxx",
    "expirationMonth": "12"
  }
},
"processorInformation": {
  "transactionId": "12345678961000"
}
}

```

Response

```

},
"paymentAccountInformation": {
  "card": {
    "type": "002"
  }
},
"paymentInformation": {
  "tokenizedCard": {
    "type": "002"
  },
  "card": {
    "type": "002"
  }
},
"pointOfSaleInformation": {
  "terminalId": "111111"
}

```

```

},
"processorInformation": {
  "approvalCode": "888888",
  "authIndicator": "1",
  "networkTransactionId": "123456789619999",
  "transactionId": "123456789619999",
  "responseCode": "100",
  "avs": {
    "code": "X",
    "codeRaw": "I1"
  }
},
"reconciliationId": "63740353AJ2NSH",
"status": "AUTHORIZED",
"submitTimeUtc": "2022-05-20T19:13:06Z"
}

```

Merchant-Initiated Transactions

Merchants can initiate a payment on the behalf of a customer. This type of transaction is called a merchant-initiated transaction (MIT). When initiating a MIT, the customer is not present. However, customers must authorize the storage of their credentials and the use of these credentials for future payments.

There are two types of MITs:

- Industry Practice transactions: Follow-on transactions to a customer-initiated transaction.
- Standing Instruction transactions: Agreed upon standing instructions from the customer for the provision of goods or services. For example, a subscription to an internet music service may involve an agreed upon standing instruction allowing the merchant to bill a customer monthly for their subscription.

Industry Practice Transactions

Industry practice transactions are follow-on transactions to a previous customer-initiated transaction. There are six types of Industry Practice transactions:

- Delayed charges
- Incremental charges
- Partial authorizations or partial shipments
- Reauthorizations
- Resubmissions
- No-shows

Merchant-Initiated Delayed Transaction

Delayed charge transaction is performed to process a supplemental account charge after original services have been rendered and respective payment has been processed.

Fields Specific to this Use Case

The following information is required with a standard authorization request:

- **processingInformation.authorizationOptions.initiator.type**—Indicate that this transaction is a merchant-initiated follow-on transaction. Set the field to `merchant`.
- **processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.reason**—Include the reason for the transaction. Set the field to 2.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Required Fields

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount**orderInformation.billTo.address1**

orderInformation.billTo.administrativeArea

orderInformation.billTo.country**orderInformation.billTo.email****orderInformation.billTo.firstName****orderInformation.billTo.lastName****orderInformation.billTo.locality****orderInformation.billTo.phoneNumber****orderInformation.billTo.postalCode**

paymentInformation.card.expirationMonth

paymentInformation.card.expirationYear

paymentInformation.card.number

processingInformation.authorizationOptions.FinancialCover and American Express

merchantInitiatedTransaction.previousTransactionId Use the transaction ID from the original transaction. For Visa, use the last successful transaction ID.

processingInformation.authorizationOptions.setTabc2:

merchantInitiatedTransaction.reason

```
processingInformation.authorizationOptions.SetInfo(merchant,
```

Card-Specific Required Fields

Discover

The listed card requires additional field:

processingInformation.authorizationOptions.Prior to the original transaction, originalAuthorization

REST Example: Merchant-Initiated Delayed Authorization Transaction

Request

Endpoint: **POST** <https://api.cybersource.com/pts/v2/payments>

```
{
  "orderInformation": {
    "billTo": {
      "country": "US",
      "lastName": "Kim",
      "address1": "201 S. Division St.",
      "postalCode": "48104-2201",
      "locality": "Ann Arbor",
      "administrativeArea": "MI",
      "firstName": "Kyong-Jin",
      "phoneNumber": "6504327113",
      "email": "test@cybs.com"
    },
    "amountDetails": {
      "totalAmount": "120.00",
      "currency": "USD"
    }
  },
  "paymentInformation": {
    "card": {
      "expirationYear": "2031",
      "number": "4111xxxxxxxxxx",
      "expirationMonth": "12"
    }
  },
  "processingInformation": {
    "authorizationOptions": {
      "initiator": {
        "type": "merchant",
        "merchantInitiatedTransaction": {
          "originalAuthorizedAmount": "100",
          // Discover only
          "previousTransactionId": "123456789619999",
          "reason": "2"
        }
      }
    }
  }
}
```

This example shows a sample response for a merchant-initiated delayed transaction.

Response

```
{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6534213653516599003001/reversals"
    },
    "self": {
      "method": "GET",

```

```

    "href": "/pts/v2/payments/6534213653516599003001"
  },
  "capture": {
    "method": "POST",
    "href": "/pts/v2/payments/6534213653516599003001/captures"
  }
},
"clientReferenceInformation": {
  "code": "1653421365327"
},
"id": "6534213653516599003001",
"orderInformation": {
  "amountDetails": {
    "authorizedAmount": "120.00",
    "currency": "USD"
  }
},
"paymentAccountInformation": {
  "card": {
    "type": "002"
  }
},
"paymentInformation": {
  "tokenizedCard": {
    "type": "002"
  },
  "card": {
    "type": "002"
  }
},
"pointOfSaleInformation": {
  "terminalId": "111111"
},
"processorInformation": {
  "approvalCode": "888888",
  "authIndicator": "1",
  "networkTransactionId": "123456789619999",
  "transactionId": "123456789619999",
  "responseCode": "100",
  "avs": {
    "code": "X",
    "codeRaw": "I1"
  }
},
"reconciliationId": "64365475T3K10Q1D",
"status": "AUTHORIZED",
"submitTimeUtc": "2022-05-24T19:42:45Z"
}

```

Merchant-Initiated Incremental Transaction

An incremental authorization is used to increase the total amount authorized for a payment if the initial authorization does not cover the total cost of goods and services. An incremental transaction is an additional amount to the original authorization. The final authorized total includes amounts for both the initial and the incremental authorizations.

Fields Specific to this Use Case

Include these fields with a standard authorization request:

- **processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.type**–Set the field to `merchant` to indicate this transaction is a merchant-initiated follow-on transaction.
- **processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.reason**–Set the reason field to `5`.
- **processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.previousTransactionID**– Provide the original transaction ID.

Endpoint

Send a `PATCH` request to this endpoint:

`pts/v2/payments/{id}` where `id` is the original authorization ID.

Required Fields

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.reason

processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.type–Set to `merchant`.

Card-Specific Required Fields

Discover

The listed card requires additional field:

processingInformation.authorizationOptions.previousTransactionID– Provide the original transaction ID.

REST Example: Merchant-Initiated Incremental Transaction

Send a `PATCH` request to this endpoint:

`pts/v2/payments/{id}` where `id` is the original authorization ID.

Request

```
{
  "orderInformation": {
    "billTo": {
      "country": "US",
      "lastName": "Kim",
      "address1": "201 S. Division St.",
      "postalCode": "48104-2201",
      "locality": "Ann Arbor",
      "administrativeArea": "MI",
      "firstName": "Kyong-Jin",
      "phoneNumber": "6504327113",
      "email": "test@cybs.com"
    },
    "amountDetails": {
      "totalAmount": "120.00",
```



```

    "currency": "USD"
  }
},
"paymentInformation": {
  "card": {
    "expirationYear": "2031",
    "number": "4111xxxxxxxxxx",
    "expirationMonth": "12"
  }
},
"processingInformation": {
  "authorizationOptions": {
    "initiator": {
      "type": "merchant",
      "merchantInitiatedTransaction": {
        "originalAuthorizedAmount": "100",
        // Required for Discover
        "previousTransactionId": "123456789619999",
        "reason": "5"
      }
    }
  }
}
}
}

```

This example shows a sample response for a merchant-initiated incremental transaction.
Example: Response

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6533225006556860003002/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6533225006556860003002"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6533225006556860003002/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "1653322500637"
  },
  "id": "6533225006556860003002",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "100.00",
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  }
}

```

```

    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "pointOfSaleInformation": {
    "terminalId": "111111"
  },
  "processorInformation": {
    "approvalCode": "888888",
    "networkTransactionId": "123456789619999",
    "transactionId": "123456789619999",
    "responseCode": "100",
    "avs": {
      "code": "X",
      "codeRaw": "I1"
    }
  },
  "reconciliationId": "64143477A3AJ4P2Z",
  "status": "AUTHORIZED",
  "submitTimeUtc": "2022-05-23T16:15:00Z"
}

```

Merchant-Initiated Reauthorization Transactions

A reauthorization occurs when the completion or fulfillment of the original order or service extends beyond the authorized amount time limit. There are two common reauthorization scenarios:

- Split or delayed shipments by a retailer
- Extended car rentals, hotel stays, or cruise line bookings

Fields Specific to this Use Case

Include these fields with a standard authorization request:

- **processingInformation.authorizationOptions.initiator.type**—Set the field to **merchant** to indicate this transaction is a merchant-initiated follow-on transaction.
- **processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.reason**—Set the reason field to **3**.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Required Fields

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

orderInformation.billTo.address1**orderInformation.billTo.administrativeArea****orderInformation.billTo.country****orderInformation.billTo.email****orderInformation.billTo.firstName****orderInformation.billTo.lastName****orderInformation.billTo.locality****orderInformation.billTo.phoneNumber****orderInformation.billTo.postalCode****paymentInformation.card.expirationMonth****paymentInformation.card.expirationYear****paymentInformation.card.number****processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.****previousTransactionId**

cards, use the transaction ID from the original transaction. For Visa, use the last successful transaction ID.

processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.reason**processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.reason**

Card-Specific Required Fields

Discover

The listed card requires additional field:

processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.previousTransactionId

REST Example: Merchant-Initiated Reauthorization Transactions

Request

Endpoint: **POST https://api.cybersource.com/pts/v2/payments**

```
{
  "processingInformation": {
    "authorizationOptions": {
      "initiator": {
        "type": "merchant",
        "merchantInitiatedTransaction": {
          "originalAuthorizedAmount": "100", // Discover Only
          "previousTransactionId": "123456789619999",
          "reason": "3"
        }
      }
    }
  },
  "orderInformation": {
```

```

"billTo": {
  "country": "US",
  "lastName": "Kim",
  "address1": "201 S. Division St.",
  "postalCode": "48104-2201",
  "locality": "Ann Arbor",
  "administrativeArea": "MI",
  "firstName": "Kyong-Jin",
  "phoneNumber": "6504327113",
  "email": "test@cybs.com"
},
"amountDetails": {
  "totalAmount": "130.00",
  "currency": "USD"
}
},
"paymentInformation": {
  "card": {
    "expirationYear": "2031",
    "number": "4111xxxxxxxxxx",
    "expirationMonth": "12"
  }
}
}
}

```

Response

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6541178668686490403003/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6541178668686490403003"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6541178668686490403003/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "1654117866849"
  },
  "id": "6541178668686490403003",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "130.00",
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  }
}

```

```

},
"paymentInformation": {
  "tokenizedCard": {
    "type": "001"
  },
  "card": {
    "type": "001"
  }
},
"pointOfSaleInformation": {
  "terminalId": "111111"
},
"processorInformation": {
  "approvalCode": "888888",
  "networkTransactionId": "123456789619999",
  "transactionId": "123456789619999",
  "responseCode": "100",
  "avs": {
    "code": "X",
    "codeRaw": "I1"
  }
},
"reconciliationId": "65313868D3TXXC05",
"status": "AUTHORIZED",
"submitTimeUtc": "2022-06-01T21:11:06Z"
}

```

Merchant-Initiated Resubmission Transaction

A resubmission transaction is used when a merchant resubmits an authorization to recover an outstanding debt from the customer. A common scenario is when a card was initially declined due to insufficient funds, but the goods or services were already delivered to the customer.

Specific Fields for this Use Case

Include these fields with a standard authorization request:

- **processingInformation.authorizationOptions.initiator.type**–
- Set the field to **merchant** to indicate this transaction is a merchant-initiated follow-on transaction.
- **processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.reason**– Set the reason field to **1**.

Required Fields

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

orderInformation.billTo.address1

orderInformation.billTo.administrativeArea

orderInformation.billTo.country

orderInformation.billTo.email**orderInformation.billTo.firstName****orderInformation.billTo.lastName****orderInformation.billTo.locality****orderInformation.billTo.phoneNumber****orderInformation.billTo.postalCode****paymentInformation.card.expirationMonth****paymentInformation.card.expirationYear****paymentInformation.card.number****processingInformation.authorizationOptions.ForDiscoverAndInitiatedTransaction.****previousTransactionId** cards, use the transaction ID from the original transaction. For Visa, use the last successful transaction ID.**processingInformation.authorizationOptions.Settlement.merchantInitiatedTransaction.reason****processingInformation.authorizationOptions.Settlement.merchant.**

Card-Specific Required Fields

Discover

The listed card requires additional field:

processingInformation.authorizationOptions.PreviousTransactionId Provide the original transaction's originalAuthorizationId.

REST Example: Merchant-Initiated Resubmitted Authorization Transactions

Request

Endpoint: **POST https://api.cybersource.com/pts/v2/payments**

```
{
  "processingInformation": {
    "authorizationOptions": {
      "initiator": {
        "type": "merchant",
        "merchantInitiatedTransaction": {
          "originalAuthorizedAmount": "100", // Discover Only
          "previousTransactionId": "123456789619999",
          "reason": "1"
        }
      }
    }
  },
  "orderInformation": {
    "billTo": {
      "country": "US",
      "lastName": "Kim",
      "address1": "201 S. Division St.",
      "postalCode": "48104-2201",

```

```

    "locality" : "Ann Arbor",
    "administrativeArea" : "MI",
    "firstName" : "Kyong-Jin",
    "phoneNumber" : "6504327113",
    "email" : "test@cybs.com"
  },
  "amountDetails": {
    "totalAmount": "100.00",
    "currency": "USD"
  }
},
"paymentInformation": {
  "card": {
    "expirationYear": "2031",
    "number": "4111xxxxxxxxxx",
    "expirationMonth": "12"
  }
}
}
}

```

This example shows a sample response for a resubmitted merchant-initiated authorization transaction.

Response

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6534232293716260503006/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6534232293716260503006"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6534232293716260503006/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "1653423229353"
  },
  "id": "6534232293716260503006",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "100.00",
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "004"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {

```

```

    "type": "004"
  },
  "card": {
    "type": "004"
  }
},
"pointOfSaleInformation": {
  "terminalId": "111111"
},
"processorInformation": {
  "approvalCode": "888888",
  "networkTransactionId": "123456789619999",
  "transactionId": "123456789619999",
  "responseCode": "100",
  "avs": {
    "code": "X",
    "codeRaw": "I1"
  }
},
"reconciliationId": "64365912G3K7HFDJ",
"status": "AUTHORIZED",
"submitTimeUtc": "2022-05-24T20:13:49Z"
}

```

Merchant-Initiated No-Show Transactions

A no-show authorization occurs when a merchant charges a customer after the customer makes a reservation, and does not show up to claim the reservation. In this situation, the customer is charged an agreed upon fee for not showing up as expected.

Fields Specific to this Use Case

Include these fields with a standard authorization request:

- **processingInformation.authorizationOptions.initiator.type**–
Set the field to `merchant` to indicate this transaction is a merchant-initiated follow on transaction.
- **processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.reason**–
Set the reason field to `4`.

Required Fields

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

orderInformation.billTo.address1

orderInformation.billTo.administrativeArea

orderInformation.billTo.country

orderInformation.billTo.email

orderInformation.billTo.firstName

orderInformation.billTo.lastName

orderInformation.billTo.locality**orderInformation.billTo.phoneNumber****orderInformation.billTo.postalCode****paymentInformation.card.expirationMonth****paymentInformation.card.expirationYear****paymentInformation.card.number****processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.****previousTransactionId** For Discover and Initiated Transaction cards, use the transaction ID from the original transaction. For Visa, use the last successful transaction ID.**processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.reason****processingInformation.authorizationOptions.initiator.merchant.**

Card-Specific Required Fields

Discover

The listed card requires additional field:

processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.originalAuthorizedAmount

REST Example: Merchant-Initiated No-Show Transaction

Request

Endpoint: **POST https://api.cybersource.com/pts/v2/payments**

```
{
  "processingInformation": {
    "authorizationOptions": {
      "initiator": {
        "type": "merchant",
        "merchantInitiatedTransaction": {
          "originalAuthorizedAmount": "100", //Discover only
          "previousTransactionId": "123456789619999",
          "reason": "4"
        }
      }
    }
  },
  "orderInformation": {
    "billTo": {
      "country": "US",
      "lastName": "Kim",
      "address1": "201 S. Division St.",
      "postalCode": "48104-2201",
      "locality": "Ann Arbor",
      "administrativeArea": "MI",
      "firstName": "Kyong-Jin",
      "phoneNumber": "6504327113",
      "email": "test@cybs.com"
    }
  }
}
```

```

    },
    "amountDetails": {
      "totalAmount": "150.00",
      "currency": "USD"
    }
  },
  "paymentInformation": {
    "card": {
      "expirationYear": "2031",
      "number": "4111xxxxxxxxxx",
      "expirationMonth": "12"
    }
  }
}
}

```

Response

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6534214295466223903006/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6534214295466223903006"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6534214295466223903006/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "1653421429522"
  },
  "id": "6534214295466223903006",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "150.00",
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "pointOfSaleInformation": {

```

```

    "terminalId": "111111"
  },
  "processorInformation": {
    "approvalCode": "888888",
    "networkTransactionId": "123456789619999",
    "transactionId": "123456789619999",
    "responseCode": "100",
    "avs": {
      "code": "X",
      "codeRaw": "I1"
    }
  },
  "reconciliationId": "64365823G3K7HFAM",
  "status": "AUTHORIZED",
  "submitTimeUtc": "2022-05-24T19:43:49Z"
}

```

Standing Instruction Transactions

Standing instruction transactions are follow-on merchant-initiated transactions to a previous customer-initiated transaction. These are the types of standing instruction transactions:

- Installment payments
- Recurring payments (Visa, Discover, and American Express only)
- Subscription payments (Mastercard only)
- Standing order payments (Mastercard only)
- Unscheduled MIT COF payments

Installment Payments

An installment payment is a single purchase of goods or services billed to a customer in multiple transactions over a period of time agreed to by you and the customer. The agreement enables you to charge a specific amount at specified intervals.

Initial CIT Installment Payment

The first transaction in an installment payment is a customer-initiated transaction (CIT). Before you can perform a subsequent merchant-initiated transaction (MIT), you must store the customer's credentials for later use. Before you can store the user's credentials, you need to get the customer's consent to store their private information. This process is also known as establishing a relationship with the customer.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Fields Specific to this Use Case

Include the following information with a standard authorization request when storing customer credentials during the initial CIT installment payment:

- **processingInformation.authorizationOptions.initiator.credentialStoredOnFile**—Set the field to `true` to indicate the customer credentials will be stored for future use.

- **processingInformation.authorizationOptions.initiator.type**—Set the field to **customer** to indicate this transaction is a customer-initiated initial transaction.
- **processingInformation.commerceIndicator**—Set the transaction type to **internet** or a payer authentication value.
- In the CIT authorization response, capture the value in the **processorInformation.networkTransactionId** field. You will need to include the network transaction ID in subsequent MIT authorization requests.

Required Fields for Authorizing Initial Installment Payments Using REST APIs

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

orderInformation.billTo.address1

orderInformation.billTo.administrativeArea

orderInformation.billTo.country

orderInformation.billTo.email

orderInformation.billTo.firstName

orderInformation.billTo.lastName

orderInformation.billTo.locality

orderInformation.billTo.phoneNumber

orderInformation.billTo.postalCode

paymentInformation.card.expirationMonth

paymentInformation.card.expirationYear

paymentInformation.card.number

processingInformation.authorizationOptions.isInitiatorCredentialStoredOnFile Set to **true**.

processingInformation.authorizationOptions.isInitiatorCustomer Set to **customer**.

processingInformation.commerceIndicator Set to **internet** or a payer authentication value.

Card-Specific Required Fields

Some card companies require additional information when making authorizations with stored credentials.

Mastercard

Mastercard requires a reason code for an initial transaction:

processingInformation.authorizationOptions.isInitiatorMerchantInitiatedTransaction Set to **9**, **merchantInitiatedTransaction.reason**

REST Example: Initial Customer-Initiated Transaction Installment Payment

Request

Endpoint: **POST** <https://api.cybersource.com/pts/v2/payments>

```
{
  "processingInformation": {
    "commerceIndicator": "internet",
    "authorizationOptions": {
      "initiator": {
        "type": "customer",
        "credentialStoredOnFile": "true",
        "merchantInitiatedTransaction": {
          "reason": "9" //Mastercard only
        }
      }
    }
  },
  "orderInformation": {
    "billTo": {
      "firstName": "John",
      "lastName": "Deo",
      "address1": "201 S. Division St.",
      "postalCode": "48104-2201",
      "locality": "Ann Arbor",
      "administrativeArea": "MI",
      "country": "US",
      "phoneNumber": "6504327113",
      "email": "test@cybs.com"
    },
    "amountDetails": {
      "totalAmount": "100.00",
      "currency": "USD"
    }
  },
  "paymentInformation": {
    "card": {
      "expirationYear": "2031",
      "number": "4111xxxxxxxxxx",
      "expirationMonth": "12"
    }
  }
}
```

Response

```
{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6528187198946076303004/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6528187198946076303004"
    }
  },
}
```

```

    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6528187198946076303004/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "1652818719876"
  },
  "id": "6528187198946076303004",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "100.00",
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "pointOfSaleInformation": {
    "terminalId": "111111"
  },
  "processorInformation": {
    "approvalCode": "888888",
    "networkTransactionId": "123456789619999",
    "transactionId": "123456789619999",
    "responseCode": "100",
    "avs": {
      "code": "X",
      "codeRaw": "I1"
    }
  },
  "reconciliationId": "63165088Z3AHV91G",
  "status": "AUTHORIZED",
  "submitTimeUtc": "2022-05-17T20:18:40Z"
}

```

Subsequent MIT Installment Payments

After the initial CIT installment payment, subsequent installment payments are merchant-initiated transactions (MITs).

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Fields Required for this Use Case

Include these information with a standard authorization request when sending a subsequent installment payment request:

- **processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.previousTransactionID**—Include the previous network transaction ID to indicate previously stored customer credentials were used.
- **processingInformation.authorizationOptions.initiator.storedCredentialUsed**—Set the field to `true`.
- **processingInformation.authorizationOptions.initiator.type**—Set the field to `merchant` to indicate a merchant-initiated transaction.
- **processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.reason**—Set the reason code field to 9.
- **processingInformation.commerceIndicator**—Set the field to `install` to indicate this is an installment transaction.

Required Fields for Authorizing Subsequent Installment Payments Using REST APIs

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

orderInformation.billTo.address1

orderInformation.billTo.administrativeArea

orderInformation.billTo.country

orderInformation.billTo.email

orderInformation.billTo.firstName

orderInformation.billTo.lastName

orderInformation.billTo.locality

orderInformation.billTo.phoneNumber

orderInformation.billTo.postalCode

paymentInformation.card.expirationMonth

paymentInformation.card.expirationYear

paymentInformation.card.number

processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.previousTransactionID

For Discover and American Express cards, use the transaction ID from the original transaction. For Visa, use the last successful transaction ID.

processingInformation.authorizationOptions.initiator.storedCredentialUsed

Set to `true`.

processingInformation.authorizationOptions.initiator.type

Set to `merchant`.

processingInformation.commerceIndicator

Set to `install`.

processingInformation.initiator.merchantInitiatedTransaction.reason

Card-Specific Required Fields

Some card companies require additional information when making authorizations with stored credentials.

Discover

Discover requires the authorization amount from the original transaction when sending a request:

processingInformation.authorizationOptions.initiator.merchantInitiatedTransaction.
originalAuthorizedAmount

REST Example: Subsequent Installment Payment Authorization

Request

Endpoint: POST <https://api.cybersource.com/pts/v2/payments>

```
{
  "processingInformation": {
    "commerceIndicator": "install",
    "authorizationOptions": {
      "initiator": {
        "storedCredentialUsed": "true",
        "type": "merchant",
        "merchantInitiatedTransaction": {
          "reason": "9",
          "previousTransactionId": "123456789619999",
          "originalAuthorizedAmount": "100" //Discover Only
        }
      }
    }
  },
  "orderInformation": {
    "billTo": {
      "firstName": "John",
      "lastName": "Deo",
      "address1": "201 S. Division St.",
      "postalCode": "48104-2201",
      "locality": "Ann Arbor",
      "administrativeArea": "MI",
      "country": "US",
      "phoneNumber": "6504327113",
      "email": "test@cybs.com"
    },
    "amountDetails": {
      "totalAmount": "100.00",
      "currency": "USD"
    }
  },
  "paymentInformation": {
    "card": {
      "expirationYear": "2031",
      "number": "4111xxxxxxxxxx",

```



```

    "expirationMonth": "12"
  }
}
}

```

Response

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6530824710046809304002/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6530824710046809304002"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6530824710046809304002/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "1653082470983"
  },
  "id": "6530824710046809304002",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "100.00",
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "002"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "002"
    },
    "card": {
      "type": "002"
    }
  },
  "pointOfSaleInformation": {
    "terminalId": "111111"
  },
  "processorInformation": {
    "approvalCode": "888888",
    "authIndicator": "1",
    "networkTransactionId": "123456789619999",
    "transactionId": "123456789619999",
    "responseCode": "100",
    "avs": {
      "code": "X",

```

```

    "codeRaw": "I1"
  }
},
"reconciliationId": "79710341A39WTT5W",
"status": "AUTHORIZED",
"submitTimeUtc": "2022-05-20T21:34:31Z"
}

```

Recurring Payments

A recurring payment is a credentials-on-file (COF) transaction in a series of payments that you bill to a customer at a fixed amount at fixed, regular intervals not to exceed one year between transactions. The series of recurring payments is the result of an agreement between you and the customer for the purchase of goods or services that are provided at regular intervals.

Mastercard uses subscription and standing order payments instead of recurring payments. See [Mastercard Subscription Payments](#) on page 135 and [Mastercard Standing Order Payments](#) on page 132.

Recurring Billing for Recurring Payments

If you are using the Cybersource Recurring Billing service, do not use this document. Cybersource saves and stores payment credentials for recurring transactions, ensuring compliance with COF best practices.

For more information on Recurring Billing, see [Recurring Billing](#).

Initial CIT Recurring Payment with a PAN

The first transaction in a recurring payment is a customer-initiated transaction (CIT). Before you can perform a subsequent merchant-initiated transaction (MIT), you must store the customer's credentials for later use. Before you can store the user's credentials, you need to get the customer's consent to store their private information. This is also known as establishing a relationship with the customer.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Fields Specific to this Use Case

Include the following information with a standard authorization request when storing customer credentials during the initial CIT recurring payment:

- **processingInformation.authorizationOptions.initiator.credentialStoredOnFile**—Set the field to `true` to indicate the customer credentials will be stored for future use.
- **processingInformation.authorizationOptions.initiator.type**—Set the field to `customer` to indicate this transaction is a customer-initiated initial transaction.
- **processingInformation.commerceIndicator**—Set the field to `internet` or a payer authentication value.
- In the CIT authorization response, save the Transaction ID. You will need to include the network transaction ID in subsequent MIT recurring authorization requests.

Required Fields for Authorizing Initial Recurring Payments Using REST APIs

For an Initial CIT Recurring Payment, you must provide either field:

processingInformation.recurringOptions.firstSector **Set to true** payment

processingInformation.authorizationOptions.Sector **Set to 7: merchantInitiatedTransaction.reason**

Use these required fields to request an initial CIT recurring payment.

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

orderInformation.billTo.address1

orderInformation.billTo.administrativeArea

orderInformation.billTo.country

orderInformation.billTo.email

orderInformation.billTo.firstName

orderInformation.billTo.lastName

orderInformation.billTo.locality

orderInformation.billTo.postalCode

paymentInformation.card.expirationMonth

paymentInformation.card.expirationYear

paymentInformation.card.number

processingInformation.authorizationOptions.Sector **Set to true** credentialStoredOnFile

processingInformation.authorizationOptions.Sector **Set to customer.**

processingInformation.commerceIndicator **Set to internet, a payer authentication value, or MOTO.**

REST Example: Initial CIT Recurring Payment

Use this REST API request for the initial CIT recurring payment. In the response, find the transaction ID value in the **processorInformation.transactionId** field and save it for subsequent MIT recurring authorization requests. If you are using a token, the transaction ID will be automatically stored.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Request

```
{
  "processingInformation": {
    "commerceIndicator": "internet",
    "authorizationOptions": {
      "initiator": {
        "credentialStoredOnFile": "true",
        "type": "customer"
      }
    }
  }
}
```

```

    }
  }
},
"orderInformation": {
  "billTo": {
    "firstName": "John",
    "lastName": "Deo",
    "address1": "201 S. Division St.",
    "postalCode": "48104-2201",
    "locality": "Ann Arbor",
    "administrativeArea": "MI",
    "country": "US",
    "phoneNumber": "6504327113",
    "email": "test@cybs.com"
  },
  "amountDetails": {
    "totalAmount": "100.00",
    "currency": "USD"
  }
},
"paymentInformation": {
  "card": {
    "expirationYear": "2031",
    "number": "4111xxxxxxxxxx",
    "expirationMonth": "12"
  }
}
}
}

```

Response

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6528187198946076303004/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6528187198946076303004"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6528187198946076303004/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "1652818719876"
  },
  "id": "6528187198946076303004",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "100.00",
      "currency": "USD"
    }
  }
},

```

```

"paymentAccountInformation": {
  "card": {
    "type": "001"
  }
},
"paymentInformation": {
  "tokenizedCard": {
    "type": "001"
  },
  "card": {
    "type": "001"
  }
},
"pointOfSaleInformation": {
  "terminalId": "111111"
},
"processorInformation": {
  "approvalCode": "888888",
  "networkTransactionId": "123456789619999",
  "transactionId": "123456789619999",
  "responseCode": "100",
  "avs": {
    "code": "X",
    "codeRaw": "I1"
  }
},
"reconciliationId": "63165088Z3AHV91G",
"status": "AUTHORIZED",
"submitTimeUtc": "2022-05-17T20:18:40Z"
}

```

Mastercard Standing Order Payments

A standing order payment is a recurring COF transaction that is a variable amount at a regular interval, such as a utility bill, not to exceed one year between transactions. The series of recurring payments is the result of an agreement between you and the customer for the purchase of goods or services that are provided at regular intervals.

Mastercard Initial CIT Standing Order Payment

The first transaction in a standing order payment is a customer-initiated transaction (CIT). Before you can perform a subsequent merchant-initiated transaction (MIT), you must store the customer's credentials for later use. Before you can store the user's credentials, you need to get the customer's consent to store their private information. This is also known as establishing a relationship with the customer.

Fields Specific to this Use Case

Include the following information with a standard authorization request:

- **processingInformation.authorizationOptions.initiator.credentialStoredOnFile**–Set the field to `true` to indicate the customer credentials will be stored for future use.
- **processingInformation.commerceIndicator**– Set the field to `internet` or a payer authentication value to indicate the transaction type.

- **processingInformation.initiator.merchantInitiatedTransaction.reason** –Set the field to reason code 8.
- In the CIT authorization response, capture the value in the **processorInformation.networkTransactionId** field. You will need to include the network transaction ID in subsequent MIT authorization requests.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Required Fields for Authorizing Standing Order Payments Using REST APIs

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

orderInformation.billTo.address1

orderInformation.billTo.administrativeArea

orderInformation.billTo.country

orderInformation.billTo.email

orderInformation.billTo.firstName

orderInformation.billTo.lastName

orderInformation.billTo.locality

orderInformation.billTo.phoneNumber

orderInformation.billTo.postalCode

paymentInformation.card.expirationMonth

paymentInformation.card.expirationYear

paymentInformation.card.number

processingInformation.authorizationOptions.Set to true **credentialStoredOnFile**

processingInformation.authorizationOptions.Set to customer.

processingInformation.commerceIndicator Set to internet or payer auth.

processingInformation.initiator.merchantInitiatedTransaction.reason Set to 8

REST Example: Initial CIT Standing Order Payments

Request

Endpoint: POST <https://api.cybersource.com/pts/v2/payments>

```
{
  "processingInformation": {
    "commerceIndicator": "internet",
    "authorizationOptions": {
      "initiator": {
        "credentialStoredOnFile": "true",
        "type": "customer",

```

```

        "merchantInitiatedTransaction": {
            "reason": "8"
        }
    }
},
"orderInformation": {
    "billTo": {
        "firstName": "John",
        "lastName": "Deo",
        "address1": "201 S. Division St.",
        "postalCode": "48104-2201",
        "locality": "Ann Arbor",
        "administrativeArea": "MI",
        "country": "US",
        "phoneNumber": "6504327113",
        "email": "test@cybs.com"
    },
    "amountDetails": {
        "totalAmount": "100.00",
        "currency": "USD"
    }
},
"paymentInformation": {
    "card": {
        "expirationYear": "2031",
        "number": "5555xxxxxxxxxx",
        "expirationMonth": "12"
    }
}
}

```

Response

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6530824710046809304002/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6530824710046809304002"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6530824710046809304002/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "1653082470983"
  },
  "id": "6530824710046809304002",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "100.00",

```

```

    "currency": "USD"
  },
  "paymentAccountInformation": {
    "card": {
      "type": "002"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "002"
    },
    "card": {
      "type": "002"
    }
  },
  "pointOfSaleInformation": {
    "terminalId": "111111"
  },
  "processorInformation": {
    "approvalCode": "888888",
    "authIndicator": "1",
    "networkTransactionId": "123456789619999",
    "transactionId": "123456789619999",
    "responseCode": "100",
    "avs": {
      "code": "X",
      "codeRaw": "I1"
    }
  },
  "reconciliationId": "79710341A39WTT5W",
  "status": "AUTHORIZED",
  "submitTimeUtc": "2022-05-20T21:34:31Z"
}

```

Mastercard Subscription Payments

A subscription payment is a recurring COF transaction that is processed at a fixed amount at regular intervals not to exceed one year between transactions. The series of recurring payments is the result of an agreement between you and the customer for the purchase of goods or services that are provided at regular intervals.

Mastercard Initial CIT Subscription Payment

The first transaction in a subscription payment is a customer-initiated transaction (CIT). Before you can perform a subsequent merchant-initiated transaction (MIT), you must store the customer's credentials for later use. Before you can store the user's credentials, you need to get the customer's consent to store their private information. This is also known as establishing a relationship with the customer.

Fields Specific to this Use Case

Include the following information with a standard authorization request when storing customer credentials during the initial CIT subscription payment:

- **processingInformation.authorizationOptions.initiator.credentialStoredOnFile**—Set the field to **true** to indicate the customer credentials will be stored for future use.
- **processingInformation.commerceIndicator**—Set the field to **internet** or a payer authentication value.
- **processingInformation.initiator.merchantInitiatedTransaction.reason** —Set the reason code field to **7**.
- In the CIT authorization response, capture the value in the **processorInformation.networkTransactionId** field. You will need to include the network transaction ID in subsequent MIT authorization requests.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Required Fields for Authorizing Subscription Payments Using REST APIs

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

orderInformation.billTo.address1

orderInformation.billTo.administrativeArea

orderInformation.billTo.country

orderInformation.billTo.email

orderInformation.billTo.firstName

orderInformation.billTo.lastName

orderInformation.billTo.locality

orderInformation.billTo.phoneNumber

orderInformation.billTo.postalCode

paymentInformation.card.expirationMonth

paymentInformation.card.expirationYear

paymentInformation.card.number

processingInformation.authorizationOptions.initiator.credentialStoredOnFile Set to **true**.

processingInformation.authorizationOptions.initiator Set to **customer** or **payer auth**.

processingInformation.commerceIndicator Set to **recurring**.

processingInformation.initiator.merchantInitiatedTransaction.reason Set to **7**.

REST Example: Initial CIT Subscription Payments

Request

Endpoint: POST <https://api.cybersource.com/pts/v2/payments>

```
{
  "processingInformation": {
    "commerceIndicator": "internet",
```

```

    "authorizationOptions": {
      "initiator": {
        "type": "customer",
        "credentialStoredOnFile": "true",
        "merchantInitiatedTransaction": {
          "reason": "7"
        }
      }
    },
    "orderInformation": {
      "billTo": {
        "firstName": "John",
        "lastName": "Deo",
        "address1": "201 S. Division St.",
        "postalCode": "48104-2201",
        "locality": "Ann Arbor",
        "administrativeArea": "MI",
        "country": "US",
        "phoneNumber": "6504327113",
        "email": "test@cybs.com"
      },
      "amountDetails": {
        "totalAmount": "100.00",
        "currency": "USD"
      }
    },
    "paymentInformation": {
      "card": {
        "expirationYear": "2031",
        "number": "4111xxxxxxxxxx",
        "expirationMonth": "12"
      }
    }
  }
}

```

Response

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6530824710046809304002/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6530824710046809304002"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6530824710046809304002/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "1653082470983"
  },
}

```

```

{id": "6530824710046809304002",
"orderInformation": {
  "amountDetails": {
    "authorizedAmount": "100.00",
    "currency": "USD"
  }
},
"paymentAccountInformation": {
  "card": {
    "type": "002"
  }
},
"paymentInformation": {
  "tokenizedCard": {
    "type": "002"
  },
  "card": {
    "type": "002"
  }
},
"pointOfSaleInformation": {
  "terminalId": "111111"
},
"processorInformation": {
  "approvalCode": "888888",
  "authIndicator": "1",
  "networkTransactionId": "123456789619999",
  "transactionId": "123456789619999",
  "responseCode": "100",
  "avs": {
    "code": "X",
    "codeRaw": "I1"
  }
},
"reconciliationId": "79710341A39WTT5W",
"status": "AUTHORIZED",
"submitTimeUtc": "2022-05-20T21:34:31Z"
}

```

Unscheduled COF Payments

An unscheduled credentials-on-file (COF) transaction uses stored payment information for a fixed or variable amount that does not occur regularly. An account top-up is one kind of unscheduled COF.

Initial CIT Unscheduled COF Payment

The first transaction in an unscheduled COF payment is a customer-initiated transaction (CIT). Before you can perform a subsequent merchant-initiated transaction (MIT), you must store the customer's credentials for later use. Before you can store the user's credentials, you need to get the customer's consent to store their private information. This is also known as establishing a relationship with the customer.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Fields Specific to this Use Case

Include the following information with a standard authorization request when storing customer credentials during the initial CIT unscheduled COF payment:

- **processingInformation.authorizationOptions.initiator.credentialStoredOnFile**—Set the field to **true** to indicate the customer credentials will be stored for future use.
- **processingInformation.authorizationOptions.initiator.type**— Set the field to **customer** to indicate this transaction is a customer-initiated initial transaction.
- **processingInformation.commerceIndicator**—Set the field to **internet**.
- In the CIT authorization response, capture the value in the **processorInformation.networkTransactionId** field. You will need to include the network transaction ID in subsequent MIT authorization requests.

Required Fields for Authorizing Initial Unscheduled COF Payments Using REST APIs

These fields are required in a subsequent authorization request for an initial unscheduled COF payment:

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

orderInformation.billTo.address1

orderInformation.billTo.administrativeArea

orderInformation.billTo.country

orderInformation.billTo.email

orderInformation.billTo.firstName

orderInformation.billTo.lastName

orderInformation.billTo.locality

orderInformation.billTo.phoneNumber

orderInformation.billTo.postalCode

paymentInformation.card.expirationMonth

paymentInformation.card.expirationYear

paymentInformation.card.number

processingInformation.authorizationOptions.initiator.credentialStoredOnFile **Set to true.**

processingInformation.authorizationOptions.initiator.type **Set to customer.**

processingInformation.commerceIndicator **Set to internet or a payer authentication value.**

REST Example: Initial CIT Unscheduled COF Payment

Request

Endpoint: POST <https://api.cybersource.com/pts/v2/payments>

```
{
  "processingInformation": {
    "commerceIndicator": "internet",
    "authorizationOptions": {
      "initiator": {
        "credentialStoredOnFile": "true",
        "type": "customer"
      }
    }
  },
  "orderInformation": {
    "billTo": {
      "firstName": "John",
      "lastName": "Deo",
      "address1": "201 S. Division St.",
      "postalCode": "48104-2201",
      "locality": "Ann Arbor",
      "administrativeArea": "MI",
      "country": "US",
      "phoneNumber": "6504327113",
      "email": "test@cybs.com"
    },
    "amountDetails": {
      "totalAmount": "100.00",
      "currency": "USD"
    }
  },
  "paymentInformation": {
    "card": {
      "expirationYear": "2031",
      "number": "4111xxxxxxxxxx",
      "expirationMonth": "12"
    }
  }
}
```

Response

```
{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6528187198946076303004/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6528187198946076303004"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6528187198946076303004/captures"
    }
  }
}
```

```

    }
  },
  "clientReferenceInformation": {
    "code": "1652818719876"
  },
  "id": "6528187198946076303004",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "100.00",
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "pointOfSaleInformation": {
    "terminalId": "111111"
  },
  "processorInformation": {
    "approvalCode": "888888",
    "networkTransactionId": "123456789619999",
    "transactionId": "123456789619999",
    "responseCode": "100",
    "avs": {
      "code": "X",
      "codeRaw": "I1"
    }
  },
  "reconciliationId": "63165088Z3AHV91G",
  "status": "AUTHORIZED",
  "submitTimeUtc": "2022-05-17T20:18:40Z"
}

```

Processing Token Management Service Payments

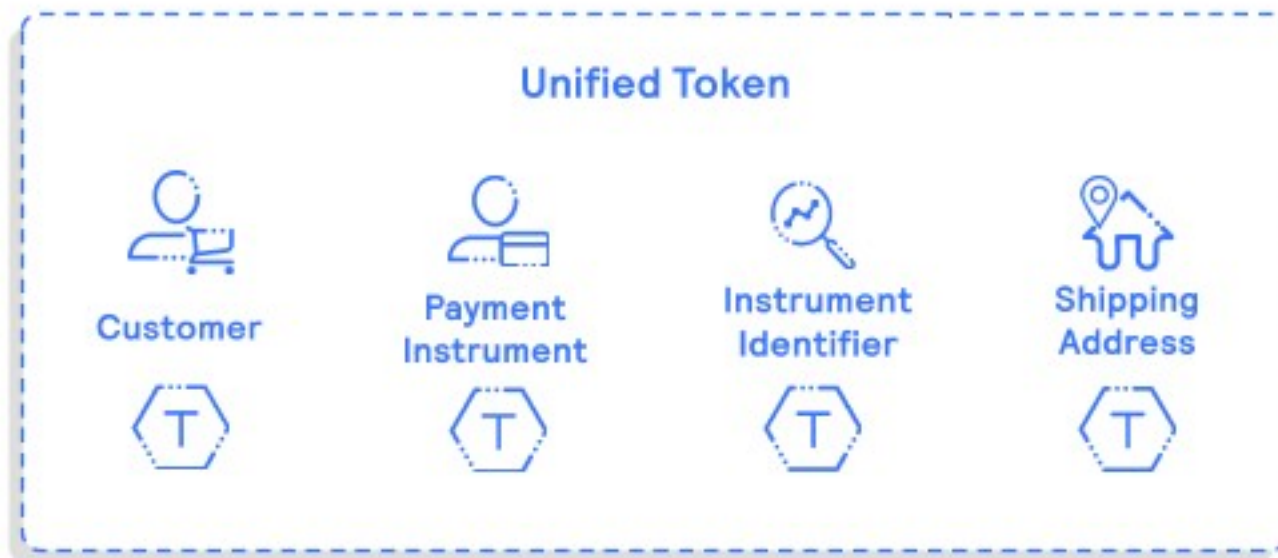
Token Management Service (TMS) links tokens across service providers, payment types, and channels for sellers, acquirers, and technology partners. TMS tokenizes, securely stores, and manages primary account number (PAN), payment card expiration date, customer data, and electronic check data.

TMS enables you to:

- Securely store a customer's payment details and their billing and shipping addresses.
- Create a network token of a customer's payment card.
- Manage sensitive data securely by creating, retrieving, updating, and deleting tokens through the [TMS API](#).

TMS simplifies your PCI DSS compliance. TMS passes back to you tokens that represent this data. You then store these tokens in your environment and databases instead of customer payment details.

TMS creates one unified token identifier, a token-of-tokens, to centralize management of all payment credentials:



The unified token identifier protects sensitive customer payment data stored in secure Visa data centers.

Authorize a Payment with a Customer Token

This section shows you how to authorize a payment with a customer token.

Endpoint

Test: `POST https://apitest.cybersource.com/pts/v2/payments`

Production: `POST https://api.cybersource.com/pts/v2/payments`

Production in India: `POST https://api.in.cybersource.com/pts/v2/payments`

Required Fields for Authorizing a Payment with a Customer Token Using the REST API

`orderInformation.amountDetails.currency`

`orderInformation.amountDetails.totalAmount`

`orderInformation.billTo.address1`

`orderInformation.billTo.administrativeArea`

`orderInformation.billTo.country`

`orderInformation.billTo.email`

`orderInformation.billTo.firstName`

`orderInformation.billTo.lastName`

`orderInformation.billTo.locality`

orderInformation.billTo.postalCode**paymentInformation.card.expirationMonth****paymentInformation.card.expirationYear****paymentInformation.card.number****paymentInformation.card.type****paymentInformation.customer.id**

Set to the ID of the customer token you want to use.

Related information

- [API Field Reference for the REST API](#)

REST Example: Authorizing a Payment with a Customer Token

Request

POST https://apitest.cybersource.com/pts/v2/payments

```
{
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "paymentInformation": {
    "customer": {
      "id": "F45FB3E443AC3C57E053A2598D0A9CFF"
    }
  },
  "orderInformation": {
    "amountDetails": {
      "currency": "USD",
      "totalAmount": "1.00"
    }
  },
  "processingInformation": {
    "capture": "false",
    "commerceIndicator": "internet"
  }
}
```

Successful Response

The request response informs the IDs of the payment instrument & shipping address used as the customer's defaults.

```
{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6760630538166130603955/reversals"
    },
    "self": {
```

```

    "method": "GET",
    "href": "/pts/v2/payments/6760630538166130603955"
  },
  "capture": {
    "method": "POST",
    "href": "/pts/v2/payments/6760630538166130603955/captures"
  }
},
"clientReferenceInformation": {
  "code": "RTS-Auth"
},
"id": "6760630538166130603955",
"orderInformation": {
  "amountDetails": {
    "authorizedAmount": "1.00",
    "currency": "USD"
  }
},
"paymentAccountInformation": {
  "card": {
    "type": "001"
  }
},
"paymentInformation": {
  "tokenizedCard": {
    "type": "001"
  },
  "instrumentIdentifier": {
    "id": "70300000000014911515",
    "state": "ACTIVE"
  },
  "shippingAddress": {
    "id": "F45FB3E443AF3C57E053A2598D0A9CFF"
  },
  "paymentInstrument": {
    "id": "F45FC6785E3C31A2E053A2598D0A5346"
  },
  "card": {
    "type": "001"
  },
  "customer": {
    "id": "F45FB3E443AC3C57E053A2598D0A9CFF"
  }
},
"pointOfSaleInformation": {
  "terminalId": "111111"
},
"processorInformation": {
  "paymentAccountReferenceNumber": "V0010013019326121174070050420",
  "approvalCode": "888888",
  "networkTransactionId": "123456789619999",
  "transactionId": "123456789619999",
  "responseCode": "100",
  "avs": {
    "code": "X",
    "codeRaw": "I1"
  }
}

```

```

    }
  },
  "reconciliationId": "69816021FDTK35GW",
  "status": "AUTHORIZED",
  "submitTimeUtc": "2023-02-10T21:04:14Z"
}

```

REST Example: Authorizing a Payment Using a Customer Token Linked to a Network Token

Request

POST <https://apitest.cybersource.com/pts/v2/payments>

```

{
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "paymentInformation": {
    "customer": {
      "id": "F60328413BAB09A4E053AF598E0A33DB"
    }
  },
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "102.21",
      "currency": "USD"
    }
  }
}

```

Successful Response

The request response informs the IDs of the payment instrument and shipping address used as the customer's defaults.

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6778647071126384904953/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6778647071126384904953"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6778647071126384904953/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "id": "6778647071126384904953",

```

```

"issuerInformation": {
  "responseRaw": "01103220000000E100002000....."
},
"orderInformation": {
  "amountDetails": {
    "authorizedAmount": "102.21",
    "currency": "USD"
  }
},
"paymentAccountInformation": {
  "card": {
    "type": "002"
  }
},
"paymentInformation": {
  "tokenizedCard": {
    "type": "002"
  }
},
"instrumentIdentifier": {
  "id": "70200000000010603216",
  "state": "ACTIVE"
},
"shippingAddress": {
  "id": "F60328413BAE09A4E053AF598E0A33DB"
},
"paymentInstrument": {
  "id": "F6032841BE33098EE053AF598E0AB0A5"
},
"card": {
  "type": "002"
},
"customer": {
  "id": "F60328413BAB09A4E053AF598E0A33DB"
},
"pointOfSaleInformation": {
  "terminalId": "08244117"
},
"processingInformation": { "paymentSolution": "014" },
"processorInformation": {
  "paymentAccountReferenceNumber": "500150U4U5UYXLV127XTONYN49CL1",
  "merchantNumber": "000844028303882",
  "approvalCode": "831000",
  "networkTransactionId": "0602MCC603474",
  "transactionId": "0602MCC603474",
  "responseCode": "00",
  "avs": {
    "code": "Y",
    "codeRaw": "Y"
  }
},
"reconciliationId": "EUHW1EMHIZ30",
"status": "AUTHORIZED",
"submitTimeUtc": "2023-03-03T17:31:48Z"
}

```

Authorize a Payment with a Non-Default Shipping Address

This section shows you how to make a payment with a non-default shipping address.

Endpoint

Test: POST `https://apitest.cybersource.com/pts/v2/payments`

Production: POST `https://api.cybersource.com/pts/v2/payments`

Production in India: POST `https://api.in.cybersource.com/pts/v2/payments`

Required Fields for Authorizing a Payment with a Non-Default Shipping Address Using the REST API

`orderInformation.amountDetails.currency`

`orderInformation.amountDetails.totalAmount`

`orderInformation.billTo.address1`

`orderInformation.billTo.administrativeArea`

`orderInformation.billTo.country`

`orderInformation.billTo.email`

`orderInformation.billTo.firstName`

`orderInformation.billTo.lastName`

`orderInformation.billTo.locality`

`orderInformation.billTo.postalCode`

`paymentInformation.card.expirationMonth`

`paymentInformation.card.expirationYear`

`paymentInformation.card.number`

`paymentInformation.card.type`

`paymentInformation.customer.id`

Set to the ID of the customer token you want to use.

`paymentInformation.shippingAddress.id`

Set to the ID of the shipping address token you want to use.

Related information

- [API Field Reference for the REST API](#)

Example: Authorize a Payment with a Non-Default Shipping Address Using the REST API

Request

POST https://apitest.cybersource.com/pts/v2/payments

```
{
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "paymentInformation": {
    "customer": {
      "id": "F45FB3E443AC3C57E053A2598D0A9CFF"
    },
    "shippingAddress": {
      "id": "F45FD8DE51B99E9CE053A2598D0AFDFA"
    }
  },
  "orderInformation": {
    "amountDetails": {
      "currency": "USD",
      "totalAmount": "1.00"
    }
  },
  "processingInformation": {
    "capture": "false",
    "commerceIndicator": "internet"
  }
}
```

Successful Response

```
{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6760633013666830003954/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6760633013666830003954"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6760633013666830003954/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "id": "6760633013666830003954",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "1.00",
```

```

    "currency": "USD"
  }
},
"paymentAccountInformation": {
  "card": {
    "type": "001"
  }
},
"paymentInformation": {
  "tokenizedCard": {
    "type": "001"
  },
  "instrumentIdentifier": {
    "id": "70300000000014911515",
    "state": "ACTIVE"
  },
  "shippingAddress": {
    "id": "F45FD8DE51B99E9CE053A2598D0AFDFA"
  },
  "paymentInstrument": {
    "id": "F45FC6785E3C31A2E053A2598D0A5346"
  },
  "card": {
    "type": "001"
  },
  "customer": {
    "id": "F45FB3E443AC3C57E053A2598D0A9CFF"
  }
},
"pointOfSaleInformation": {
  "terminalId": "111111"
},
"processorInformation": {
  "paymentAccountReferenceNumber": "V0010013019326121174070050420",
  "approvalCode": "888888",
  "networkTransactionId": "123456789619999",
  "transactionId": "123456789619999",
  "responseCode": "100",
  "avs": {
    "code": "X",
    "codeRaw": "I1"
  }
},
"reconciliationId": "698162204DTIATQ2",
"status": "AUTHORIZED",
"submitTimeUtc": "2023-02-10T21:08:21Z"
}

```

Authorize a Payment with Non-Default Payment Instrument

This section shows you how to authorize a payment with a non-default payment instrument.

Endpoint

Test: POST <https://apitest.cybersource.com/pts/v2/payments>

Production: POST <https://api.cybersource.com/pts/v2/payments>

Production in India: POST <https://api.in.cybersource.com/pts/v2/payments>

Required Fields for Authorizing a Payment with Non-Default Payment Instrument Using the REST API

paymentInformation.paymentInstrument.id Set to the ID of the payment instrument token you want to use.

Related information

- [API Field Reference for the REST API](#)

Optional Fields for Authorizing a Payment with Non-Default Payment Instrument Using the REST API

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

orderInformation.billTo.address1

orderInformation.billTo.administrativeArea

orderInformation.billTo.country

orderInformation.billTo.email

orderInformation.billTo.firstName

orderInformation.billTo.lastName

orderInformation.billTo.locality

orderInformation.billTo.postalCode

paymentInformation.card.expirationMonth

paymentInformation.card.expirationYear

paymentInformation.card.number

paymentInformation.card.type

Related information

- API Field Reference for the REST API

Example: Authorize a Payment with Non-Default Payment Instrument Using the REST API

Request

POST <https://apitest.cybersource.com/pts/v2/payments>

```
{
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "customer": {
    "id": "F45FB3E443AC3C57E053A2598D0A9CFF"
  },
  "paymentInstrument": {
    "id": "F4D5E715F7BD9910E053A2598D0A7278"
  },
  "orderInformation": {
    "amountDetails": {
      "currency": "USD",
      "totalAmount": "1.00"
    }
  },
  "processingInformation": {
    "capture": "false",
    "commerceIndicator": "internet"
  }
}
```

Successful Response

```
{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6765713628736138103955/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6765713628736138103955"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6765713628736138103955/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
}
```

```

{id": "6765713628736138103955",
"orderInformation": {
  "amountDetails": {
    "authorizedAmount": "1.00",
    "currency": "USD"
  }
},
"paymentAccountInformation": {
  "card": {
    "type": "001"
  }
},
"paymentInformation": {
  "tokenizedCard": {
    "type": "001"
  },
  "instrumentIdentifier": {
    "id": "70100000000016241111",
    "state": "ACTIVE"
  },
  "paymentInstrument": {
    "id": "F4D5E715F7BD9910E053A2598D0A7278"
  },
  "card": {
    "type": "001"
  },
  "customer": {
    "id": "F4D5E715F75E9910E053A2598D0A7278"
  }
},
"pointOfSaleInformation": {
  "terminalId": "111111"
},
"processorInformation": {
  "approvalCode": "888888",
  "networkTransactionId": "123456789619999",
  "transactionId": "123456789619999",
  "responseCode": "100",
  "avs": {
    "code": "X",
    "codeRaw": "I1"
  }
},
"reconciliationId": "60561224BE37KN5W",
"status": "AUTHORIZED",
"submitTimeUtc": "2023-02-16T18:16:03Z"
}

```

Authorize a Payment with a Payment Instrument

This section shows you how to authorize a payment with a payment instrument.

Endpoint

Test: POST <https://apitest.cybersource.com/pts/v2/payments>

Production: POST <https://api.cybersource.com/pts/v2/payments>

Production in India: POST <https://api.in.cybersource.com/pts/v2/payments>

Required Fields for Authorizing a Payment with a Payment Instrument Using the REST API

`paymentInformation.paymentInstrument.id` Set to the ID of the payment instrument token you want to use.

Related information

- [API Field Reference for the REST API](#)

Optional Fields for Authorizing a Payment with a Payment Instrument Using the REST API

`orderInformation.amountDetails.currency`

`orderInformation.amountDetails.totalAmount`

`orderInformation.billTo.address1`

`orderInformation.billTo.administrativeArea`

`orderInformation.billTo.country`

`orderInformation.billTo.email`

`orderInformation.billTo.firstName`

`orderInformation.billTo.lastName`

`orderInformation.billTo.locality`

`orderInformation.billTo.postalCode`

`paymentInformation.card.expirationMonth`

`paymentInformation.card.expirationYear`

`paymentInformation.card.number`

`paymentInformation.card.type`

Related information

- [API Field Reference for the REST API](#)

REST Example: Authorizing a Payment with a Payment Instrument

Request

```
POST https://apitest.cybersource.com/pts/v2/payments
```

```
{
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "paymentInformation": {
    "paymentInstrument": {
      "id": "F4D5E715F7BD9910E053A2598D0A7278"
    }
  },
  "orderInformation": {
    "amountDetails": {
      "currency": "USD",
      "totalAmount": "1.00"
    }
  },
  "processingInformation": {
    "capture": "false",
    "commerceIndicator": "internet"
  }
}
```

Successful Response

```
{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6765713628736138103955/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6765713628736138103955"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6765713628736138103955/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "id": "6765713628736138103955",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "1.00",
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
```

```

    "type": "001"
  },
  "instrumentIdentifier": {
    "id": "70100000000016241111",
    "state": "ACTIVE"
  },
  "paymentInstrument": {
    "id": "F4D5E715F7BD9910E053A2598D0A7278"
  },
  "card": {
    "type": "001"
  },
  "customer": {
    "id": "F4D5E715F75E9910E053A2598D0A7278"
  }
},
"pointOfSaleInformation": {
  "terminalId": "111111"
},
"processorInformation": {
  "approvalCode": "888888",
  "networkTransactionId": "123456789619999",
  "transactionId": "123456789619999",
  "responseCode": "100",
  "avs": {
    "code": "X",
    "codeRaw": "I1"
  }
},
"reconciliationId": "60561224BE37KN5W",
"status": "AUTHORIZED",
"submitTimeUtc": "2023-02-16T18:16:03Z"
}

```

Authorize a Payment with an Instrument Identifier

This section shows you how to authorize a payment with an instrument identifier token.

Endpoint

Test: POST <https://apitest.cybersource.com/pts/v2/payments>

Production: POST <https://api.cybersource.com/pts/v2/payments>

Production in India: POST <https://api.in.cybersource.com/pts/v2/payments>

Required Fields for Authorizing a Payment with an Instrument Identifier Using the REST API

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount**orderInformation.billTo.address1****orderInformation.billTo.administrativeArea****orderInformation.billTo.country****orderInformation.billTo.email****orderInformation.billTo.firstName****orderInformation.billTo.lastName****orderInformation.billTo.locality****orderInformation.billTo.postalCode****paymentInformation.card.expirationMonth****paymentInformation.card.expirationYear****paymentInformation.card.number****paymentInformation.card.type****paymentInformation.instrumentIdentifier.id** Set to the ID of the instrument identifier token you want to use.

Related information

- [API Field Reference for the REST API](#)

Example: Authorize a Payment with an Instrument Identifier Using the REST API

Request

POST https://apitest.cybersource.com/pts/v2/payments

```
{
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "paymentInformation": {
    "card": {
      "expirationYear": "2031",
      "expirationMonth": "12",
      "type": "001"
    },
    "instrumentIdentifier": {
      "id": "{{tms-instrumentIdentifier-id}}"
    }
  },
  "orderInformation": {
    "amountDetails": {
      "currency": "USD",
      "totalAmount": "1.00"
    }
  }
}
```

```

    }
  },
  "processingInformation": {
    "capture": "false",
    "commerceIndicator": "internet"
  }
}

```

Successful Response

```

{
  "_links": {
    "void": {
      "method": "POST",
      "href": "/pts/v2/credits/6764065889926478803955/voids"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/credits/6764065889926478803955"
    }
  },
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "creditAmountDetails": {
    "currency": "USD",
    "creditAmount": "1.00"
  },
  "id": "6764065889926478803955",
  "orderInformation": {
    "amountDetails": {
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "instrumentIdentifier": {
      "id": "70100000000016241111",
      "state": "ACTIVE"
    },
    "card": {
      "type": "001"
    }
  },
  "processorInformation": {
    "approvalCode": "888888",
    "responseCode": "100"
  },
  "reconciliationId": "600445691E3A4D4B",

```

```

"status": "PENDING",
"submitTimeUtc": "2023-02-14T20:29:49Z"
}

```

Authorize a Payment with Legacy Token

This section shows you how to authorize a payment with a legacy token.

Endpoint

Test: POST <https://apitest.cybersource.com/pts/v2/payments>

Production: POST <https://api.cybersource.com/pts/v2/payments>

Production in India: POST <https://api.in.cybersource.com/pts/v2/payments>

Required Fields for Authorizing a Payment with Legacy Token Using the REST API

clientReferenceInformation.code

paymentInformation.legacyToken.id

Include the ID of the legacy token you want to use to authorize a payment.

orderInformation.amountDetails.currency

orderInformation.amount.totalAmount

processingInformation.capture

processingInformation.commerceIndicator

Related information

- [API Field Reference for the REST API](#)

Example: Authorize a Payment with Legacy Token Using the REST API

Request

```

POST https://apitest.cybersource.com/pts/v2/payments
{
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "paymentInformation": {
    "legacyToken": {
      "id": "70300000000014911515"
    }
  },
  "orderInformation": {
    "amountDetails": {

```



```

    "currency": "USD",
    "totalAmount": "1.00"
  }
},
"processingInformation": {
  "capture": "false",
  "commerceIndicator": "internet"
}
}

```

Successful Response

```

{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6777870199906201003955/reversals"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6777870199906201003955"
    },
    "capture": {
      "method": "POST",
      "href": "/pts/v2/payments/6777870199906201003955/captures"
    }
  },
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "id": "6777870199906201003955",
  "orderInformation": {
    "amountDetails": {
      "authorizedAmount": "1.00",
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
  "pointOfSaleInformation": {
    "terminalId": "111111"
  },
  "processorInformation": {
    "paymentAccountReferenceNumber": "V0010013019326121174070050420",
    "approvalCode": "888888",

```

```

    "networkTransactionId": "123456789619999",
    "transactionId": "123456789619999",
    "responseCode": "100",
    "avs": {
      "code": "1"
    }
  },
  "reconciliationId": "61886994VE00CC0K",
  "status": "AUTHORIZED",
  "submitTimeUtc": "2023-03-02T19:57:00Z"
}

```

Make a Credit with a Customer Token

This section shows you how to make a credit with a customer token.

Endpoint

Test: POST <https://apitest.cybersource.com/pts/v2/credits>

Production: POST <https://api.cybersource.com/pts/v2/credits>

Production in India: POST <https://api.in.cybersource.com/pts/v2/credits>

Required Fields for Making a Credit with a Customer Token Using the REST API

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

orderInformation.billTo.address1

orderInformation.billTo.administrativeArea

orderInformation.billTo.country

orderInformation.billTo.email

orderInformation.billTo.firstName

orderInformation.billTo.lastName

orderInformation.billTo.locality

orderInformation.billTo.postalCode

paymentInformation.card.expirationMonth

paymentInformation.card.expirationYear

paymentInformation.card.number

paymentInformation.card.type

paymentInformation.customer.id

Set to the ID of the customer token you want to use.

Related information

- API Field Reference for the REST API

Example: Make a Credit with a Customer Token Using the REST API

Request

POST https://apitest.cybersource.com/pts/v2/credits

```
{
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "paymentInformation": {
    "customer": {
      "id": "F45FB3E443AC3C57E053A2598D0A9CFF"
    }
  },
  "orderInformation": {
    "amountDetails": {
      "currency": "USD",
      "totalAmount": "1.00"
    }
  },
  "processingInformation": {
    "capture": "false",
    "commerceIndicator": "internet"
  }
}
```

Successful Response

```
{
  "_links": {
    "void": {
      "method": "POST",
      "href": "/pts/v2/credits/6760630831286132403955/voids"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/credits/6760630831286132403955"
    }
  },
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "creditAmountDetails": {
    "currency": "USD",
    "creditAmount": "1.00"
  },
  "id": "6760630831286132403955",
  "orderInformation": {
    "amountDetails": {
      "currency": "USD"
    }
  }
}
```

```

    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "instrumentIdentifier": {
      "id": "70300000000014911515",
      "state": "ACTIVE"
    },
    "shippingAddress": {
      "id": "F45FB3E443AF3C57E053A2598D0A9CFF"
    },
    "paymentInstrument": {
      "id": "F45FC6785E3C31A2E053A2598D0A5346"
    },
    "card": {
      "type": "001"
    },
    "customer": {
      "id": "F45FB3E443AC3C57E053A2598D0A9CFF"
    }
  },
  "processorInformation": {
    "paymentAccountReferenceNumber": "V0010013019326121174070050420",
    "approvalCode": "888888",
    "responseCode": "100"
  },
  "reconciliationId": "69608060KDTHKXAL",
  "status": "PENDING",
  "submitTimeUtc": "2023-02-10T21:04:43Z"
}

```

Make a Credit with Non-Default Payment Instrument

This section shows you how to make a credit with non-default payment instrument.

Endpoint

Test: POST <https://apitest.cybersource.com/pts/v2/credits>

Production: POST <https://api.cybersource.com/pts/v2/credits>

Production in India: POST <https://api.in.cybersource.com/pts/v2/credits>

Required Fields for Making a Credit with Non-Default Payment Instrument Using the REST API

`paymentInformation.paymentInstrument.id` Set to the ID of the payment instrument token you want to use.

Related information

- API Field Reference for the REST API

Optional Fields for Making a Credit with Non-Default Payment Instrument Using the REST API

`orderInformation.amountDetails.currency`
`orderInformation.amountDetails.totalAmount`
`orderInformation.billTo.address1`
`orderInformation.billTo.administrativeArea`
`orderInformation.billTo.country`
`orderInformation.billTo.email`
`orderInformation.billTo.firstName`
`orderInformation.billTo.lastName`
`orderInformation.billTo.locality`
`orderInformation.billTo.postalCode`
`paymentInformation.card.expirationMonth`
`paymentInformation.card.expirationYear`
`paymentInformation.card.number`
`paymentInformation.card.type`

Related information

- API Field Reference for the REST API

Example: Make a Credit with Non-Default Payment Instrument Using the REST API

Request

POST <https://apitest.cybersource.com/pts/v2/credits>

```
{
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  }
}
```

```

},
},
"customer": {
  "id": "F45FB3E443AC3C57E053A2598D0A9CFF"
},
"paymentInstrument": {
  "id": "F4D5E715F7BD9910E053A2598D0A7278"
},
},
"orderInformation": {
  "amountDetails": {
    "currency": "USD",
    "totalAmount": "1.00"
  }
},
},
"processingInformation": {
  "capture": "false",
  "commerceIndicator": "internet"
}
}

```

Successful Response

```

{
  "_links": {
    "void": {
      "method": "POST",
      "href": "/pts/v2/credits/6765711323826105903955/voids"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/credits/6765711323826105903955"
    }
  },
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "creditAmountDetails": {
    "currency": "USD",
    "creditAmount": "1.00"
  },
  "id": "6765711323826105903955",
  "orderInformation": {
    "amountDetails": {
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    }
  },
}

```

```

    "instrumentIdentifier": {
      "id": "70100000000016241111",
      "state": "ACTIVE"
    },
    "paymentInstrument": {
      "id": "F4D5E715F7BD9910E053A2598D0A7278"
    },
    "card": {
      "type": "001"
    },
    "customer": {
      "id": "F4D5E715F75E9910E053A2598D0A7278"
    }
  },
  "processorInformation": {
    "approvalCode": "888888",
    "responseCode": "100"
  },
  "reconciliationId": "60422331QE3AHBZX",
  "status": "PENDING",
  "submitTimeUtc": "2023-02-16T18:12:12Z"
}

```

Make a Credit with a Payment Instrument

This section shows you how to make a credit with payment instrument.

Endpoint

Test: POST <https://apitest.cybersource.com/pts/v2/credits>

Production: POST <https://api.cybersource.com/pts/v2/credits>

Production in India: POST <https://api.in.cybersource.com/pts/v2/credits>

Required Fields for Making a Credit with a Payment Instrument Using the REST API

orderInformation.amountDetails.currency

orderInformation.amountDetails.totalAmount

orderInformation.billTo.address1

orderInformation.billTo.administrativeArea

orderInformation.billTo.country

orderInformation.billTo.email

orderInformation.billTo.firstName

orderInformation.billTo.lastName

orderInformation.billTo.locality

orderInformation.billTo.postalCode**paymentInformation.card.expirationMonth****paymentInformation.card.expirationYear****paymentInformation.card.number****paymentInformation.card.type****paymentInformation.paymentInstrument.id** Set to the ID of the payment instrument token you want to use.**Related information**

- [API Field Reference for the REST API](#)

REST Example: Making a Credit with a Payment Instrument

Request

POST https://apitest.cybersource.com/pts/v2/credits

```
{
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "paymentInstrument": {
    "id": "F4D5E715F7BD9910E053A2598D0A7278"
  },
  "orderInformation": {
    "amountDetails": {
      "currency": "USD",
      "totalAmount": "1.00"
    }
  },
  "processingInformation": {
    "capture": "false",
    "commerceIndicator": "internet"
  }
}
```

Successful Response

```
{
  "_links": {
    "void": {
      "method": "POST",
      "href": "/pts/v2/credits/6765711323826105903955/voids"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/credits/6765711323826105903955"
    }
  }
}
```



```

},
"clientReferenceInformation": {
  "code": "RTS-Auth"
},
"creditAmountDetails": {
  "currency": "USD",
  "creditAmount": "1.00"
},
"id": "6765711323826105903955",
"orderInformation": {
  "amountDetails": {
    "currency": "USD"
  }
},
"paymentAccountInformation": {
  "card": {
    "type": "001"
  }
},
"paymentInformation": {
  "tokenizedCard": {
    "type": "001"
  },
  "instrumentIdentifier": {
    "id": "70100000000016241111",
    "state": "ACTIVE"
  },
  "paymentInstrument": {
    "id": "F4D5E715F7BD9910E053A2598D0A7278"
  },
  "card": {
    "type": "001"
  },
  "customer": {
    "id": "F4D5E715F75E9910E053A2598D0A7278"
  }
},
"processorInformation": {
  "approvalCode": "888888",
  "responseCode": "100"
},
"reconciliationId": "60422331QE3AHBZX",
"status": "PENDING",
"submitTimeUtc": "2023-02-16T18:12:12Z"
}

```

Make a Credit with an Instrument Identifier

This section shows you how to make a credit with an instrument identifier token.

Endpoint

Test: **POST** <https://apitest.cybersource.com/pts/v2/credits>

Production: POST <https://api.cybersource.com/pts/v2/credits>

Production in India: POST <https://api.in.cybersource.com/pts/v2/credits>

Required Fields for Making a Credit with an Instrument Identifier Using the REST API

`orderInformation.amountDetails.currency`

`orderInformation.amountDetails.totalAmount`

`orderInformation.billTo.address1`

`orderInformation.billTo.administrativeArea`

`orderInformation.billTo.country`

`orderInformation.billTo.email`

`orderInformation.billTo.firstName`

`orderInformation.billTo.lastName`

`orderInformation.billTo.locality`

`orderInformation.billTo.postalCode`

`paymentInformation.card.expirationMonth`

`paymentInformation.card.expirationYear`

`paymentInformation.card.number`

`paymentInformation.card.type`

`paymentInformation.instrumentIdentifier.id` Set to the ID of the instrument identifier token you want to use.

Related information

- [API Field Reference for the REST API](#)

Example: Make a Credit with an Instrument Identifier Using the REST API

Request

POST <https://apitest.cybersource.com/pts/v2/credits>

```
{
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "paymentInformation": {
    "card": {
      "expirationYear": "2031",
      "expirationMonth": "12",
      "type": "001"
    }
  }
}
```

```

    },
    "instrumentIdentifier": {
      "id": "70100000000016241111"
    }
  },
  "orderInformation": {
    "amountDetails": {
      "currency": "USD",
      "totalAmount": "1.00"
    }
  },
  "processingInformation": {
    "capture": "false",
    "commerceIndicator": "internet"
  }
}

```

Successful Response

```

{
  "_links": {
    "void": {
      "method": "POST",
      "href": "/pts/v2/credits/6760635718126158603955/voids"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/credits/6760635718126158603955"
    }
  },
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "creditAmountDetails": {
    "currency": "USD",
    "creditAmount": "1.00"
  },
  "id": "6760635718126158603955",
  "orderInformation": {
    "amountDetails": {
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "instrumentIdentifier": {
      "id": "70100000000016241111",
      "state": "ACTIVE"
    }
  },
}

```

```

    "card": {
      "type": "001"
    },
    "processorInformation": {
      "paymentAccountReferenceNumber": "V0010013019326121174070050420",
      "approvalCode": "888888",
      "responseCode": "100"
    },
    "reconciliationId": "69605700EDTK9EXS",
    "status": "PENDING",
    "submitTimeUtc": "2023-02-10T21:12:52Z"
  }
}

```

Make a Credit with Legacy Token

This section shows you how to make a credit with a legacy token.

Endpoint

Test: POST <https://apitest.cybersource.com/pts/v2/credits>

Production: POST <https://api.cybersource.com/pts/v2/credits>

Production in India: POST <https://api.in.cybersource.com/pts/v2/credits>

Required Fields for Making a Credit with Legacy Token Using the REST API

clientReferenceInformation.code

paymentInformation.legacyToken.id

Include the ID of the legacy token you want to use to authorize a payment.

orderInformation.amountDetails.currency

orderInformation.amount.totalAmount

processingInformation.capture

processingInformation.commerceIndicator

Related information

- [API Field Reference for the REST API](#)

Example: Make a Credit with Legacy Token Using the REST API

Request

```

{
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },

```

```

"paymentInformation": {
  "legacyToken": {
    "id": "7030000000014911515"
  }
},
"orderInformation": {
  "amountDetails": {
    "currency": "USD",
    "totalAmount": "1.00"
  }
},
"processingInformation": {
  "capture": "false",
  "commerceIndicator": "internet"
}
}

```

Successful Response

```

{
  "_links": {
    "void": {
      "method": "POST",
      "href": "/pts/v2/credits/6777870847736270903954/voids"
    },
    "self": {
      "method": "GET",
      "href": "/pts/v2/credits/6777870847736270903954"
    }
  },
  "clientReferenceInformation": {
    "code": "RTS-Auth"
  },
  "creditAmountDetails": {
    "currency": "USD",
    "creditAmount": "1.00"
  },
  "id": "6777870847736270903954",
  "orderInformation": {
    "amountDetails": {
      "currency": "USD"
    }
  },
  "paymentAccountInformation": {
    "card": {
      "type": "001"
    }
  },
  "paymentInformation": {
    "tokenizedCard": {
      "type": "001"
    },
    "card": {
      "type": "001"
    }
  },
}

```

```
"processorInformation": {  
  "paymentAccountReferenceNumber": "V0010013019326121174070050420",  
  "approvalCode": "888888",  
  "responseCode": "100"  
},  
"reconciliationId": "61854023HEO0IGGE",  
"status": "PENDING",  
"submitTimeUtc": "2023-03-02T19:58:05Z"  
}
```

Debit and Prepaid Card Processing

A Debit cards is linked to a cardholder's checking account. A merchant who accepts the debit card can deduct funds directly from the linked cardholder's account.

You can process debit cards using these services:

- Credit card services
- PIN debit services
- Partial authorizations, which are a special feature available for debit cards
- Balance inquiries, which are a special feature available for debit cards

In Canada, to process domestic debit transactions on Visa Platform Connect with Mastercard, you must contact customer support to have your account configured for this feature.

Processing Debit and Prepaid Authorizations

This section provides the minimum set of information required to perform a successful authorization using debit and prepaid cards.

Endpoint

POST <https://api.cybersource.com/pts/v2/payments>

Required Fields for Debit and Prepaid Authorizations Using REST APIs

clientReferenceInformation.code
orderInformation.amountDetails.currency
orderInformation.amountDetails.totalAmount
orderInformation.billTo.address1

orderInformation.billTo.administrativeArea
orderInformation.billTo.country
orderInformation.billTo.email
orderInformation.billTo.firstName
orderInformation.billTo.lastName
orderInformation.billTo.locality
orderInformation.billTo.postalCode
paymentInformation.card.expirationMonth
paymentInformation.card.expirationYear
paymentInformation.card.number
paymentInformation.card.type

Country Specific Required Fields for Processing Basic Authorizations Using REST APIs

Argentina

merchantInformation.taxId	Required for Mastercard transactions.
merchantInformation.transactionLocalDateTime	Required in Argentina when the time zone is not included in your account. Otherwise, this field is optional.

Brazil

paymentInformation.card.sourceAccountType	Required for combo card transactions.
paymentInformation.card.sourceAccountType	Required for combo card line-of-credit and prepaid-card transactions.

Chile

merchantInformation.taxId	Required for Mastercard transactions.
----------------------------------	---------------------------------------

Paraguay

merchantInformation.taxId	Required for Mastercard transactions.
----------------------------------	---------------------------------------

Saudi Arabia

processingInformation.authorizationOptions.transactionMode

Taiwan

paymentInformation.card.hashedException

Optional Field for Debit and Prepaid Authorizations Using REST APIs

processingInformation.linkId

Set this field to the request ID that was returned in the response message from the original authorization request.

REST Example: Debit and Prepaid Authorizations

Request

Endpoint: **POST** <https://api.cybersource.com/pts/v2/payments>

```
{
  "orderInformation": {
    "billTo": {
      "country": "US",
      "firstName": "John",
      "lastName": "Deo",
      "address1": "901 Metro Center Blvd",
      "postalCode": "40500",
      "locality": "Foster City",
      "administrativeArea": "CA",
      "email": "test@cybs.com"
    },
    "amountDetails": {
      "totalAmount": "100.00",
      "currency": "USD"
    }
  },
  "paymentInformation": {
    "card": {
      "expirationYear": "2031",
      "number": "4111111111111111",
      "securityCode": "123",
      "expirationMonth": "12",
      "type": "001"
    }
  }
}
```

Response

```
{
  "_links": {
    "authReversal": {
      "method": "POST",
      "href": "/pts/v2/payments/6595482584316313203494/reversals"
    },
    "self": {
      "method": "GET",

```

```

    "href" : "/pts/v2/payments/6595482584316313203494"
  },
  "capture" : {
    "method" : "POST",
    "href" : "/pts/v2/payments/6595482584316313203494/captures"
  }
},
"clientReferenceInformation" : {
  "code" : "RTS-Auth"
},
"consumerAuthenticationInformation" : {
  "token" : "Axj/7wSTZYq1MhJBMfMmAEQs2auWrRwyauGjNi2ZsWbJgzaOWiaVA
+JbKAU0qB8S2VpA6cQIp4ZNvG2YbC9eM4E5NlirUyEkEx8yYAAA4A1c"
},
"id" : "6595482584316313203494",
"orderInformation" : {
  "amountDetails" : {
    "authorizedAmount" : "100.00",
    "currency" : "USD"
  }
},
"paymentAccountInformation" : {
  "card" : {
    "type" : "001"
  }
},
"paymentInformation" : {
  "tokenizedCard" : {
    "type" : "001"
  },
  "card" : {
    "type" : "001"
  }
},
"processorInformation" : {
  "systemTraceAuditNumber" : "853428",
  "approvalCode" : "831000",
  "cardVerification" : {
    "resultCodeRaw" : "M",
    "resultCode" : "M"
  },
  "merchantAdvice" : {
    "code" : "01",
    "codeRaw" : "M001"
  },
  "responseDetails" : "ABC",
  "networkTransactionId" : "016153570198200",
  "retrievalReferenceNumber" : "221517853428",
  "consumerAuthenticationResponse" : {
    "code" : "2",
    "codeRaw" : "2"
  },
  "transactionId" : "016153570198200",
  "responseCode" : "00",
  "avs" : {
    "code" : "Y",

```

```

    "codeRaw" : "Y"
  }
}
}

```

Enabling Debit and Prepaid Partial Authorizations

Partial authorizations and balance responses are special features that are available for debit cards and prepaid cards. This section provides the minimum set of information required to enable partial authorizations for a specific transaction.

To globally process domestic debit transactions on Visa Platform Connect with Mastercard in Canada, you must contact customer support to have your account configured for this feature.

Fields Specific to this Use Case

Include the following fields in addition to the fields required for a standard authorization request:

- Indicate that this request is a partial authorization.
Set the `processingInformation.authorizationOptions.partialAuthIndicator` to `true`.

Required Field for Enabling Debit and Prepaid Partial Authorizations Using REST APIs

```

clientReferenceInformation.code
orderInformation.amountDetails.currency
orderInformation.amountDetails.totalAmount
orderInformation.billTo.address1
orderInformation.billTo.administrativeArea
orderInformation.billTo.country
orderInformation.billTo.email
orderInformation.billTo.firstName
orderInformation.billTo.lastName
orderInformation.billTo.locality
orderInformation.billTo.postalCode
paymentInformation.card.expirationMonth
paymentInformation.card.expirationYear
paymentInformation.card.number

```

paymentInformation.card.type

processingInformation.authorizationOptions.partialAuthIndicator: Set this field to **true**:

Optional Field for Enabling Debit and Prepaid Partial Authorizations Using REST APIs

processingInformation.linkId

Set this field to the request ID that was returned in the response message from the original authorization request.

Enabling Partial Authorizations

1. Enable partial authorizations for the transaction by setting the **processingInformation.authorizationOptions.partialAuthIndicator** field to **true** in a request.
2. Send the service request to this endpoint:

<https://api.cybersource.com/pts/v2/payments>

Example: Enabling Debit and Prepaid Partial Authorizations Using the REST API

Request

```
{
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "orderInformation": {
    "billTo": {
      "country": "US",
      "lastName": "Deo",
      "address2": "Address 2",
      "address1": "201 S. Division St.",
      "postalCode": "48104-2201",
      "locality": "Ann Arbor",
      "administrativeArea": "MI",
      "firstName": "John",
      "phoneNumber": "999999999",
      "district": "MI",
      "buildingNumber": "123",
      "company": "Visa",
      "email": "test@cybs.com"
    },
    "amountDetails": {
      "totalAmount": "1000.00",
      "currency": "USD"
    }
  },
  "paymentInformation": {
    "card": {
```

```

    "expirationYear": "2031",
    "number": "5555555555554444",
    "securityCode": "123",
    "expirationMonth": "12",
    "type": "002"
  }
},
"processingInformation": {
  "authorizationOptions": {
    "partialAuthIndicator": "true"
  }
}
}
}

```

Response

```

{
  "_links": {
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6595549144566655003494"
    }
  },
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "id": "6595549144566655003494",
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "1000.00",
      "authorizedAmount": "499.01",
      "currency": "USD"
    }
  },
  "paymentInformation": {
    "accountFeatures": {
      "currency": "usd",
      "balanceAmount": "0.00"
    }
  },
  "pointOfSaleInformation": {
    "terminalId": "261996"
  },
  "processorInformation": {
    "merchantNumber": "000000092345678",
    "approvalCode": "888888",
    "cardVerification": {
      "resultCode": ""
    }
  },
  "networkTransactionId": "123456789619999",
  "transactionId": "123456789619999",
  "responseCode": "100",
  "avs": {
    "code": "X",
    "codeRaw": "I1"
  }
}

```

```

},
"reconciliationId" : "56059417N6C86KTJ",
"status" : "PARTIAL_AUTHORIZED",
"submitTimeUtc" : "2022-08-03T19:28:34Z"
}

```

Disabling Debit and Prepaid Partial Authorizations

This topic shows the minimum set of information required to enable partial authorizations for specific transactions. To disable all authorization and capture requests for partial authorizations, call customer support.

Required Field for Disabling Debit and Prepaid Partial Authorizations Using REST APIs

`processingInformation.authorizationOptions.partialAuthIndicator` is set to `false` in an authorization or sale request. When you do so, only that specific transaction is disabled for partial authorization.

Optional Field for Disabling Debit and Prepaid Partial Authorizations Using REST APIs

`processingInformation.linkId`

Set this field to the request ID that was returned in the response message from the original authorization request.

Disabling Partial Authorizations

1. Enable partial authorizations for the transaction by setting the `processingInformation.authorizationOptions.partialAuthIndicator` field to `true` in a request.
2. Send the service request to this endpoint:
<https://api.cybersource.com/pts/v2/payments>

Example: Disabling Debit and Prepaid Partial Authorizations Using the REST API

Request

```

{
  "processingInformation":{
    "authorizationOptions":{

```

```

    "partialAuthIndicator": "false"
  }
},
"clientReferenceInformation": {
  "code": "TC50171_3"
},
"orderInformation": {
  "billTo": {
    "country": "US",
    "lastName": "Deo",
    "address2": "Address 2",
    "address1": "201 S. Division St.",
    "postalCode": "48104-2201",
    "locality": "Ann Arbor",
    "administrativeArea": "MI",
    "firstName": "John",
    "phoneNumber": "999999999",
    "district": "MI",
    "buildingNumber": "123",
    "company": "Visa",
    "email": "test@cybs.com"
  },
  "amountDetails": {
    "totalAmount": "501.00",
    "currency": "USD"
  }
},
"paymentInformation": {
  "card": {
    "expirationYear": "2031",
    "number": "5555555555554444",
    "securityCode": "123",
    "expirationMonth": "12",
    "type": "002"
  }
}
}

```

Response

```

{
  "_links": {
    "self": {
      "method": "GET",
      "href": "/pts/v2/payments/6595545423896900104953"
    }
  },
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "errorInformation": {
    "reason": "PROCESSOR_DECLINED",
    "message": "Decline - General decline of the card. No other information provided by the issuing bank."
  },
  "id": "6595545423896900104953",
  "pointOfSaleInformation": {

```

```
    "terminalId": "111111"
  },
  "processorInformation": {
    "networkTransactionId": "123456789619999",
    "transactionId": "123456789619999",
    "responseCode": "100",
    "avs": {
      "code": "X",
      "codeRaw": "I1"
    }
  },
  "status": "DECLINED"
}
```